Ontology

Relation Types

Concept Links

無題のファイル

Question → Traversal Mapping

Question → Traversal Mapping

Question → Traversal Mapping は
質問の種類に応じて Knowledge Graph の探索経路（Traversal）を決定する方法である。

Knowledge Graph は

```
Case
Pattern
Mechanism
Kernel
```

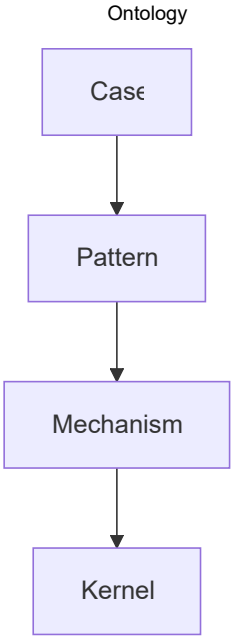
という階層を持つため、
質問の種類によって探索開始点や方向が変わる。

Question → Traversal Mapping の目的

この方法の目的

- 推論の効率化
- 誤った探索の防止
- Reasoning Strategy の選択

Knowledge Graph の探索階層



質問はこのどこからでも始まる。

Question の主な種類

Knowledge Graph では質問は主に5種類に分類できる。

質問タイプ	意味
What	現象の説明
Why	原因探索
How	メカニズム
Example	具体例
Analogy	類推

Question → Traversal Mapping

What Question

例

> これは何か？

Traversal

```
Case → Pattern
```

Why Question

例

```
なぜ起きるのか？
```

Traversal

```
Pattern → Mechanism
```

How Question

例

```
どうやって起きるのか？
```

Traversal

```
Mechanism
```

Example Question

例

```
具体例は？
```

Traversal

```
Pattern → Case
```

Analogy Question

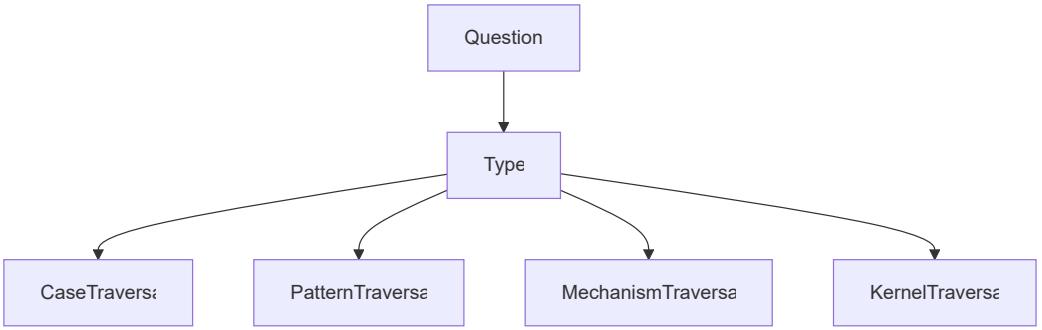
例

```
似た現象は？
```

Traversal

```
Pattern ⇔ Pattern
Mechanism ⇔ Mechanism
```

Mapping 図



Traversal の選択

質問の種類ごとに
探索開始点が変わる。

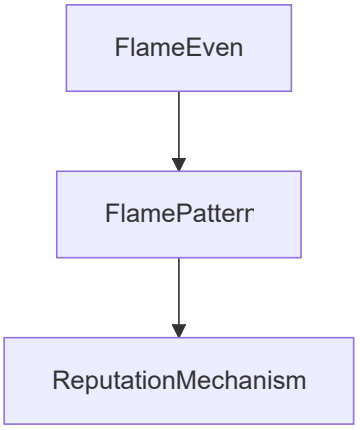
質問	開始ノード
何か	Case
なぜ	Pattern
どうやって	Mechanism
原理	Kernel

Traversal 例

質問

```
なぜ炎上が起きるのか
```

Traversal



複合質問

多くの質問は
複数の Traversal を含む。

例

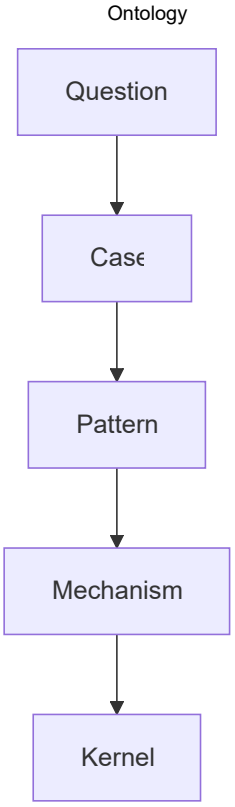
```
なぜ炎上が起きるのか？
具体例は？
```

Traversal

```
Pattern → Mechanism
Pattern → Case
```

Reasoning Path

Question → Traversal Mapping により
Reasoning Path が形成される。



Mapping の利点

この仕組みがあると

- 推論が高速になる
- 不必要な探索を減らせる
- LLM reasoning を安定させる

LLM と Question Mapping

LLM は質問から

```
question type
```

を推定し
Traversal を決定する。

Mapping の注意

質問の誤分類

誤った Traversal を選ぶ。

抽象度不一致

```
case → kernel
```

のジャンプが起きる。

---

pattern を飛ばす

推論が壊れる。

関連ノート

- Graph Traversal Rules
- 02_zettelkasten/04_knowledge_graph/Reasoning Strategy
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph Structure
- 02_zettelkasten/04_knowledge_graph/Pattern
- Mechanism

Knowledge Graph Structure

Knowledge Graph Structure

Knowledge Graph Structure は
Zettelkasten 全体をグラフとして構造化するための設計ルールである。

Knowledge Graph では、知識は
ノード（concept）とエッジ（relation）で構成される。

Knowledge Graph の目的

Knowledge Graph は次を目的とする。

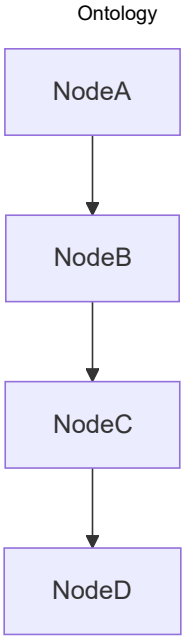
- 知識の関係を明確化
- 推論経路の生成
- LLM 推論の補助

Knowledge Graph の基本要素

Knowledge Graph は次の2要素で構成される。

要素	説明
Node	知識単位
Edge	関係

Knowledge Graph 基本図

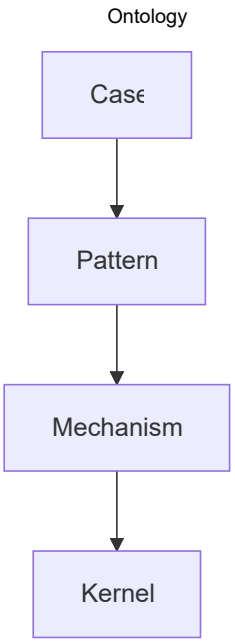


Node の種類

Knowledge Graph ではノードは階層構造を持つ。

```
Case
Pattern
Mechanism
Kernel
```

Node 階層



Case

Case は
具体的事例である。

例

```
フランス革命
企業炎上
市場崩壊
```

Pattern

Pattern は
複数 case に現れる構造である。

例

```
革命パターン
炎上パターン
権力争いパターン
```

Mechanism

Mechanism は
Pattern の原因となる因果プロセスである。

例

```
同調
競争
評判
```

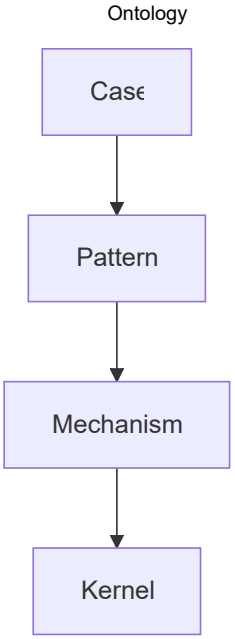
Kernel

Kernel は
最も抽象的な原理である。

例

```
社会性
競争
適応
```

Knowledge Graph 全体図

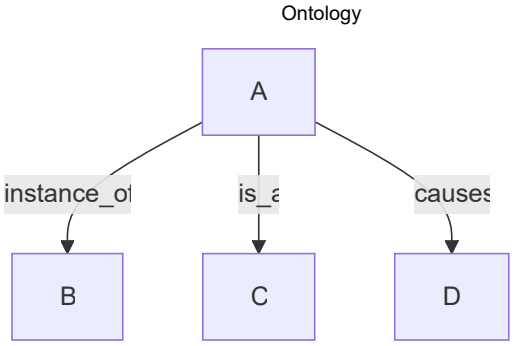


Edge の種類

Knowledge Graph では relation は複数存在する。

relation	意味
instance_of	具体例
is_a	分類
part_of	構成要素
causes	因果
related_to	関連

Edge 図



Knowledge Graph の特徴

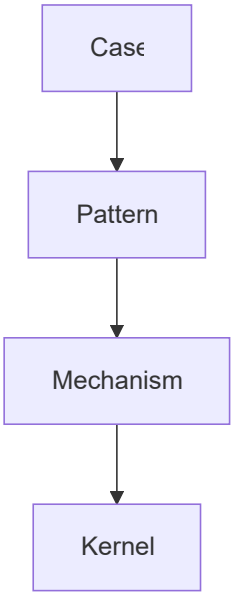
Knowledge Graph は

- 階層構造
- ネットワーク構造

の両方を持つ。

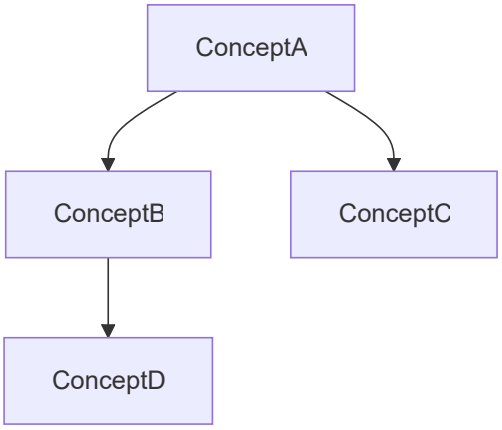
Hierarchy

抽象化構造



Network

知識関係



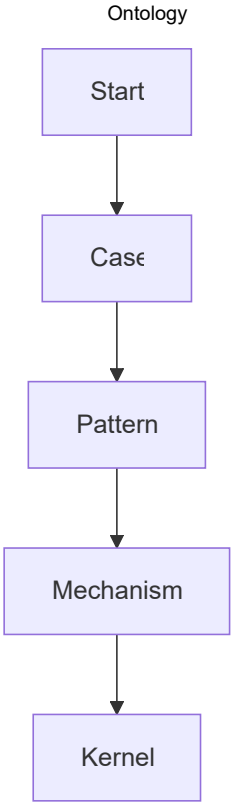
Knowledge Graph の探索

Knowledge Graph は
Traversal によって探索される。

例

```
Case → Pattern → Mechanism → Kernel
```

Traversal 図



Knowledge Graph と Zettelkasten

Zettelkasten は

> ノート集合

Knowledge Graph は

> ノート関係構造

である。

Knowledge Graph の利点

Knowledge Graph を使うと

- 知識検索が容易
- 推論が可能

- 構造理解が可能

になる。

Knowledge Graph と LLM

LLM は Knowledge Graph を使うことで

- reasoning path 生成
- analogy 推論
- causal 推論

を行いやすくなる。

関連ノート

- Graph Traversal Rules
- 02_zettelkasten/04_knowledge_graph/Pattern
- Mechanism
- Kernel
- 02_zettelkasten/04_knowledge_graph/Bridge Concept
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

Graph Traversal

Graph Traversal Rules

Graph Traversal Rules は
Knowledge Graph 上で推論や探索を行うための基本ルールである。

Knowledge Graph はノードの集合ではなく
関係ネットワークである。

そのため

> どの順序で辿るか

が推論の質を決める。

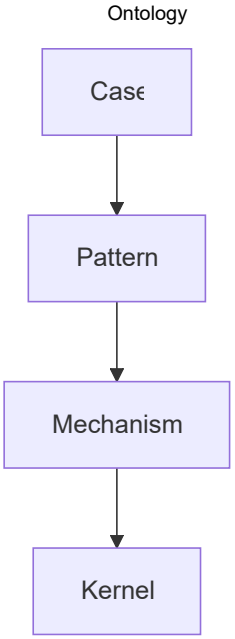
Graph Traversal の目的

Graph Traversal の目的

- 推論経路の生成
- 関係発見
- 抽象化
- 問題解決

基本 Traversal

Knowledge Graph の基本探索は



Traversal の種類

Knowledge Graph では
主に次の4種類の traversal を使う。

種類	目的
Abstraction Traversal	抽象化
Instantiation Traversal	具体化
Causal Traversal	原因探索
Analogy Traversal	類推

Abstraction Traversal

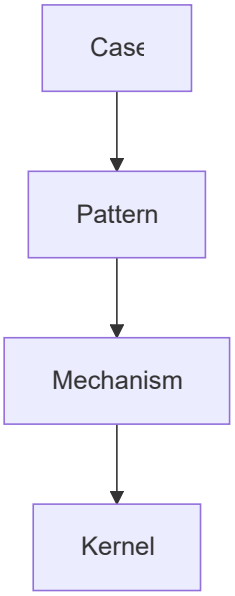
具体 → 抽象

```
Case → Pattern → Mechanism → Kernel
```

例

```
炎上事件
↓
炎上パターン
↓
評判メカニズム
↓
社会性原理
```

Abstraction Traversal 図



Instantiation Traversal

抽象 → 具体

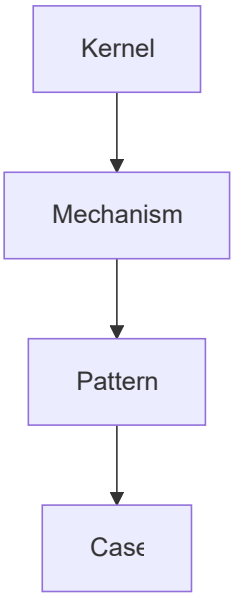
```
Kernel → Mechanism → Pattern → Case
```

例

```
社会性
↓
同調
```

```
↓
炎上
↓
SNS事件
```

Instantiation Traversal 図



Causal Traversal

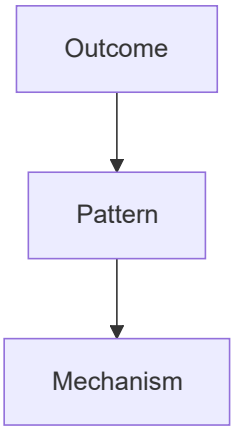
原因探索

```
Outcome → Cause
```

例

```
組織崩壊
↓
権力争い
↓
競争メカニズム
```

Causal Traversal 図



Analogy Traversal

類推探索

```
Pattern → Pattern
Mechanism → Mechanism
```

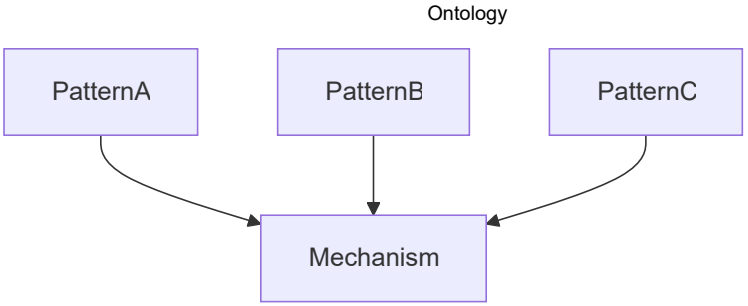
例

```
企業競争
政治競争
生物進化
```

共通

```
選択メカニズム
```

Analogy Traversal 図



Graph Traversal の基本ルール

Rule1

抽象度を意識する

```
case
pattern
mechanism
kernel
```

Rule2

relation を守る

主な relation

```
instance_of
is_a
causes
part_of
```

Rule3

探索目的を決める

探索には目的がある

例

```
原因探索
構造理解
予測
```

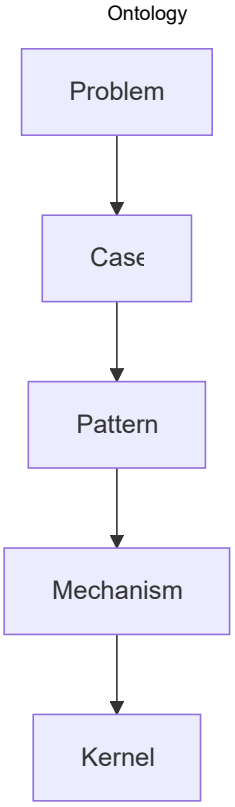
Graph Traversal Strategy

推論は通常次の順序で行う

```
問題
↓
case
↓
pattern
↓
mechanism
↓
kernel
```

Reasoning Path

Traversal は
Reasoning Path を形成する。



Traversal の失敗例

---

1 抽象ジャンプ

```
case → kernel
```

---

2 relation 無視

無関係ノードを辿る

---

3 pattern を飛ばす

```
case → mechanism
```

Traversal と LLM

Knowledge Graph があると
LLM は

- reasoning path を生成
- causal explanation を構築
- analogy 推論

が可能になる。

関連ノート

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph Structure
- 02_zettelkasten/04_knowledge_graph/Reasoning Strategy
- 02_zettelkasten/04_knowledge_graph/Question → Traversal Mapping
- 02_zettelkasten/04_knowledge_graph/Pattern
- Mechanism

Reasoning Strategy

Reasoning Strategy

Reasoning Strategy は、Knowledge Graph を用いて
どのような順序・方法で推論を行うかを定義する戦略ノートである。

Knowledge Graph には

- case
- pattern
- mechanism
- concept
- kernel

など多くのノードが存在する。

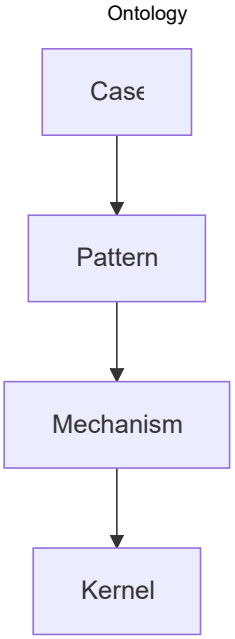
しかし **推論の進み方を決めないと** 次の問題が起きる。

- 思考が迷子になる
- 抽象に飛びすぎる
- case に閉じる
- 推論の再現性がなくなる

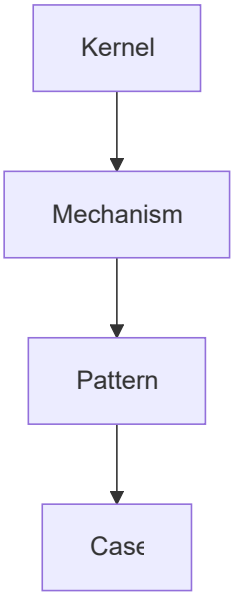
Reasoning Strategy は
Knowledge Graph の **思考ルート設計** である。

推論の基本構造

Knowledge Graph の推論は通常次の階層を上がる。



また逆方向の推論もある。



推論の主要戦略

Reasoning Strategy は大きく5種類ある。

---

1 Inductive Reasoning

具体 → 抽象

```
case
 ↓
pattern
 ↓
mechanism
```

目的

- 新しい pattern を発見
- mechanism 仮説を作る

---

2 Deductive Reasoning

抽象 → 具体

```
mechanism
 ↓
pattern
 ↓
case
```

目的

- 予測
- 応用

---

3 Analogical Reasoning

類比推論

```
domain A
 ↓
bridge concept
 ↓
domain B
```

目的

- cross domain insight

---

4 Comparative Reasoning

比較推論

```
case A
case B
case C
 ↓
共通構造
```

目的

- pattern 発見

---

5 Mechanistic Reasoning

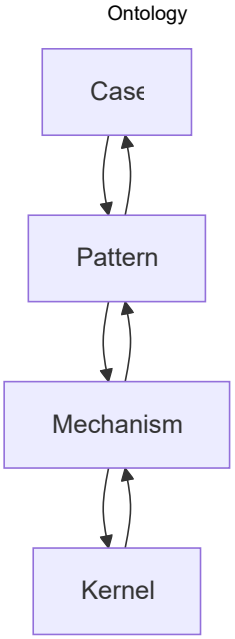
因果推論

```
結果
 ↓
原因
```

目的

- mechanism 推定

推論戦略の図



Reasoning Strategy の選択

推論は問題の種類によって変わる。

問題	戦略
未知構造	induction
予測	deduction
異分野理解	analogy
比較	comparison
原因分析	mechanistic

Reasoning Strategy の実行

推論は次の手順で行う。

Step1

問題を定義する。

Step2

関連 case を集める。

Step3

pattern を抽出する。

Step4

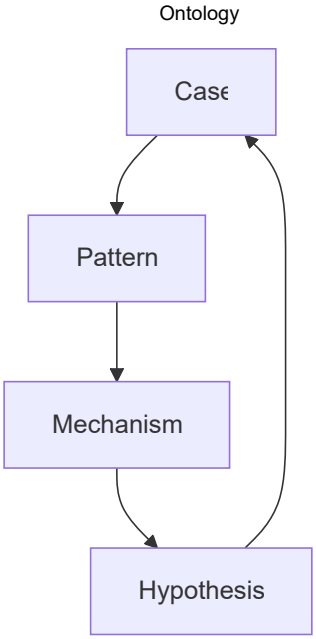
mechanism を推測する。

Step5

bridge concept を探す。

推論のループ

Knowledge Graph 推論は
通常ループする。



推論の注意

---

1 過度な抽象化

抽象だけでは現実を説明できない。

---

2 単一 case

case が少ないと誤推論になる。

---

3 類比の誤用

似ているだけで同じとは限らない。

Reasoning Strategy と Knowledge Graph

Knowledge Graph は
推論を支える構造である。

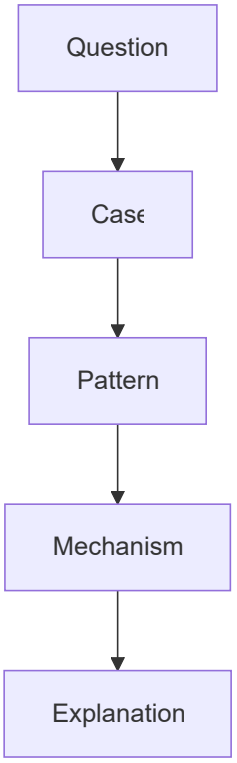
```
node
edge
traversal
```

Reasoning Strategy は

```
どの順で traversal するか
```

を決める。

Reasoning Strategy の図



LLM にとっての意味

Reasoning Strategy があると
LLM は

- 一貫した推論

- 再現可能な思考
- 知識の横断利用

ができる。

関連ノート

- 02_zettelkasten/04_knowledge_graph/Question → Traversal Mapping
- Graph Traversal Rules
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph Structure
- 02_zettelkasten/04_knowledge_graph/Pattern
- Mechanism
- 02_zettelkasten/04_knowledge_graph/Bridge Concept

Cross Domain Mapping

Cross Domain Mapping

Cross Domain Mapping は、Knowledge Graph において
異なる domain の構造を対応づける方法である。

Bridge Concept が **橋（bridge）** だとすると、
Cross Domain Mapping は **対応表（mapping）** である。

この方法を使うと

- 知識の再利用
- 新しい仮説の発見
- pattern の一般化

が可能になる。

定義

Cross Domain Mapping とは

ある domain の構造を
別の domain の構造に対応づける操作

である。

Cross Domain Mapping の役割

---

1 構造理解

複雑な現象を
別の分野の構造で理解できる。

---

2 仮説生成

未知の領域の
仮説を作ることができる。

---

3 Pattern 転用

ある domain の pattern を
別 domain に応用できる。

---

4 Mechanism 発見

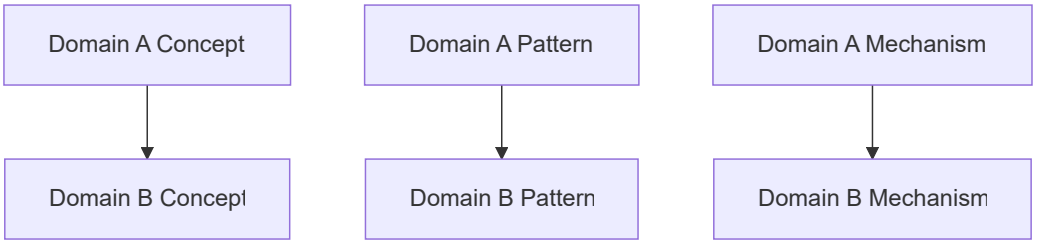
同じ因果が
複数領域に存在することを発見する。

Cross Domain Mapping の基本構造

Cross Domain Mapping は
次の対応関係を作る。

```
domain A structure
↓
domain B structure
```

Mapping 図



Mapping の対象

Mapping は次を対応させる。

要素	説明
concept	概念
pattern	進行構造
mechanism	因果構造
actor	主体
resource	資源

Mapping 手順

Step1

domain A の構造を整理する。

Step2

domain B の構造を整理する。

Step3

対応関係を探す。

Step4

Bridge Concept を見つける。

Step5

Mapping ノートを作る。

Mapping 例（抽象）

例

Domain A

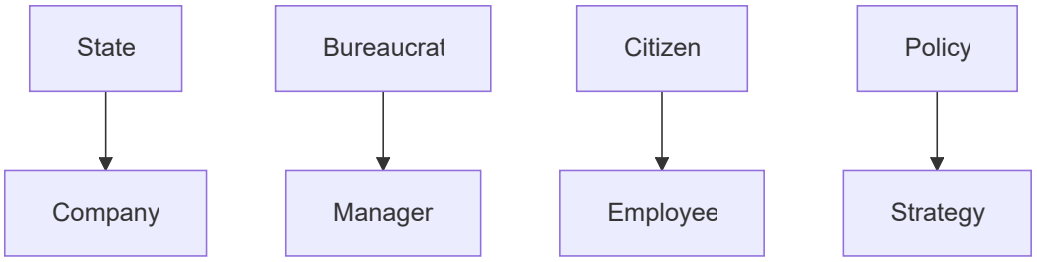
政治

Domain B

企業

政治	企業
国家	企業
官僚	管理職
国民	社員
政策	戦略

Mapping 図



Pattern Mapping

pattern も mapping できる。

例

```
政治権力争い
```

```
組織内派閥争い
```

共通 pattern

```
権力争い
```

Mechanism Mapping

mechanism も mapping できる。

例

```
市場競争
進化
```

共通 mechanism

```
選択
```

Cross Domain Mapping の注意

---

1 表面類似

似ているだけで
同じ構造とは限らない。

---

2 domain 特殊性

領域固有の要素を無視しない。

---

3 過度な一般化

抽象化しすぎると意味がなくなる。

Mapping の粒度

Mapping は3段階ある。

Concept Mapping

概念対応

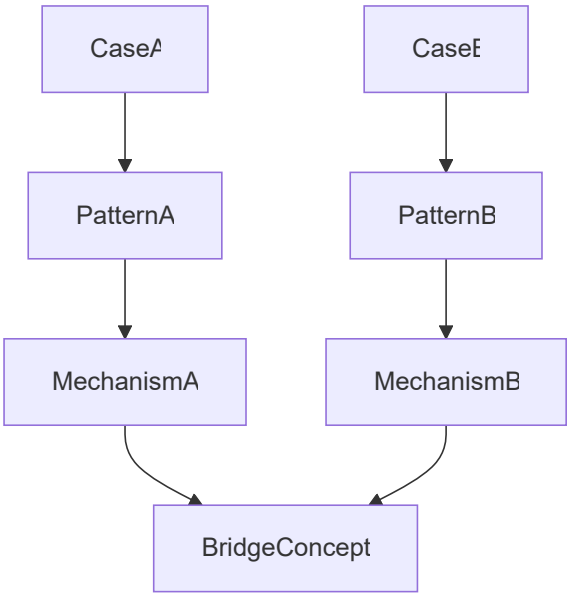
Pattern Mapping

進行構造対応

Mechanism Mapping

因果対応

Cross Domain Mapping の図



Cross Domain Mapping と Knowledge Graph

Knowledge Graph の中では

```
domain
↓
pattern
↓
mechanism
```

を横断して mapping を作る。

LLM にとっての意味

Cross Domain Mapping があると
LLM は

- analogy reasoning
- 新しい仮説
- pattern 転用

を行いやすくなる。

関連ノート

- 02_zettelkasten/04_knowledge_graph/Bridge Concept
- 02_zettelkasten/04_knowledge_graph/Bridge Detection Method
- Pattern Comparison 1
- 02_zettelkasten/04_knowledge_graph/Mechanism Comparison
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

Knowledge Graph

Question → Traversal Mapping

Question → Traversal Mapping は、Knowledge Graph において
質問の種類に応じてどのノードを辿るべきかを定義する構造である。

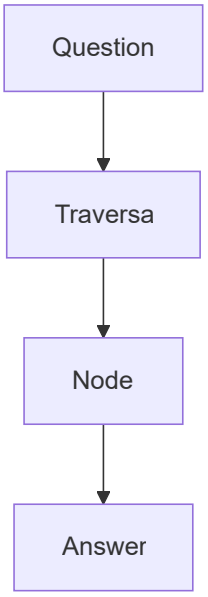
Knowledge Graph はノードとエッジで構成されているが、
質問が来たときに **どの順で探索するか** を決めないと

- 無関係なノードを辿る
- 抽象に飛びすぎる
- case ばかり参照する
- 推論が一貫しない

といった問題が起きる。

Question → Traversal Mapping は
質問 → 推論ルート を対応づける。

基本構造



質問タイプ

Knowledge Graph では
質問は大きく5種類に分類できる。

質問タイプ	目的
definition	概念理解
causal	原因理解
pattern	構造理解
comparison	差異理解
prediction	未来予測

Traversal の基本階層

Knowledge Graph は通常次の層を持つ。

```
case
pattern
mechanism
concept
kernel
```

Traversal は
この階層を上下する。

Definition Question

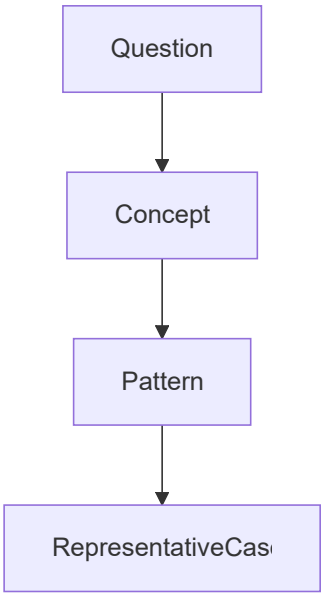
例

```
○○とは何か
```

Traversal

```
concept
 ↓
pattern
 ↓
representative case
```

図



Causal Question

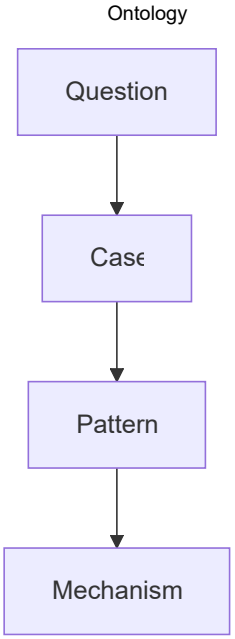
例

> なぜ○○が起きたのか

Traversal

```
case
 ↓
pattern
 ↓
mechanism
```

図



Pattern Question

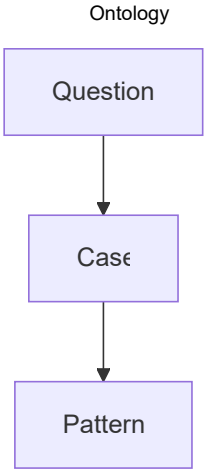
例

```
○○はどんな構造か
```

Traversal

```
case
 ↓
pattern
```

図



Comparison Question

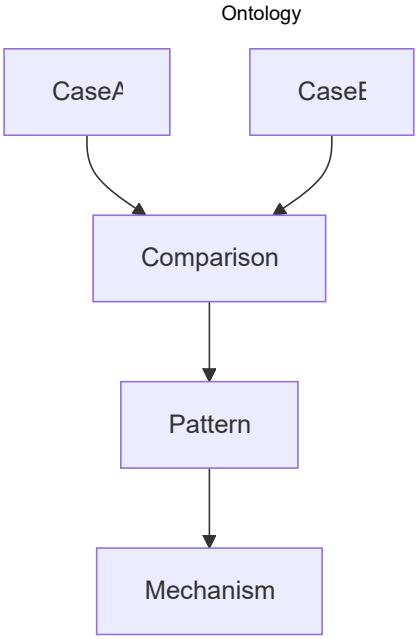
例

```
AとBの違い
```

Traversal

```
case A
case B
 ↓
pattern
 ↓
mechanism
```

図



Prediction Question

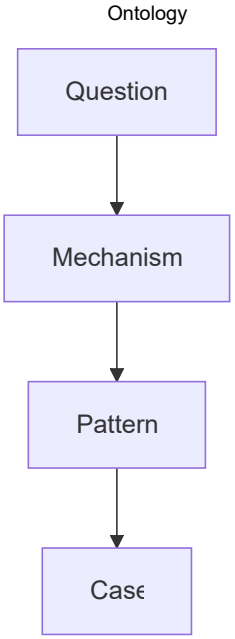
例

> 今後どうなるか

Traversal

```
mechanism
 ↓
pattern
 ↓
case
```

図



Cross Domain Question

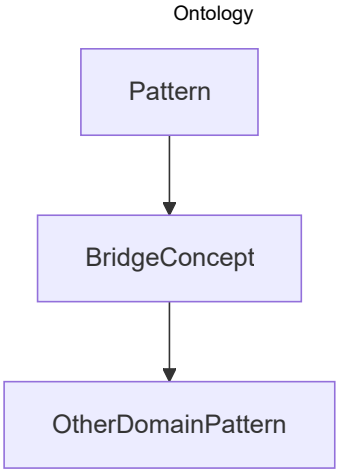
例

> ○○は他の分野では何に似ているか

Traversal

```
pattern
 ↓
bridge concept
 ↓
other domain pattern
```

図



Traversal のルール

Traversal は次を守る。

Rule1

まず **case から始める**

Rule2

次に **pattern**

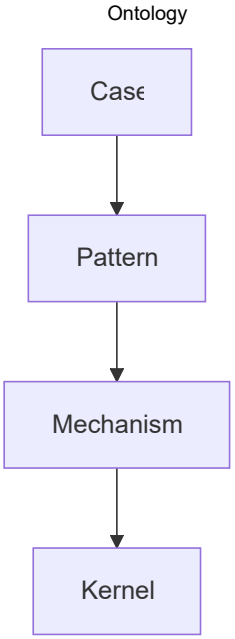
Rule3

その後 **mechanism**

Rule4

最後に **kernel**

Traversal 図



Question → Traversal の表

質問	Traversal
定義	concept → pattern
原因	case → pattern → mechanism
構造	case → pattern
比較	case → pattern → mechanism
予測	mechanism → pattern → case

Knowledge Graph との関係

Knowledge Graph は
ノードの集合である。

Question → Traversal Mapping は

> どの順でノードを辿るか

を決める。

LLM にとっての意味

Question → Traversal Mapping があると

LLM は

- 推論の順序を守る
- 関連ノードを探索する
- 思考の再現性が高まる

関連ノート

- 02_zettelkasten/04_knowledge_graph/Reasoning Strategy
- Graph Traversal Rules
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph Structure
- 02_zettelkasten/04_knowledge_graph/Pattern
- Mechanism
- 02_zettelkasten/04_knowledge_graph/Bridge Concept

Pattern

Pattern

Pattern は、Knowledge Graph において
複数の case に繰り返し現れる進行構造（recurring structure）を表すノードである。

Pattern は

- case の観察
- case の比較

から抽出される。

Pattern は単なる概念ではなく、
時間を伴う構造である。

Pattern の定義

Pattern とは

複数の case に繰り返し現れる
行動・相互作用・結果の進行構造

である。

Pattern の役割

Pattern は Knowledge Graph で次の役割を持つ。

---

1 Case の抽象化

```
case
case
case
 ↓
pattern
```

---

2 Mechanism の入口

Pattern は mechanism の表れである。

```
pattern
 ↓
mechanism
```

---

3 構造理解

複雑な出来事を構造として理解できる。

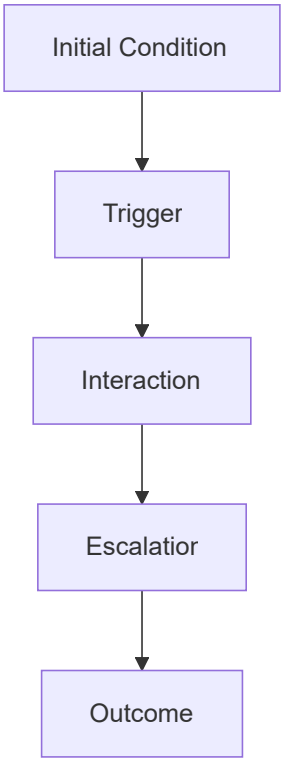
---

4 予測

Pattern が分かると
将来の展開を予測できる。

Pattern の基本構造

Pattern は通常
次の構造を持つ。



Pattern の構成要素

Pattern は次の要素から構成される。

要素	説明
condition	初期状態
trigger	きっかけ
actor	主体
interaction	相互作用
outcome	結果

Pattern の例（抽象）

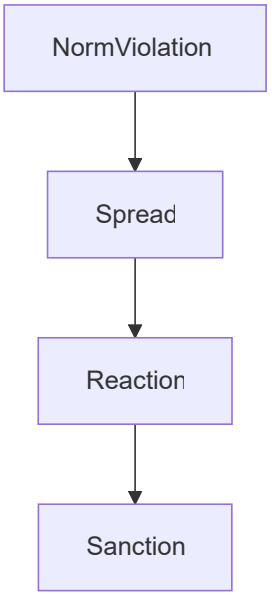
例

```
規範逸脱炎上
```

構造

```
規範逸脱
  ↓
SNS拡散
  ↓
集団反応
  ↓
評判制裁
```

Pattern の図



Pattern の種類

Pattern はいくつかの種類に分けられる。

Social Pattern

社会行動のパターン

例

- 炎上
- 集団同調
- 排除

Organizational Pattern

組織内のパターン

例

- 権力争い
- 官僚化

Economic Pattern

市場のパターン

例

- バブル
- 競争

Psychological Pattern

心理行動のパターン

例

- 認知バイアス
- 習慣形成

Pattern と Case の違い

項目	Pattern	Case
抽象度	高	低
再現性	ある	一回
例	炎上パターン	具体炎上事件

Pattern と Mechanism の違い

項目	Pattern	Mechanism
意味	進行構造	因果構造

Pattern の作り方

Pattern は次の流れで作る。

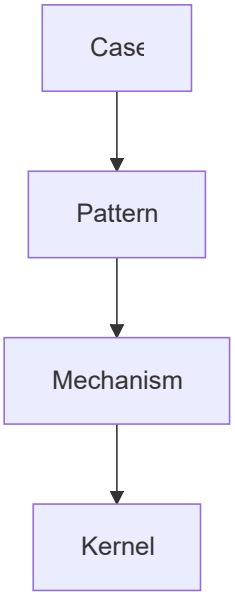
```
case
 ↓
case comparison
 ↓
pattern
```

Pattern ノートの構造

Pattern ノートには次を書く。

```
概要
構造
actor
trigger
interaction
outcome
representative case
mechanism
```

Pattern の図



Pattern の条件

良い pattern は次を満たす。

- 複数 case に現れる
- 構造が明確
- actor が存在
- mechanism と接続できる

Pattern の注意

1 Outcome を pattern にしない

例

```
崩壊
```

これは結果。

---

2 Case を pattern にしない

例

```
〇〇事件
```

---

3 Mechanism と混同しない

例

```
同調
```

mechanism の可能性。

Pattern と Knowledge Graph

Knowledge Graph では

```
case
 ↓
pattern
 ↓
mechanism
 ↓
kernel
```

という階層になる。

LLM にとっての意味

Pattern があると
LLM は

- case を抽象化

- 新しい事例の理解
- 予測

を行いやすくなる。

関連ノート

- old_zettelkasten/pattern/Pattern Hub
- Pattern Extraction Method 1
- Pattern Comparison 1
- 02_zettelkasten/04_knowledge_graph/Representative Case Rule
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

Bridge Detection Method

Bridge Detection Method

Bridge Detection Method は、Knowledge Graph において
異なる domain の知識を接続する Bridge Concept を発見する方法である。

Bridge Concept は Knowledge Graph の横方向の構造を作るが、
自然に見つかるとは限らない。

多くの場合、Bridge Concept は

- case 比較
- pattern 比較
- mechanism 比較

から発見される。

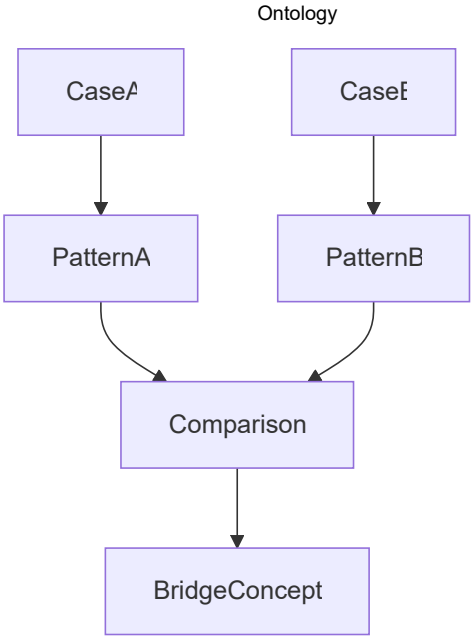
Bridge Detection の目的

Bridge Detection は次を目的とする。

- cross domain insight を得る
- 知識の再利用性を高める
- pattern の一般化
- mechanism の普遍化

Bridge Detection の基本構造

Bridge Concept は次の流れで発見される。



Bridge Detection の主要手法

Bridge Concept の発見には
次の4手法がある。

---

1 Case Comparison

異なる domain の case を比較する。

例

```
企業炎上
政治スキャンダル
SNSキャンセル
```

共通構造

```
規範逸脱
 ↓
集団反応
 ↓
評判制裁
```

Bridge Concept

```
評判
```

---

2 Pattern Comparison

似た pattern を比較する。

例

```
権力争い
組織内派閥争い
コミュニティ内対立
```

共通概念

```
権力
```

---

3 Mechanism Comparison

同じ因果メカニズムが
異なる領域に存在する。

例

```
動物行動
市場
就職
```

共通 mechanism

```
シグナリング
```

Bridge Concept

```
信号
```

---

4 Analogy Detection

類似構造を探す。

例

```
進化
市場競争
```

共通概念

```
選択
```

Bridge Detection 手順

Step1

複数 domain の case を集める。

Step2

pattern を抽出する。

Step3

共通構造を比較する。

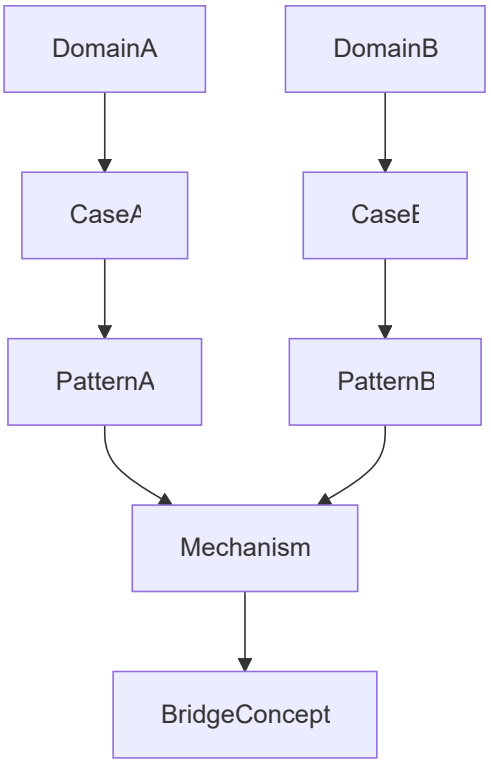
Step4

共通概念を抽象化する。

Step5

Bridge Concept として登録する。

Bridge Detection の図



Bridge Detection のヒント

Bridge Concept は次の場所で見つかる。

異なる domain に同じ構造がある

例

```
国家
企業
コミュニティ
```

同じ問題が現れる

例

```
信頼問題
```

同じ因果関係がある

例

```
評判 → 協力
```

Bridge Detection の注意

Bridge Detection は次に注意する。

---

1 表面類似

似ているだけで
同じ構造とは限らない。

---

2 抽象過剰

Bridge Concept が曖昧になる。

---

3 domain 特殊性

領域ごとの差を無視しない。

Bridge Detection の例

例（抽象）

```
動物行動
市場
```

```
就職
```

共通構造

```
信号
```

Bridge Concept

```
シグナリング
```

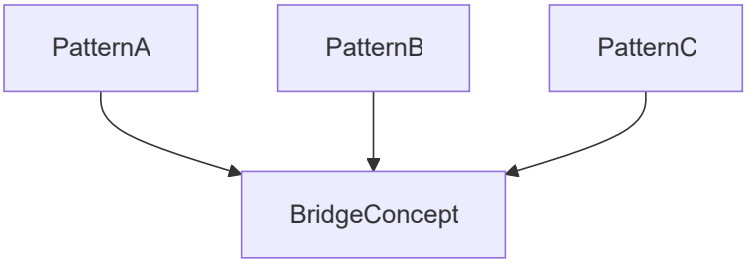
Bridge Detection と Knowledge Graph

Bridge Detection は

```
pattern layer
mechanism layer
```

を横断して
Bridge Concept を作る。

Bridge Detection の図



LLM にとっての意味

Bridge Detection があると
LLM は

- cross domain analogy
- 新しい仮説

- pattern 転用

を発見しやすくなる。

関連ノート

- 02_zettelkasten/04_knowledge_graph/Bridge Concept
- 02_zettelkasten/04_knowledge_graph/Bridge Concept Rule
- 02_zettelkasten/04_knowledge_graph/Cross Domain Mapping
- Pattern Comparison 1
- 02_zettelkasten/04_knowledge_graph/Mechanism Comparison
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

Bridge Concept Rule

Bridge Concept Rule

Bridge Concept Rule は、Knowledge Graph において
どの概念を Bridge Concept として扱うかを決めるルールである。

Bridge Concept は便利だが、
むやみに増やすと次の問題が起きる。

- 抽象が増えすぎる
- 概念の意味が曖昧になる
- domain 特殊性が消える
- 誤った類比が増える

Bridge Concept Rule はこれを防ぐ。

Bridge Concept の定義（再確認）

Bridge Concept とは、

複数 domain に現れる構造を接続する概念

である。

Bridge Concept の条件

Bridge Concept として採用するには
次の条件を満たす必要がある。

条件1

複数 domain に存在する

最低でも

```
2 domain
```

以上に現れること。

例

```
権力
```

```
政治
企業
コミュニティ
```

条件2

構造を持つ

単なる言葉ではなく
pattern や mechanism と接続できる。

条件3

抽象概念である

具体的事件ではない。

条件4

pattern を説明できる

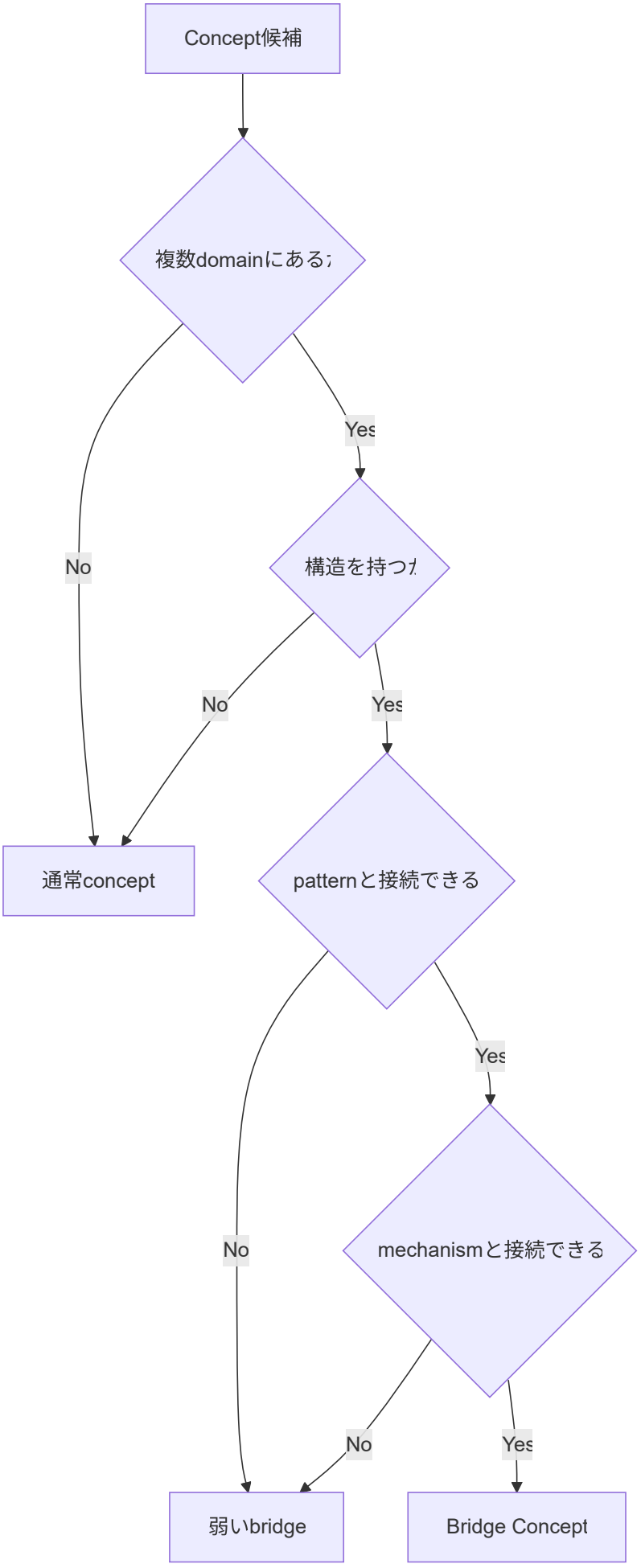
Bridge Concept は
複数 pattern の共通構造になる。

条件5

mechanism を接続できる

因果構造を横断する。

Bridge Concept 判定フロー


Concept候補
複数domainにある
Yes
No
構造を持つ
No
Yes
通常concept
patternと接続できる
Yes
No
mechanismと接続できる
No
Yes
弱いbridge
Bridge Concept

Bridge Concept の粒度

Bridge Concept は
抽象度によって3段階ある。

Level1

Domain Bridge

2つの domain を接続。

例

```
信号
```

```
動物行動
市場
```

Level2

Multi Domain Bridge

複数 domain を接続。

例

```
権力
```

```
政治
企業
```

```
社会
```

Level3

Universal Bridge

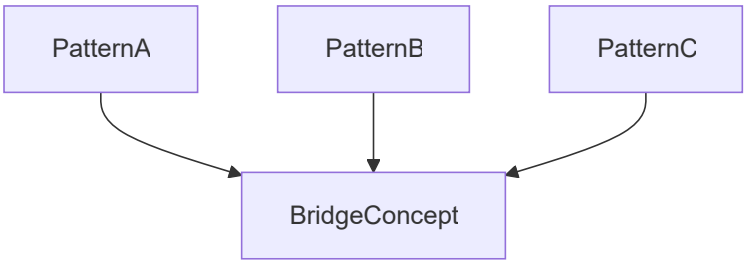
非常に多くの domain を接続。

例

```
協力
競争
信頼
```

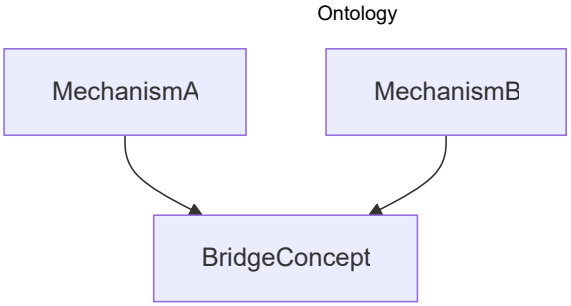
Bridge Concept と Pattern の関係

Bridge Concept は
複数 pattern を束ねる。



Bridge Concept と Mechanism

Bridge Concept は
mechanism を横断する。



Bridge Concept の書き方

Bridge Concept ノートには次を書く。

- 定義
- 関係 domain
- 関係 pattern
- 関係 mechanism
- representative case

Bridge Concept の例

例（抽象）

権力

domain

```
政治
企業
コミュニティ
```

pattern

```
権力集中
権力争い
```

mechanism

```
資源制御
情報制御
```

信頼

domain

```
社会
市場
コミュニティ
```

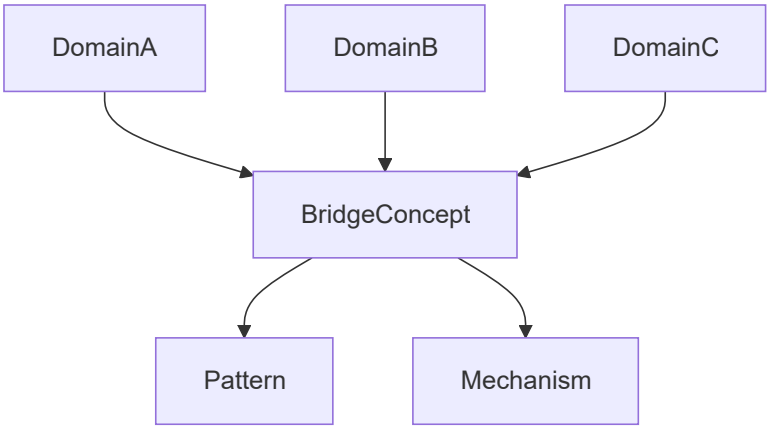
pattern

```
協力
裏切り
```

mechanism

```
評判
シグナリング
```

Bridge Concept の図



Bridge Concept の注意

Bridge Concept を作るとき
次に注意する。

---

1 抽象過剰

意味が曖昧になる。

---

2 domain 固有概念

特定領域にしかない概念は
bridge ではない。

---

3 類比錯覚

似ているだけで
同じ構造とは限らない。

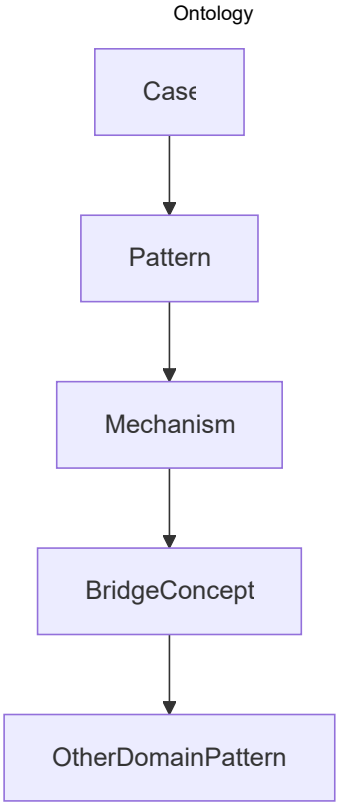
Bridge Concept の役割

Knowledge Graph において
Bridge Concept は

> 横方向の接続

を作る。

Bridge Concept と Knowledge Graph



LLM にとっての意味

Bridge Concept があると

LLM は

- cross domain analogy
- 新しい仮説
- pattern 転用

を行いやすくなる。

関連ノート

- 02_zettelkasten/04_knowledge_graph/Bridge Concept
- 02_zettelkasten/04_knowledge_graph/Bridge Detection Method
- 02_zettelkasten/04_knowledge_graph/Cross Domain Mapping
- 02_zettelkasten/04_knowledge_graph/Pattern
- Mechanism

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

Pattern Boundary Rule

Pattern Boundary Rule

Pattern Boundary Rule は、Knowledge Graph において
pattern 同士の境界を明確に保つための設計ルールである。

Pattern は本来、現実に繰り返し現れる構造や進行を抽象化したものである。
しかし pattern を増やしていくと、次の問題が必ず発生する。

- 似た pattern が増える
- 同じ case が複数 pattern に当てはまる
- pattern の定義が曖昧になる
- pattern と mechanism が混ざる

Pattern Boundary Rule は、これを防ぐ。

Pattern 境界が崩れる理由

Pattern 境界が曖昧になるのは、主に次の理由による。

1. 抽象レベルの違い

例

- 炎上パターン
- 規範逸脱可視化パターン
- 評判崩壊パターン

これらは同じ現象を違う抽象度で見ている可能性がある。

---

2. mechanism と混同

例

- 同調パターン
- 社会的証明

これは実際には pattern と mechanism が混ざっている。

---

3. outcome を pattern としてしまう

例

- 崩壊パターン
- 成功パターン

結果だけを pattern にしてしまうと境界が不明瞭になる。

---

4. role を pattern にしてしまう

例

- リーダーパターン
- 裏切り者パターン

これは pattern ではなく role である可能性が高い。

---

5. case を pattern に昇格してしまう

例

- 〇〇炎上パターン
- △△崩壊パターン

特定事件に依存するものは pattern ではなく case である。

Pattern Boundary の基本原則

Pattern の境界を安定させるには、次の原則を守る。

原則1

pattern は **時間構造** を持つ。

pattern は

- 状態
- 変化

- 結果

の流れを含む。

つまり pattern は

```
状態 → 行動 → 反応 → 結果
```

という **進行構造** を持つ。

原則2

pattern は **複数 case に現れる**。

1つの事例だけなら pattern ではない。

原則3

pattern は **mechanism と区別する**。

mechanism は

「なぜ起きるか」

pattern は

「どういう形で起きるか」

である。

原則4

pattern は **role を含めるが role ではない**。

pattern は多くの場合、

- actor
- role
- interaction

を含むが、

role 単体は pattern ではない。

原則5

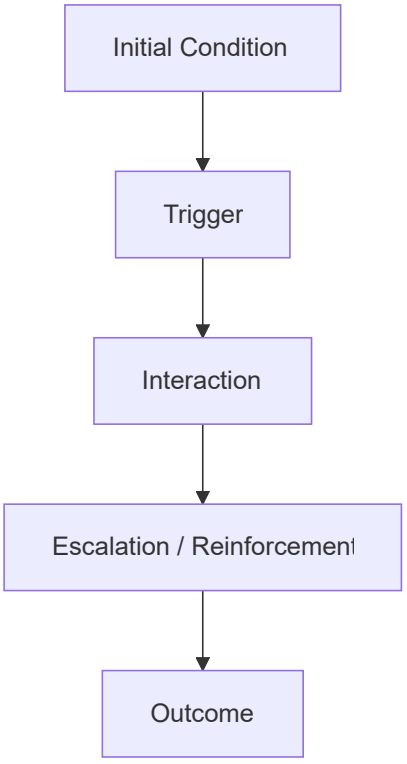
pattern は **結果ではない**

結果は outcome である。

pattern はその前の進行である。

Pattern の基本構造

pattern は一般に次の構造を持つ。



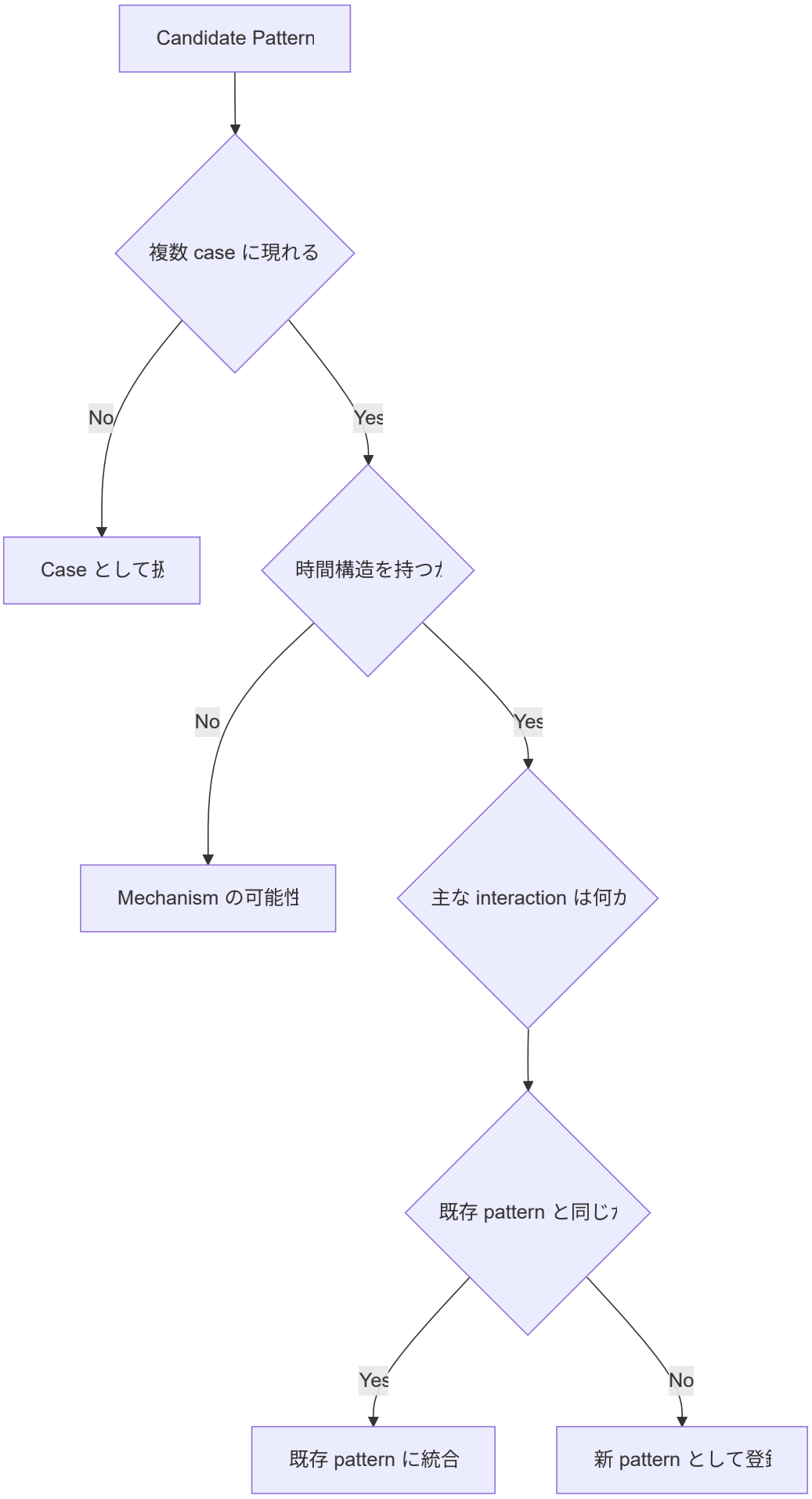
Pattern Boundary を決める方法

Pattern 境界は次の3つで決まる。

1. Trigger
2. Interaction
3. Outcome

この組み合わせが変わると
別 pattern になる。

Pattern 境界判定フロー


Candidate Pattern
複数 case に現れる
No
Yes
Case として扱
時間構造を持つか
No
Yes
Mechanism の可能性
主な interaction は何か
既存 pattern と同じか
Yes
No
既存 pattern に統合
新 pattern として登録

Pattern の境界を作る3つの軸

Pattern の境界は次の軸で区別できる。

---

1 Trigger の違い

同じ結果でも、
トリガーが違えば別 pattern。

例

- 規範逸脱トリガー
- 利益衝突トリガー
- 情報漏洩トリガー

---

2 Interaction の違い

同じトリガーでも
相互作用が違えば別 pattern。

例

- 同調拡散
- 対立激化
- 集団分裂

---

3 Outcome の違い

進行が同じでも
結果が違えば別 pattern。

例

- 炎上 → 沈静
- 炎上 → 社会制裁
- 炎上 → 文化変化

Pattern と Mechanism の違い

項目	Pattern	Mechanism
意味	起き方	原因
形式	進行	因果
時間	ある	ない場合もある
例	炎上パターン	同調メカニズム

Pattern と Structure の違い

項目	Pattern	Structure
意味	進行	配置
時間	ある	ない
例	権力争いパターン	階層構造

Pattern と Case の違い

項目	Pattern	Case
抽象度	高い	低い
再現性	ある	一度限り
例	規範逸脱炎上	具体事件

Pattern Boundary を守るためのルール

Rule1

pattern は **進行を持つこと**

Rule2

pattern は **最低3 case 以上**から抽象すること

Rule3

mechanism は pattern に入れない

Rule4

role は pattern にしない

Rule5

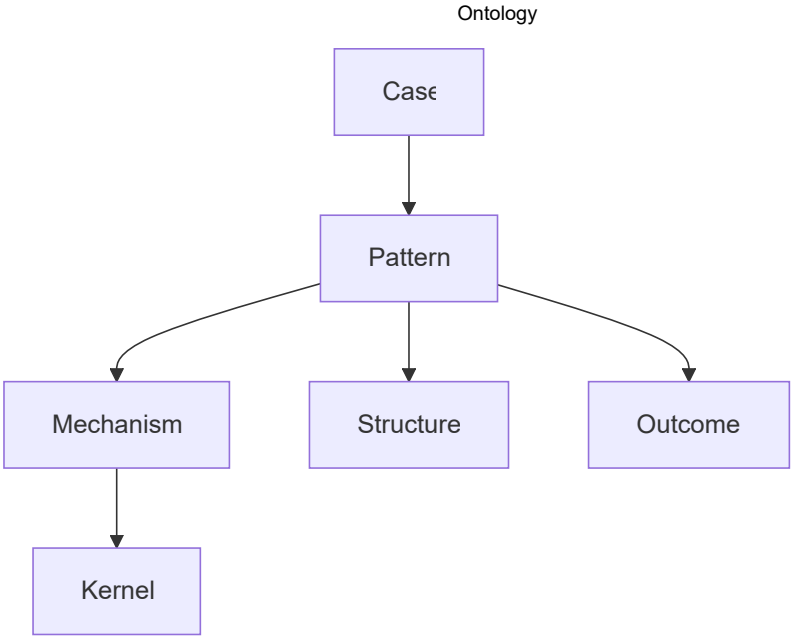
outcome は pattern 名にしない

Pattern Boundary のチェックリスト

新しい pattern を作る前に次を確認する。

- 複数 case に現れるか
- 時間進行があるか
- mechanism ではないか
- role ではないか
- outcome ではないか
- 既存 pattern に統合できないか

Pattern Boundary の図



Pattern Boundary を破る典型例

1 抽象レベル混乱

例

- 炎上パターン
- SNS炎上パターン
- 企業炎上パターン

これは階層化すべき。

---

2 mechanism 混入

例

- 同調パターン

同調は mechanism である。

---

3 outcome pattern

例

- 崩壊パターン

これは結果。

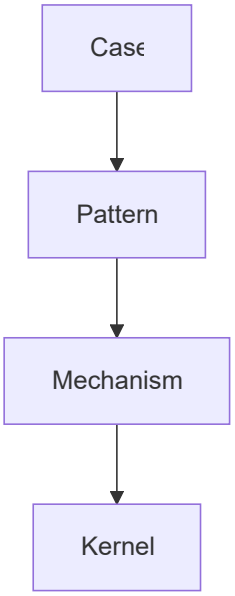
良い Pattern の条件

良い pattern は次を満たす。

- 複数 case に出る
- 進行が明確
- actor / role が分かる
- mechanism が説明できる
- outcome が複数ありうる

Knowledge Graph での位置

Pattern は Knowledge Graph の中間層に位置する。



LLM にとっての意味

Pattern Boundary が明確だと、

LLM は

- case を pattern に昇格できる
- pattern を mechanism に分解できる
- pattern 同士を比較できる

逆に境界が曖昧だと、

- 同じ現象を別名で語る
- mechanism と混同する
- reasoning が崩れる

この Vault での運用

この Vault では次を推奨する。

pattern は

- domain/pattern
- model/pattern

などに置く。

各 pattern には必ず

- 代表 case
- mechanism
- outcome

を書く。

関連ノート

- 02_zettelkasten/04_knowledge_graph/Pattern
- 02_zettelkasten/04_knowledge_graph/Representative Case Rule
- Mechanism
- Structure
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph
- 02_zettelkasten/04_knowledge_graph/Anchor Case
- old_zettelkasten/pattern/Pattern Hub

Hub Template

Hub Template

このテンプレートは、Knowledge Graph における Hub ノートを
目次ではなく、地図・入口・導線・橋の所在表示 として機能させるための標準形である。

Hub は単なる一覧ではない。
このテンプレートでは、次の4機能を必ず意識する。

1. Map
2. Entry Point
3. Traversal Guide
4. Bridge Locator

使用ルール

- Hub には子ノートの全文を書かない
- 中核ノードを優先して載せる
- ノード同士の関係を書く
- 読む順序を示す
- 関連 Hub と Bridge Node を明示する
- index 的一覧は別ノートに分けてもよい

テンプレート本文

```
---
note_type: hub
layer: structure
status: stable
maturity: canonical
hub_type:
domain:
parent_hub:
related_hubs:
tags:
---
```

```
# {{Hub Name}}

## 概要
この Hub は [[{{対象領域}}]] に関する主要ノードを束ねるための Hub である。
主に [[{{node type 1}}]], [[{{node type 2}}]], [[{{node type 3}}]] を中核に扱い、
[[{{用途1}}]], [[{{用途2}}]], [[{{用途3}}]] の入口として機能する。

---

## この Hub の役割
この Hub は次の役割を持つ。

- **地図**：この領域の骨格を示す
- **入口**：初学者や LLM がどこから入るかを示す
- **導線**：どの順で辿ると理解しやすいかを示す
- **橋の表示**：他領域へ接続する Bridge Node を示す

---

## 中核ノード
この領域の中心となるノード。

- [[{{Core Node 1}}]]
- [[{{Core Node 2}}]]
- [[{{Core Node 3}}]]
- [[{{Core Node 4}}]]
- [[{{Core Node 5}}]]

### 補助ノード
- [[{{Support Node 1}}]]
- [[{{Support Node 2}}]]
- [[{{Support Node 3}}]]

---

## 関係の見取り図
この領域では、主要ノードはおおむね次のように関係する。

- [[{{Core Node 1}}]] は [[{{Core Node 2}}]] の前提・基盤となる
- [[{{Core Node 2}}]] は [[{{Core Node 3}}]] を説明する
- [[{{Core Node 3}}]] は [[{{Core Node 4}}]] に現れやすい
- [[{{Core Node 4}}]] は [[{{Core Node 5}}]] に接続する
```

```
- [[{{Bridge Node 1}}]] は他 Hub への横断経路になる

---

## 全体構造

```mermaid
graph TD
    A[{{Hub Name}}] --> B[{{Core Node 1}}]
    A --> C[{{Core Node 2}}]
    A --> D[{{Core Node 3}}]
    A --> E[{{Core Node 4}}]
    A --> F[{{Core Node 5}}]

    B --> C
    C --> D
    D --> E
    C --> F
```

読む順序

初学者向け

- {{Core Node 1}} → {{Core Node 2}} → {{Core Node 3}}

分析向け

- {{Core Node 2}} → {{Core Node 4}} → {{Core Node 5}}

実務向け

- {{Framework or Method Node 1}} → {{Tool Node 1}} → {{Representative Case 1}}

代表的な Traversal

1. 定義理解ルート

- {{Concept Node}} → {{Contrast Node}} → {{Anchor Case}}

2. 分析ルート
```

- {{Question Node}} → {{Mechanism Node}} → {{Structure Node}} → {{Case Node}}

3. 抽象化ルート

- {{Case Node}} → {{Pattern Node}} → {{Mechanism Node}} → {{Kernel Node}}

4. 横断ルート

- {{Bridge Node 1}} → {{Other Domain Pattern}} → {{Other Domain Case}}

Bridge Node

この Hub から他領域へ移る際の橋となるノード。

- {{Bridge Node 1}}
  - 接続先: {{Related Hub 1}}, {{Related Hub 2}}
- {{Bridge Node 2}}
  - 接続先: {{Related Hub 3}}
- {{Bridge Node 3}}
  - 接続先: {{Related Hub 4}}

Anchor Case

この Hub の抽象ノードを具体に接地させる代表 case。

- {{Anchor Case 1}}
  - 何を示すか: {{Concept / Pattern / Mechanism 1}}
- {{Anchor Case 2}}
  - 何を示すか: {{Concept / Pattern / Mechanism 2}}
- {{Anchor Case 3}}
  - 何を示すか: {{Bridge Concept 1}}

Representative Pattern / Mechanism

この領域を理解するうえで特に重要な型・作動。

Pattern

- {{Pattern 1}}

- {{Pattern 2}}
- {{Pattern 3}}

Mechanism

- {{Mechanism 1}}
- {{Mechanism 2}}
- {{Mechanism 3}}

主要な問い

この Hub から立てやすい問い。

- {{Question 1}}
- {{Question 2}}
- {{Question 3}}
- {{Question 4}}

関連 Method / Tool

この領域を扱うときによく使う方法・道具。

Method

- {{Method 1}}
- {{Method 2}}
- {{Method 3}}

Tool

- {{Tool 1}}
- {{Tool 2}}
- {{Tool 3}}

関連 Hub

Parent Hub

- {{Parent Hub}}

Sibling Hubs

- {{Sibling Hub 1}}
- {{Sibling Hub 2}}

Child Hubs

- {{Child Hub 1}}
- {{Child Hub 2}}

Cross Domain Hubs

- {{Cross Domain Hub 1}}
- {{Cross Domain Hub 2}}

ノート追加時の指針

新しいノートをこの Hub にぶら下げるときは、次を確認する。

1. そのノートは中核か補助か
2. どの既存ノードと relation を持つか
3. 読む順序に入れるべきか
4. Bridge Node になりうるか
5. Anchor Case として使えるか

この Hub の限界

この Hub では扱いきれない論点。

- {{Outside Scope 1}}
- {{Outside Scope 2}}
- {{Outside Scope 3}}

必要なら以下の Hub へ移る。

- {{Other Hub 1}}
- {{Other Hub 2}}

関連ノート

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph
- 02_zettelkasten/04_knowledge_graph/Traversal
- 02_zettelkasten/04_knowledge_graph/Reasoning Path
- 02_zettelkasten/04_knowledge_graph/Bridge Concept
- 02_zettelkasten/04_knowledge_graph/Anchor Case

```
---

# 最小版テンプレート

Hub をまず軽く作りたい場合は、これだけでもよい。

```markdown
---
note_type: hub
layer: structure
status: draft
hub_type:
---

# {{Hub Name}}

## 概要
この Hub は [[{{領域}}]] の主要ノードを束ねる。

## 中核ノード
- [[{{Core Node 1}}]]
- [[{{Core Node 2}}]]
- [[{{Core Node 3}}]]

## 関係の見取り図
- [[{{Core Node 1}}]] は [[{{Core Node 2}}]] に関わる
- [[{{Core Node 2}}]] は [[{{Core Node 3}}]] を説明する

## 読む順序
- [[{{Core Node 1}}]] → [[{{Core Node 2}}]] → [[{{Core Node 3}}]]

## Bridge Node
- [[{{Bridge Node 1}}]]
- [[{{Bridge Node 2}}]]

## Anchor Case
```
```

```
- [[{{Anchor Case 1}}]]
- [[{{Anchor Case 2}}]]

## 関連 Hub
- [[{{Related Hub 1}}]]
- [[{{Related Hub 2}}]]
```

Domain Hub 用の補正

Domain Hub では、sub hub を見せることが重要になる。

追加推奨セクション:

- sub hub 一覧
- domain 特有の主要問い
- domain で頻出の pattern / mechanism
- 他 domain との bridge

Pattern Hub 用の補正

Pattern Hub では、次を強める。

- pattern 間差分
- representative case
- pattern boundary
- related mechanism

追加推奨セクション:

- 類似 pattern との違い
- pattern 昇格候補
- 代表的な崩れ方 / outcome

Method Hub 用の補正

Method Hub では、次を強める。

- どんな問いに使うか

- 入力に何が必要か
- 出力として何が得られるか
- tool との接続

追加推奨セクション:

- 適用場面
- 非適用場面
- 併用しやすい method

Meta Hub 用の補正

Meta Hub では、次を強める。

- node type / edge type / traversal の関係
- vault 全体の読み方
- hub 間ネットワーク
- reasoning engine との接続

記入時の注意

1. Hub を百科事典にしない

各ノートの中身まで全部書かない。

2. index と混同しない

網羅一覧は別ノートでもよい。

3. relation を書く

リンクを並べるだけにしない。

4. 読み順を書く

Hub は導線を持つべきである。

5. bridge と anchor を必ず入れる

Hub を横にも下にも開く。

Hub 記述チェックリスト

- 概要がある
- 中核ノードがある
- relation の見取り図がある
- 読む順序がある
- bridge node がある
- anchor case がある
- 関連 hub がある
- 子ノートの本文を食っていない

このテンプレートの使いどころ

- Domain Hub
- Model Hub
- Pattern Hub
- Method Hub
- Meta Hub

関連ノート

- 02_zettelkasten/04_knowledge_graph/Hub Design Rule
- 02_zettelkasten/04_knowledge_graph/Knowledge Graph
- 02_zettelkasten/04_knowledge_graph/Traversal
- 02_zettelkasten/04_knowledge_graph/Bridge Concept
- 02_zettelkasten/04_knowledge_graph/Anchor Case

Hub Design Rule

Hub Design Rule

Hub Design Rule は、Knowledge Graph において
Hub ノートをどのように設計すれば、一覧ではなく思考導線として機能するか を定めるルールである。

Hub は Vault の中心に見えやすいが、
作り方を誤るとすぐに壊れる。

- 目次の羅列になる
- 個別ノートの中身を食い潰す
- 一覧と解説とルールが混ざる
- 重要ノードと周辺ノードの差が見えない
- 読む順番が分からない
- LLM が hub だけを読んで誤って満足する

Hub Design Rule は、Hub を
単なる集積点ではなく、Traversal を導く中継点 として設計するための規則である。

定義

Hub Design Rule とは、
Hub ノートを **地図・導線・束ね・関係可視化** の役割に保つための設計規則である。

Hub の主目的は次の通りである。

1. ノード群の全体像を示す
2. 重要ノードへの入口を与える
3. 読む順序を明示する
4. 抽象と具体の接続点を示す
5. Bridge Node を見つけやすくする
6. Traversal の中距離移動を助ける

Hub は「内容の本体」ではなく、
内容へ辿るための構造ノード である。

なぜ必要か

Hub は便利なので、すぐに肥大化する。
しかし肥大化した Hub は Graph を弱くする。

典型的な失敗は次の通りである。

1. 目次化

リンク一覧だけで、なぜ重要なのか分からない。

2. 百科事典化

各ノートの中身まで Hub に書いてしまい、個別ノートが不要になる。

3. 混載

Hub, Index, Rule, Framework, Note Collection が同じノートに同居する。

4. 導線不在

どこから読むべきか、どの順で辿るべきかが示されない。

5. 階層不明

重要ノードと補助ノードが同列に並ぶ。

6. Graph 非対応

Node Type や Edge Type の見取り図がなく、Traversal に使えない。

Hub Design Rule は、こうした崩れを防ぐ。

全体構造

Hub の本質

Hub の本質は、
一覧を作ること ではなく
意味のある入口を作ること にある。

Hub は次の4役を担う。

1. Map

何がこの領域にあるかを示す。

2. Entry Point

初めて読む人や LLM がどこから入ればよいかを示す。

3. Traversal Guide

どの順で辿ると理解が深まるかを示す。

4. Bridge Locator

どのノードが他領域へつながる橋なのかを示す。

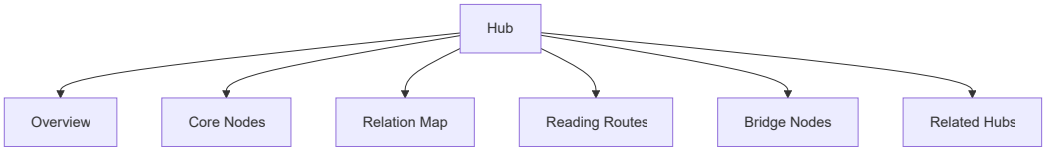
この4つを満たさない Hub は、
単なるリンク集になりやすい。

良い Hub の最小構成

良い Hub は、最低でも次の5要素を持つ。

1. この領域は何を扱うか
2. 中核ノードは何か
3. それらがどう関係するか
4. どこから読むべきか
5. 他のどの Hub へつながるか

Hub の標準構造



---

1. Overview

Hub の冒頭では、
この領域が何を扱うか を短く定義する。

書くべきこと:

- 何のための Hub か
- 何を束ねるか
- どの node type が多いか
- 何に使うと便利か

悪い例:

- ここではいろいろなノートをまとめる

良い例:

- この Hub は、人間行動を説明する concept / mechanism / pattern を束ね、意思決定や観察の起点となる

---

2. Core Nodes

Hub には、全ノードではなく
中核ノード を置く。

中核ノードとは:

- この領域の骨格になる
- 他ノードへの分岐点になる
- traversal 上の通り道になる
- 他領域と接続しやすい

たとえば Human Model Hub なら:

- 認知
- 感情
- 動機
- アイデンティティ
- 学習

などが Core Nodes になる。

---

3. Relation Map

Hub は、リンクの羅列でなく
関係の見取り図 を持つ必要がある。

たとえば:

- 認知は判断に関わる
- 感情は注意と評価を変形する
- 動機は選好と行動方向を与える
- アイデンティティは自己整合性に関わる

のように、
ノード同士がどう結びつくかを書く。

---

4. Reading Routes

Hub の重要機能は、
どう読むかの順路 を与えることにある。

代表的には次のような順路を書く。

初学者向け

概念 → 代表 mechanism → 代表 pattern → 代表 case

分析向け

question → mechanism → structure → case

実務向け

framework → method → tool → case

読む順番がない Hub は、
探索の起点になりにくい。

---

5. Bridge Nodes

Hub には、
他領域へつながる橋ノードを明示すると強い。

たとえば Social Pattern Hub なら:

- 信頼
- 同調
- 規範
- 排除
- 正統性

などは、
psychology, politics, business, history をまたぎやすい。

Bridge Node を書くことで、
Graph 全体の横断性が見えやすくなる。

---

6. Related Hubs

Hub は孤立してはいけない。
親 Hub・兄弟 Hub・子 Hub・横断 Hub への接続を示す。

種類:

- Parent Hub
- Sibling Hub
- Child Hub
- Cross Domain Hub

これにより、Hub 自体が Graph の node として機能する。

Hub の推奨セクション

Hub は次のセクションで組むと安定しやすい。

```
1. 概要
2. この Hub の役割
3. 中核ノード
4. ノード群の関係
5. 読む順序
6. 代表的な探索経路
7. Bridge Node
8. 関連 Hub
9. 関連する case / pattern / method
```

Hub の標準テンプレート

```
# ○○ Hub
```

```
## 概要
この Hub は何を扱うか。

## この Hub の役割
地図 / 入口 / 導線 / 橋のどれを担うか。

## 中核ノード
- [[A]]
- [[B]]
- [[C]]

## 関係の見取り図
A は B とどうつながり、C はどこを媒介するか。

## 読む順序
- 初学者向け: [[A]] → [[B]] → [[C]]
- 分析向け: [[B]] → [[D]] → [[E]]

## 代表的な探索経路
- Question → Mechanism → Pattern
- Case → Pattern → Kernel

## Bridge Node
- [[X]]
- [[Y]]

## 関連 Hub
- [[Parent Hub]]
- [[Sibling Hub]]
- [[Child Hub]]
```

Hub に書いてよいこと / 書きすぎないこと

書いてよいこと

- 概要
- 関係の見取り図
- 優先順位
- 読む順序
- 重要差分
- 代表ノードの短い説明

書きすぎないこと

- 各子ノートの本文全文
- 事例の詳細叙述
- ルールの全文
- 巨大な一覧表
- method の詳細手順全部

Hub は「中継点」であり、
「終着点」ではない。

Hub の抽象度

Hub は中位抽象を保つべきである。

低すぎる Hub:

- case の羅列
- 細かい observation の束

高すぎる Hub:

- 世界とは何か
- すべては関係である

良い Hub:

- ある領域の主要構成要素と主要経路が分かる
- 個別ノートに降りる余地がある

Hub の分類

Hub にはいくつかの型がある。

---

1. Domain Hub

分野全体を束ねる。

例:

- History Hub
- Law Hub
- Business Hub
- Tourism Hub

役割:

- 分野入口
- 主要 sub hub の導線

---

2. Model Hub

モデル群や構成要素を束ねる。

例:

- Human Model Hub
- Decision Model Hub
- Reading Model Hub

役割:

- concept / mechanism / structure を整理する

---

3. Pattern Hub

反復現象を束ねる。

例:

- Social Pattern Hub
- Organization Pattern Hub
- Failure Pattern Hub

役割:

- case を pattern にまとめる
- pattern 間差分を見せる

---

4. Method Hub

分析法や思考法を束ねる。

例:

- Analysis Method Hub
- Inquiry Method Hub

役割:

- question から method への導線

---

5. Meta Hub

Vault 全体の設計原理を束ねる。

例:

- Knowledge Graph Hub
- Thinking Engine Hub
- Structure Hub

役割:

- 全体理解
- schema 理解
- 横断導線

Hub 間の関係

Hub は単独で置くのでなく、
Hub 自体がネットワーク化されるべきである。

親 Hub

より大きい領域を束ねる Hub

例:

- Human Model Hub → World Model Hub

子 Hub

特化領域を束ねる Hub

例:

- Social Pattern Hub → 排除パターン Hub

兄弟 Hub

同じ階層の別領域 Hub

例:

- Human Model Hub ↔ Social Structure Hub

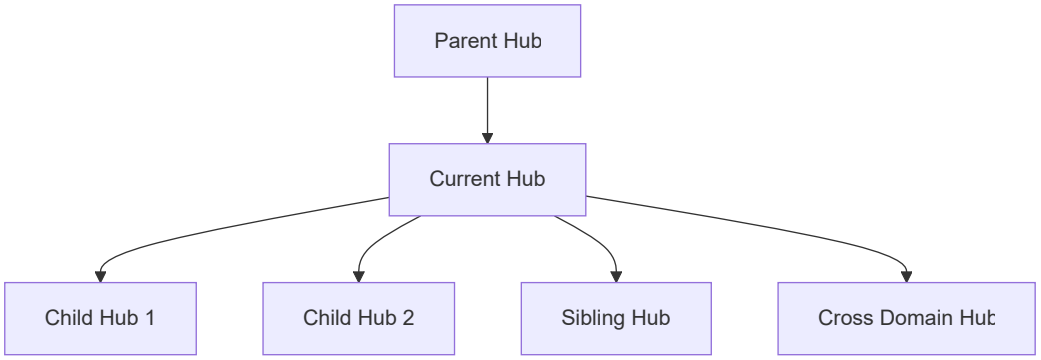
横断 Hub

複数領域をつなぐ Hub

例:

- Knowledge Graph Hub
- Thinking Engine Hub

Hub 関係の図



Hub と Index の違い

これは非常に重要である。

Hub

- 地図

- 関係を示す
- 読む順序を示す
- 重要度を示す

Index

- 一覧
- 網羅
- 機械的配列
- 管理寄り

Index を Hub に入れすぎると、
Hub が目詰まりする。

Hub と Framework の違い

Hub

何があるか、どうつながるかを示す

Framework

どう考えるか、どう処理するかを示す

Hub は地図、Framework は操作枠である。

Hub と Rule の違い

Hub

対象ノード群の見取り図

Rule

書き方・判断基準・運用規則

Hub に Rule を大量に入れると機能がぼやける。

良い Hub の条件

1. 一読で領域の骨格が分かる

2. 中核ノードが見える

3. どこから読めばいいか分かる

4. 他 Hub への接続がある

5. 子ノートを読む意味が残っている

6. Traversal の経路が示されている

悪い Hub のパターン

1. Table of Contents Only

リンクだけ並んでいて意味がない

2. Mini Encyclopedia

Hub に内容を書きすぎて、個別ノートが死ぬ

3. Mixed Function Hub

Hub, rule, index, framework が混ざる

4. Dead End Hub

他の hub や core node につながらない

5. Flat Hub

全ノードが同列で、重要度差がない

Hub を分割すべき条件

次のどれかに当てはまるなら分割を検討する。

1. セクションが多すぎる
2. Core Node が多すぎる
3. pattern / method / case が全部同居している
4. 1ノートで読み順を示しきれない

5. 個別ノートに書くべき内容が侵食されている

分割の方向

1. 領域分割

例:

- Human Model Hub
- Human Emotion Hub
- Human Cognition Hub

2. 役割分割

例:

- Pattern Hub
- Method Hub
- Case Index

3. 抽象度分割

例:

- World Model Hub
- Domain Hub
- Case Hub

LLM にとっての意味

良い Hub があると、LLM は

- どこが中核かを先に把握でき
- 周辺ノードに無駄に拡散せず
- 読む順序を参考に探索し
- bridge node を辿って横断しやすくなる

逆に悪い Hub だと、

- リンクの山から重要度を判別できず
- Hub 本文で誤って完結し

- 子ノートの探索が浅くなる

Hub は LLM にとって、
探索の優先順位を示す標識 である。

この Vault における実装方針

この Vault では、Hub を作るとき最低限次を守る。

必須

- 概要
- 中核ノード
- 関係の見取り図
- 読む順序
- 関連 Hub

推奨

- Bridge Node
- 代表 traversal
- 初学者向け route
- 分析向け route

分離推奨

- 網羅一覧は Index
- 書き方規則は Rule
- 実務手順は Framework / Method
- 詳細解説は子ノート

他ノートとの接続

上位

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

近接

- 02_zettelkasten/04_knowledge_graph/Graph Maintenance
- 02_zettelkasten/04_knowledge_graph/Traversal

- 02_zettelkasten/04_knowledge_graph/Reasoning Path
- 02_zettelkasten/04_knowledge_graph/Bridge Concept
- 02_zettelkasten/04_knowledge_graph/Anchor Case

下位候補

- Domain Hub Rule
- Model Hub Rule
- Pattern Hub Rule
- Method Hub Rule
- 02_zettelkasten/04_knowledge_graph/Hub Template

まとめ

Hub Design Rule は、Hub ノートを
一覧ではなく、思考導線として機能させるための設計規則 である。

Hub は、

- 地図であり
- 入口であり
- 導線であり
- 橋の所在を示す標識である

そのため良い Hub は、
内容を全部抱え込まず、

- 中核ノードを示し
- 関係の骨格を見せ
- 読む順番を与え
- 他 Hub へ接続する

形で設計されるべきである。

Hub が Graph の都市計画図だとすれば、
個別ノートはその都市を構成する建物である。

Bridge Concept

Bridge Concept

Bridge Concept は、Knowledge Graph において
複数の domain・model・pattern・method を横断的に接続する中継概念である。

Knowledge Graph は node と edge だけでは十分ではない。
各領域がそれぞれ整っていても、間をつなぐ概念がなければ、Vault 全体は

- history は history の中だけ
- business は business の中だけ
- psychology は psychology の中だけ
- law は law の中だけ

で閉じやすい。

Bridge Concept は、こうした分野間の断絶を越えて、
異なる領域にある知識を
同じ構造・同じメカニズム・同じ問いのもとで読み替えるための接続点である。

定義

Bridge Concept とは、
複数の領域やノード群にまたがって現れ、
それらのあいだに **共通の意味軸・構造軸・作動軸** を与える概念である。

その主な役割は次の通りである。

1. domain 横断を可能にする
2. 類比・比較・応用の中継点になる
3. isolated cluster を接続する
4. reasoning path のジャンプ点になる
5. pattern と mechanism の再利用性を高める
6. LLM に横断的 retrieval の足場を与える

Bridge Concept は、
単に「いろいろな所で使われる言葉」ではない。
重要なのは、**異なる領域の知識を、同じ観点で結び直せること**である。

なぜ必要か

Vault が大きくなると、
普通は縦方向の整備から始まる。

- domain ごとの hub ができる
- concept 群ができる
- pattern 群ができる
- method 群ができる

しかし次の段階で問題が起こる。

1. domain 島化

各分野が内部では豊かだが、相互に行き来しない。

2. 類比不能

似た構造があっても、橋概念がないため見つけられない。

3. 応用不能

心理で学んだことを営業や歴史に移せない。

4. hub 間断絶

Hub がそれぞれ立派でも、横断導線が弱い。

5. LLM の近傍検索偏重

表層語彙が違うと、別 cluster として扱われやすい。

Bridge Concept は、
Vault 全体を「フォルダの集合」ではなく
横断可能な知識網 にするために必要である。

全体構造

Bridge Concept の本質

Bridge Concept の本質は、
別々に見える現象を、共通の問いで読めるようにすること にある。

たとえば「権力」という概念は、

- 国家支配
- 企業組織
- fandom 内部秩序
- 家族内役割
- 学校制度
- 宗教組織

のすべてに現れる。

ここで重要なのは、
「権力という単語が同じ」ことではなく、
資源配分・命令可能性・制裁可能性・正統化・抵抗
といった観点を共通に適用できることである。

Bridge Concept とは、
知識を横断するための **共通のレンズ** である。

Bridge Concept の条件

何でも bridge になるわけではない。
よい Bridge Concept には次の条件がある。

---

1. 多領域出現性

複数の domain に現れる。

例:

- 信頼 → psychology / business / politics / tourism
- 正統性 → history / politics / organization / culture
- 制約 → economics / engineering / psychology / law

---

2. 構造再利用性

その概念を使うことで、
ある領域の構造を別領域へ写せる。

例:

- center-periphery
- principal-agent
- signaling
- feedback

---

3. 問い生成力

その概念を持ち込むと、新しい問いが立つ。

例:

- この組織では誰が正統性を供給しているか
- この市場では何が signal になっているか
- この共同体ではどこに境界が引かれているか

---

4. Traversal 中継力

Question から別 domain へ移る中継点として使える。

---

5. 抽象度の適切さ

抽象すぎず、固有すぎない。

悪い bridge:

- 世界
- すべて
- 人間らしさ

固有すぎる bridge:

- ある一国固有制度名
- 一作品固有設定

良い bridge:

- 正統性
- 信頼
- インセンティブ
- アイデンティティ
- 制約
- 境界

Bridge Concept の主な型

Bridge Concept にはいくつかの類型がある。

---

1. Structural Bridge

配置や骨格を横断する概念。

例:

- 階層
- 中心周辺
- 境界
- ネットワーク
- 分業
- 委任

使い道:

- 社会構造
- 組織構造
- 国家構造
- 都市構造
- fandom 構造

---

2. Mechanistic Bridge

作動過程を横断する概念。

例:

- シグナリング

- 同調
- 協調
- フィードバック
- 学習
- 逸脱制裁

使い道:

- 市場
- 組織
- SNS
- 政治
- 教育

---

3. Normative Bridge

価値・正当化・規範を横断する概念。

例:

- 正統性
- 責任
- 公正
- 義務
- 権利
- 規範

使い道:

- 歴史
- 法
- 組織
- 宗教
- 文化研究

---

4. Interpretive Bridge

意味づけ・自己理解・物語化を横断する概念。

例:

- アイデンティティ
- 物語
- 承認
- 象徴
- 記憶
- 意味

使い道:

- 人格
- 国家
- fandom
- ブランド
- 歴史叙述

---

5. Operational Bridge

行動設計・実務設計に関わる横断概念。

例:

- インセンティブ
- 評価
- 情報非対称
- principal-agent
- KPI
- ボトルネック

使い道:

- business
- public policy
- organization
- education
- platform design

---

6. Meta Bridge

システム全体の状態や変化を読む概念。

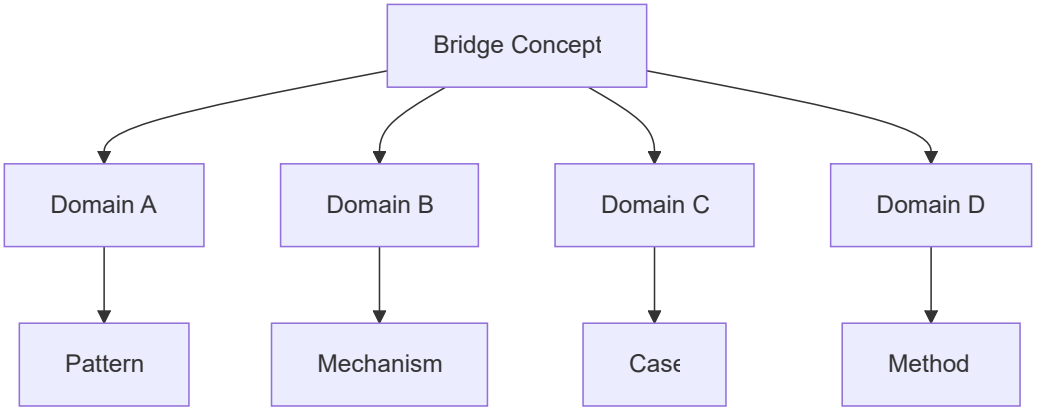
例:

- システム
- 適応
- レジリエンス
- 崩壊
- 安定
- 再編

使い道:

- 歴史
- 生態
- 組織
- 地域社会
- 事業設計

Bridge Concept の図



代表的な Bridge Concept 例

以下は特に強い Bridge Concept である。

---

1. Power / 権力

つなぐ領域:

- history
- politics
- business
- organization
- culture
- family

つなぐ問い:

- 誰が何を決められるか
- 誰が制裁できるか
- 誰が正当化を独占しているか

---

2. Legitimacy / 正統性

つなぐ領域:

- 国家
- 組織
- 宗教
- fandom
- リーダーシップ
- ブランド

つなぐ問い:

- なぜ従うのか
- 何によって支配は受容されるのか
- 何が崩れると秩序は揺らぐのか

---

3. Constraint / 制約

つなぐ領域:

- economics
- psychology
- engineering
- law
- management
- urban planning

つなぐ問い:

- 何が可能範囲を決めているか
- 何が行為を制限しているか
- 理想と実行の差はどこから来るか

---

4. Trust / 信頼

つなぐ領域:

- social psychology
- organization
- market
- tourism
- community
- politics

つなぐ問い:

- 協力は何によって成立するか
- 裏切りは何を壊すか
- 可視性と評判はどう機能するか

---

5. Signaling / シグナリング

つなぐ領域:

- labor market
- branding
- education
- mating
- politics
- platform economy

つなぐ問い:

- 観察できない品質をどう伝えるか
- なぜ高コスト signal が効くのか
- 何が credential になるのか

---

6. Identity / アイデンティティ

つなぐ領域:

- personality
- nation
- organization culture
- fandom
- narrative analysis
- memory studies

つなぐ問い:

- 自己像はどう維持されるか
- 集団帰属はどう働くか
- 異物排除は何を守っているのか

---

7. Boundary / 境界

つなぐ領域:

- law
- community
- ethnicity
- fandom
- organization
- geography

つなぐ問い:

- 内と外はどこで区切られるか
- 誰が membership を決めるか
- 越境はなぜ摩擦を起こすか

---

8. Incentive / インセンティブ

つなぐ領域:

- economics
- management

- education
- public policy
- platform design

つなぐ問い:

- 人は何に反応して行動を変えるか
- 目標設定がどんな歪みを生むか
- 評価制度は何を誘発するか

Bridge Concept の作り方

Bridge Concept は最初から与えられることもあるが、
多くは運用の中で発見される。

Step 1. cluster を観察する

別 domain に似た問題や mechanism がないかを見る。

例:

- 歴史の支配構造
- 組織の承認構造
- fandom の規範構造

Step 2. 共通して問える観点を探す

例:

- 誰が命令できるか
- 誰が membership を決めるか
- 何が signal になるか

Step 3. 固有名を外し、共通概念に上げる

例:

- 王権、経営権、モデレーション権限
  → 権力 / 制裁権 / 正統性

Step 4. domain ごとに接続する

Bridge Concept を中心に、

- domain A の pattern
- domain B の mechanism
- domain C の case
  をつなぐ。

Step 5. 代表 path を作る

Bridge Concept を経由する reasoning path を一つ以上作る。

Bridge Concept の記述内容

Bridge Concept ノートには最低限、次を書くとよい。

1. 定義

何を共通軸として扱うか。

2. なぜ bridge なのか

どの domain をつなぐか。

3. 代表 domain

どこで現れるか。

4. 代表 pattern / mechanism

どのような形で現れるか。

5. 代表 case

抽象を支える anchor case は何か。

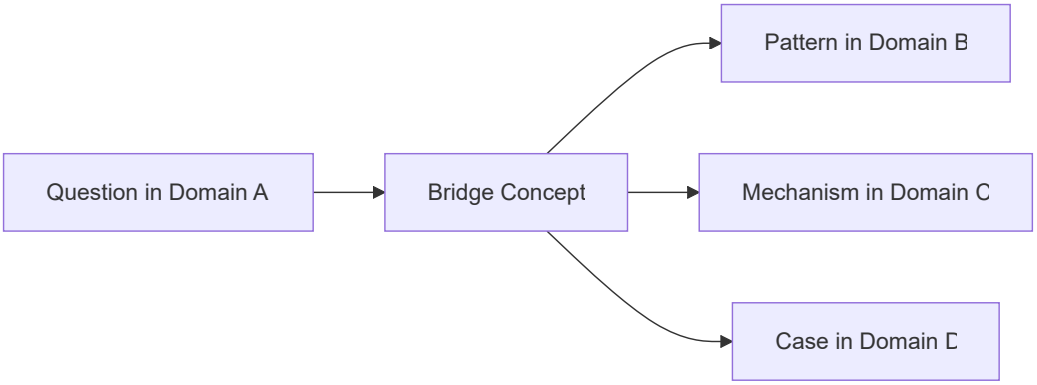
6. contrasts_with

近いが違う概念は何か。

7. traversal 例

どのように別領域へ飛べるか。

Bridge Concept の典型 traversal



例:

- 組織の責任回避はなぜ起きるか
  → 権力
  → 委任構造
  → 責任分散メカニズム
  → 国家官僚制 case

Bridge Concept と類比の関係

Bridge Concept は類比を助けるが、
類比そのものではない。

類比

A と B が似ていると言う

Bridge Concept

A と B を共通軸 C で読む

例:

- 国家官僚制と企業官僚制は似ている
  これは類比
- 両者を「階層」「委任」「正統性」「責任分散」で読む
  これが Bridge Concept の使用

つまり Bridge Concept は、
類比の **根拠軸** を与える。

Bridge Concept と Hub の関係

Bridge Concept は、
しばしば Hub 間をつなぐノードになる。

例:

- Human Model Hub ↔ Social Pattern Hub
  - アイデンティティ
  - 信頼
  - 規範
- Business Hub ↔ Law Hub
  - 責任
  - インセンティブ
  - 02_zettelkasten/02_world_model/meta/model/social/information/情報非対称
- History Hub ↔ Organization Hub
  - 権力
  - 正統性
  - 委任

Hub に bridge node を明示することで、
cross-domain traversal が強くなる。

良い Bridge Concept の条件

1. 多領域に現れる

2. 問いを生む

3. pattern や mechanism を横断接続できる

4. 過度に曖昧でない

5. case と往復可能

6. 他 bridge とも接続できる

悪い Bridge Concept のパターン

1. 抽象過剰

何でも説明できるが、何も区別できない。

例:

- 世界
- 関係性
- 人間性

2. 固有過剰

特定作品・特定制度・特定事件にしか使えない。

3. 単語一致だけ

同じ言葉が使われているだけで、実際は構造が違う。

4. 橋でなく終点

bridge と言いながら、他 domain への path がない。

5. mechanism 不在

横断できるが、何が共通かが薄い。

Bridge Concept の発見質問

新しい bridge 候補を見つけるために、次の問いが使える。

1. この概念は別 domain にも現れるか

2. この概念を持ち込むと新しい問いが立つか

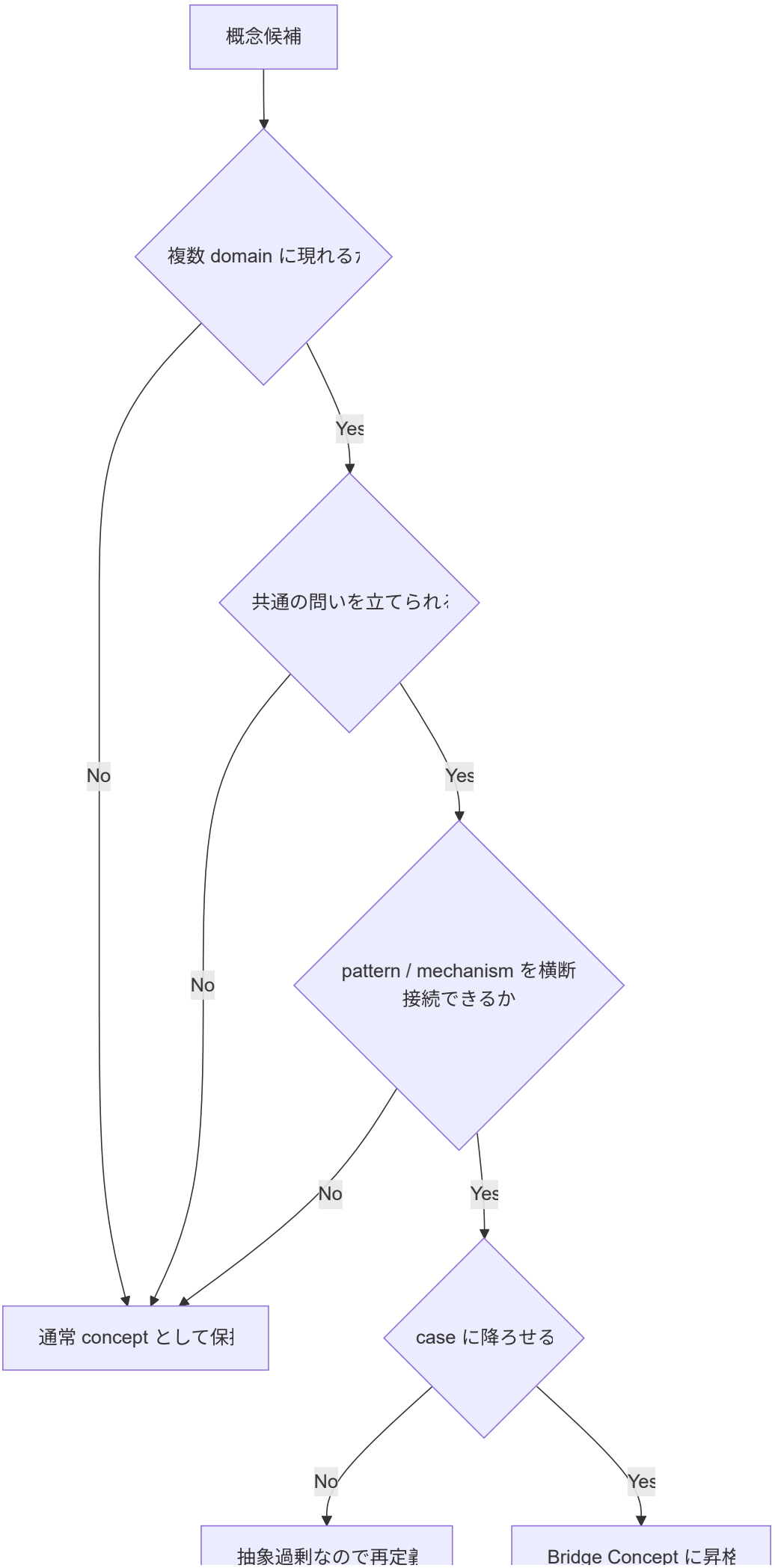
3. この概念で複数 case を比較できるか

4. この概念は pattern や mechanism を横断できるか

5. この概念は hub 間の導線になるか

6. 具体例に降ろせるか

Bridge Concept 判定フロー


概念候補
複数 domain に現れる
Yes
No
共通の問いを立てられ
Yes
No
pattern / mechanism を横断
接続できるか
No
Yes
通常 concept として保
case に降ろせる
No
Yes
抽象過剰なので再定
Bridge Concept に昇格

LLM にとっての意味

Bridge Concept が整っていると、LLM は

- 別 domain の知識を共通軸で取り寄せやすくなり
- 類比や応用を、表層語彙でなく構造で行いやすくなり
- hub 間を迷わず横断しやすくなり
- 新しい問いに対して再利用可能な枠組みを持ちやすくなる

つまり Bridge Concept は、
LLM にとっての **横断推論の橋脚** である。

この Vault における実装方針

この Vault では、Bridge Concept を意識的に整備すると、
各 domain OS がつながりやすくなる。

まず優先して整えるとよい候補:

- 権力
- 正統性
- 02_zettelkasten/02_world_model/concept/制約
- 信頼
- 02_zettelkasten/02_world_model/meta/model/social/information/シグナリング
- アイデンティティ
- 境界
- インセンティブ
- 02_zettelkasten/02_world_model/meta/model/social/information/情報非対称

- 責任

運用方針:

- hub に bridge node として明記する
- 代表 domain を本文に列挙する
- 各 bridge に最低2つ以上の anchor case をつける
- traversal 例を一つ以上書く

他ノートとの接続

上位

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

近接

- 02_zettelkasten/04_knowledge_graph/Traversal
- 02_zettelkasten/04_knowledge_graph/Reasoning Path
- 02_zettelkasten/04_knowledge_graph/Hub Design Rule
- 02_zettelkasten/04_knowledge_graph/Anchor Case
- 02_zettelkasten/04_knowledge_graph/Case to Pattern Promotion

下位候補

- Power as Bridge Concept
- Legitimacy as Bridge Concept
- Constraint as Bridge Concept
- Identity as Bridge Concept
- Trust as Bridge Concept

まとめ

Bridge Concept は、Knowledge Graph において
複数の domain・pattern・mechanism・case を横断的につなぐ中継概念である。

これにより、

- domain 島化を防ぎ
- 類比と応用を可能にし

- hub 間導線を強め
- LLM の横断推論を安定させる

ことができる。

個別知識が島なら、
Bridge Concept はそれらを結ぶ橋である。

Edge Type

Edge Type

Edge Type は、Knowledge Graph において
ノードとノードの間にどの種類の関係があるか を定義する構造である。

Knowledge Graph が単なる「つながっている図」で終わるか、
実際に reasoning に使える構造になるかは、
Node の数よりもむしろ **Edge の型づけ** に依存する。

Node Type が「それは何か」を定めるものだとすれば、
Edge Type は「それは何とどういう関係にあるか」を定めるものである。

定義

Edge Type とは、
Knowledge Graph におけるノード間関係の**意味的分類**である。

単なるリンクは「関係がある」ことしか示さない。
しかし reasoning に必要なのは、

- 上位下位関係なのか
- 部分全体関係なのか
- 原因結果関係なのか
- 具体例と抽象の関係なのか
- 適用関係なのか
- 対比関係なのか

を明確に区別することである。

Edge Type の目的は次の通りである。

1. ノード間関係の意味を固定する
2. 推論の方向を安定させる
3. 類似・因果・所属の混同を防ぐ
4. 上向き探索と下向き探索を可能にする
5. LLM が説明文を生成しやすくする

なぜ必要か

Edge が曖昧な Graph は、見た目は豊かでも実用性が低い。

たとえば次のリンクは全部違う。

- 韓国併合 → 植民地化
- 植民地化 → 帝国主義
- 情報非対称 → 逆選択
- 同調 → 集団意思決定
- 比較分析 → 歴史研究
- 信頼 → 裏切り

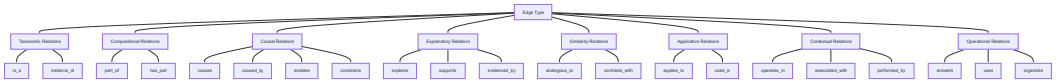
これらを全部「related_to」で済ませると、

- 何が原因か分からない
- 何が上位概念か分からない
- 何が具体例か分からない
- どう辿ればよいか分からない

結果として、Graph は検索補助にはなっても、
思考補助にはならない。

Edge Type は、知識を**辿れるもの**に変えるための条件である。

全体構造



Edge Type の基本原則

1. Edge は意味を持つ

リンクは単なる接続ではなく、
「どのような関係か」を明示する必要がある。

2. Edge には方向がある

多くの Edge は非対称である。

例:

- 情報非対称 causes 逆選択

- 逆選択 causes 情報非対称

この2つは同じではない。

3. Edge は推論の道になる

Graph は図ではなく経路である。
Edge は推論の移動方向を決める。

4. Edge は少数精鋭にする

関係語を増やしすぎると崩れる。
まずは少数の強い型で統一する。

5. Edge は Node Type と整合する

たとえば、

- Case instance_of Pattern
- Mechanism grounded in Kernel
- Method applies_to Domain

のように、使われやすい組み合わせがある。

中核 Edge Type

この Vault で最初に固定すべき中核 Edge は次の10個で十分である。

- is_a
- part_of
- instance_of
- causes
- caused_by
- explains
- supports
- contrasts_with
- analogous_to
- applies_to

これを中核語彙とする。

必要なら拡張語彙を後から追加する。

各 Edge Type の定義

1. is_a

分類関係。
A は B の一種である。

例:

- 植民地化 is_a 支配形態
- 同調 is_a 社会的影響
- 期待価値モデル is_a 意思決定モデル

用途:

- 上位概念に遡る
- 概念分類を安定させる
- 抽象化に使う

典型接続:

- Concept → Concept
- Pattern → Pattern
- Structure → Structure
- Model → Model

注意:

- 「一部である」ではない
- 「具体例である」でもない

---

2. part_of

部分関係。
A は B の構成要素である。

例:

- 動員装置 part_of 総力戦体制
- 問題定義 part_of 問題解決プロセス
- 承認段階 part_of 意思決定構造

用途:

- 全体構造を把握する
- 分解分析をする
- 構造ノートを作る

典型接続:

- Structure → Structure
- Step → Process
- Element → System

注意:

- is_a と混同しやすい
- A が B の種類なのか、部品なのかを区別する

---

3. instance_of

具体例関係。
A は B の具体的事例である。

例:

- 韓国併合 instance_of 植民地化パターン
- ある炎上案件 instance_of 規範違反可視化パターン
- ある会議崩壊事例 instance_of 責任回避パターン

用途:

- 具体から抽象へ上がる
- Case を Pattern や Concept に接続する
- 類例探索を可能にする

典型接続:

- Case → Pattern
- Event → Pattern
- Actor 行動 → Role Pattern
- Observation → Hypothesis Candidate

注意:

- instance_of は一回限りの所属ではなく、抽象との接続
- case を is_a でつなげない

4. causes

原因関係。
A が B を引き起こす。

例:

- 情報非対称 causes 逆選択
- 注意資源不足 causes 判断短絡
- 権力集中 causes 責任不透明化
- 過剰複雑化 causes 制度疲労

用途:

- 因果推論
- root cause analysis
- hypothesis 生成
- intervention 設計

典型接続:

- Kernel → Mechanism
- Mechanism → Outcome
- Structure → Mechanism
- Event → Event

注意:

- 相関を causes と書かない
- 因果が弱い場合は supports や associated_with を使う

---

5. caused_by

被原因関係。
A は B によって生じる。

例:

- 制度疲労 caused_by 過剰複雑化
- 不信拡大 caused_by 裏切り経験
- 意思決定遅延 caused_by 承認多段化

用途:

- 結果から原因を逆探索する
- 診断思考を補助する
- Why 系の問いに使う

典型接続:

- Outcome → Mechanism
- Failure → Constraint
- Event → Prior Event

注意:

- caused_by は causes の逆向き
- 両方を書きたくなるが、運用上は片側でもよい
- 検索性重視なら両方持つのも可

---

6. explains

説明関係。
A は B を説明する。

例:

- 限定合理性 explains 不完全意思決定
- アイデンティティ防衛 explains 認知的不協和の回避
- シグナリング理論 explains 学歴競争の一部

用途:

- 説明文生成
- 理論と現象の橋渡し
- 「なぜそう見えるのか」を記述する

典型接続:

- Kernel → Pattern
- Mechanism → Case
- Model → Phenomenon
- Concept → Distinction

注意:

- explains は causes より広い
- 説明はできても、厳密な原因ではない場合がある

---

7. supports

支援・根拠関係。
A は B を支える、または妥当性を補強する。

例:

- 複数事例群 supports 植民地化パターン
- 統計データ supports 仮説
- 観察記録 supports 問題定義
- 文献知見 supports 比較仮説

用途:

- エビデンスを結びつける
- 仮説の妥当性を補強する
- 議論の根拠を束ねる

典型接続:

- Observation → Hypothesis
- Data → Claim
- Case Cluster → Pattern
- Source → Interpretation

注意:

- supports は証明ではない
- evidence の強さには差がある

---

8. contrasts_with

対比関係。
A は B と重要な違いを持つ。

例:

- 正統性 contrasts_with 強制
- 市場調整 contrasts_with 計画統制

- 責任分散 contrasts_with 責任集中
- 予防型制度 contrasts_with 事後対処型制度

用途:

- 概念の輪郭を明確にする
- 誤同一視を防ぐ
- 比較思考を助ける

典型接続:

- Concept ↔ Concept
- Pattern ↔ Pattern
- Model ↔ Model
- Strategy ↔ Strategy

注意:

- 反対語でなくてもよい
- 重要な差異があることが核心

---

9. analogous_to

類比関係。
A は B と似た構造や作動を持つ。

例:

- 企業官僚制 analogous_to 国家官僚制
- fandom 炎上 analogous_to 異端審問
- 就活シグナル競争 analogous_to 市場での品質シグナル
- 中心周辺構造 analogous_to 本社支社支配構造

用途:

- 分野横断
- 発想転用
- 抽象構造の再利用

典型接続:

- Domain Pattern ↔ Domain Pattern
- Structure ↔ Structure
- Case ↔ Case

- Model ↔ Model

注意:

- 類比は同一性ではない
- 似ている面と違う面の両方を書くと強い

---

10. applies_to

適用関係。
A は B 領域や対象で有効である。

例:

- principal-agent problem applies_to 組織運営
- シグナリング applies_to 就活市場
- 権力分析 applies_to 歴史解釈
- 感情調整理論 applies_to 接客教育

用途:

- 抽象知を実務へ接続する
- 横断知を domain に落とす
- Tool / Method の使い道を示す

典型接続:

- Method → Domain
- Model → Domain
- Mechanism → Problem Type
- Tool → Task

注意:

- 「関係がある」だけでは applies_to にしない
- 実際に使えることが重要

拡張 Edge Type

中核10種で足りない場合、次を拡張語彙として使える。

---

11. has_part

全体側から部分を指す関係。

例:

- 総力戦体制 has_part 動員装置
- 営業プロセス has_part 提案設計

用途:

- part_of の逆方向
- 構造把握を両方向にする

---

12. enables

A が B を可能にする。
直接原因ではないが、成立条件を与える。

例:

- 共通規範 enables 協力
- インフラ整備 enables 市場拡大
- 可視化 enables 介入判断

用途:

- 条件分析
- causes より弱い因果
- 基盤条件の整理

---

13. constrains

A が B を制約する。
禁止ではないが、範囲を狭める。

例:

- 資源制約 constrains 行動選択
- 法制度 constrains 組織戦略
- 注意資源制約 constrains 学習速度

用途:

- 制約分析
- feasible set の把握
- 理想論への暴走防止

---

14. evidenced_by

A は B によって観察的に裏づけられる。

例:

- 仮説 evidenced_by インタビュー記録
- パターン evidenced_by 複数ケース
- 解釈 evidenced_by 文献引用

用途:

- supports をさらに証拠寄りにしたい時
- 調査ノートに便利

---

15. operates_in

A は B という構造・場・文脈で作動する。

例:

- 責任分散 operates_in 多段委任構造
- 同調形成 operates_in 集団環境
- フリーライダー化 operates_in 公共財状況

用途:

- Mechanism と Structure の接続
- 文脈依存性の可視化

---

16. associated_with

弱い関連関係。
まだ型が未確定だが、つながりは重要である。

例:

- ある観察 associated_with 不信感
- 特定時代背景 associated_with 国家主義高揚

用途:

- 仮置き
- 未確定リンク
- 仮説前段階

注意:

- 乱用すると Graph が濁る
- 後でより強い Edge に置換する前提で使う

---

17. performed_by

行為・役割・イベントが誰によって担われたかを示す。

例:

- 併合推進 performed_by 日本政府
- 仲介交渉 performed_by 外交官僚
- 煽動行為 performed_by 特定メディア

用途:

- Actor と Event / Role を結ぶ
- 誰が動いたかを明確にする

---

18. uses

Method や Tool が何を使うかを示す。

例:

- 比較分析 uses 比較軸
- case template uses actor/event/constraint
- reasoning engine uses question node

用途:

- 運用設計

- Tool 間接続
- Workflow 可視化

---

19. answers

A が B という問いに答える関係。

例:

- 植民地化パターン answers 韓国併合は植民地化か
- 限定合理性 answers なぜ人は最適に動かないのか

用途:

- Question node の接続
- AI retrieval の入口整理

---

20. organizes

Hub や Framework が何を束ねるかを示す。

例:

- Human Model Hub organizes 認知・感情・動機
- Pattern Hub organizes social pattern 群

用途:

- Meta ノードを整理する
- Hub の役割を明示する

Edge Type の群別整理

1. 分類関係

- is_a
- instance_of

2. 構成関係

- part_of
- has_part

3. 因果関係

- causes
- caused_by
- enables
- constrains

4. 説明・根拠関係

- explains
- supports
- evidenced_by

5. 類似・対比関係

- analogous_to
- contrasts_with

6. 適用・運用関係

- applies_to
- uses
- answers
- organizes

7. 文脈・主体関係

- operates_in
- performed_by
- associated_with

Edge Type の方向性

多くの Edge には方向がある。
方向を揃えると Graph が格段に使いやすくなる。

上向き方向

具体 → 抽象

例:

- 韓国併合 instance_of 植民地化パターン
- 責任回避事例 instance_of 責任回避パターン

下向き方向

抽象 → 具体

例:

- シグナリング applies_to 就活市場
- 限定合理性 explains 不完全意思決定

因果順方向

原因 → 結果

例:

- 情報非対称 causes 逆選択
- 権力集中 causes 責任不透明化

逆因果方向

結果 → 原因

例:

- 逆選択 caused_by 情報非対称
- 判断短絡 caused_by 注意資源不足

構成方向

部分 → 全体

例:

- 動員装置 part_of 総力戦体制
- 承認段階 part_of 意思決定構造

Edge Type の典型組み合わせ

Knowledge Graph では、Node Type と Edge Type の相性がある。

Case から出やすい Edge

- instance_of → Pattern
- explained_by → Mechanism
- performed_by → Actor
- associated_with → Context

Pattern から出やすい Edge

- is_a → 上位 Pattern
- explained_by → Mechanism
- applies_to → Domain
- contrasts_with → 他 Pattern

Mechanism から出やすい Edge

- causes → Outcome
- operates_in → Structure
- grounded_in 相当として → Kernel
- applies_to → Domain

Structure から出やすい Edge

- part_of → 上位 Structure
- enables → Mechanism
- constrains → Action

- analogous_to → 他 Structure

Kernel から出やすい Edge

- explains → Pattern / Mechanism
- constrains → Choice
- contrasts_with → 他 Kernel
- applies_to → 多数 Domain

Method / Tool から出やすい Edge

- applies_to → Domain / Task
- uses → Data / Schema
- answers → Question
- organizes → Node Cluster

良い Edge 設計の条件

1. 同じ関係語を継続的に使う

毎回別表現にすると Graph が壊れる。

悪い例:

- leads_to
- results_in
- produces
- creates

が混在する

よい例:

- 因果は causes に統一する

---

2. 強い関係と弱い関係を分ける

確信があるなら causes
まだ弱いなら supports / associated_with

---

3. 因果と説明を分ける

explanation は always cause ではない。

例:

- アイデンティティ防衛 explains 態度硬直化
  これは説明として有効だが、
  常に単一原因とは限らない。

---

4. 部分関係と種類関係を分ける

- 寡占構造 is_a 市場構造
- 市場集中度 part_of 競争構造評価

この区別が重要。

---

5. 類比を乱用しない

analogous_to は面白いが危険。
似ている次元を明示するとよい。

よくある誤用

1. is_a の乱用

「韓国併合 is_a 植民地化」は不自然。
これは instance_of の方が適切。

---

2. causes の乱用

相関や併存をすぐ causes にすると誤推論が増える。

---

3. part_of と instance_of の混同

- 承認段階 part_of 意思決定プロセス
- ある会議 instance_of 意思決定場面

これは別物。

---

4. related_to への逃避

便利だが、Graph の価値を下げやすい。
未確定なら associated_with を暫定的に使う方がまだよい。

---

5. Edge を増やしすぎる

関係語が20個を超えると管理が崩れやすい。
まずは中核10種を徹底する。

最小運用語彙

この Vault でまず固定するなら、以下を推奨する。

```
is_a
part_of
instance_of
causes
caused_by
explains
supports
contrasts_with
analogous_to
applies_to
```

拡張が必要になったら、以下を追加する。

```
enables
constrains
operates_in
performed_by
```

```
uses
answers
organizes
associated_with
has_part
evidenced_by
```

Edge Type 判定のための質問

新しいリンクを張るときは、次の質問で判定する。

1. これは種類関係か

→ is_a

2. これは部分関係か

→ part_of

3. これは具体例か

→ instance_of

4. これは原因か

→ causes

5. これは結果から見た原因か

→ caused_by

6. これは説明として効くか

→ explains

7. これは根拠になるか

→ supports / evidenced_by

8. これは対比で理解した方がよいか

→ contrasts_with

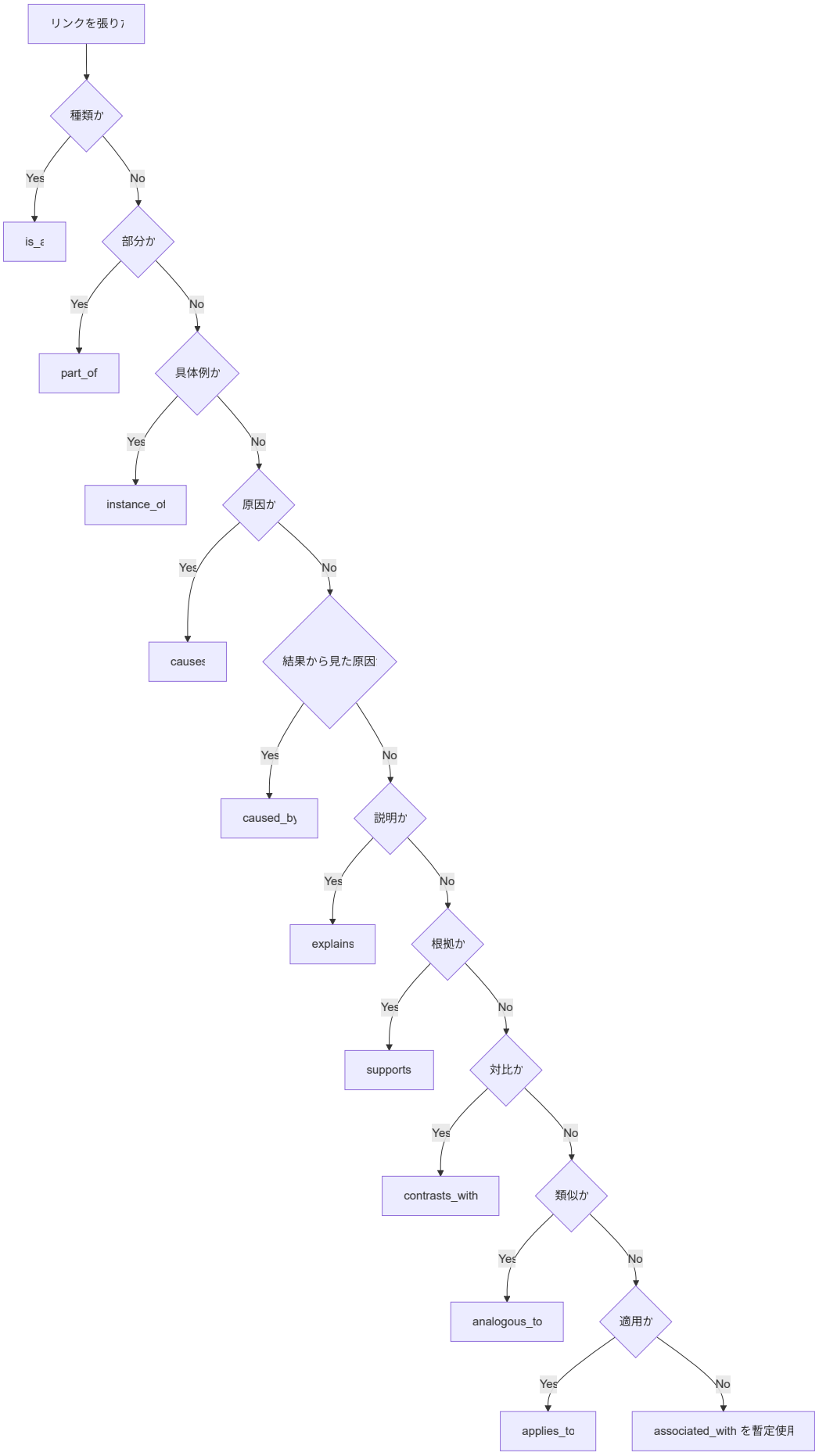
9. これは似た構造を持つか

→ analogous_to

10. これは別領域に使えるか

→ applies_to

Edge Type 判定フロー


リンクを張り
種類か
Yes
No
is_a
部分か
Yes
No
part_of
具体例か
Yes
No
instance_of
原因か
Yes
No
causes
結果から見た原因
Yes
No
caused_by
説明か
Yes
No
explains
根拠か
Yes
No
supports
対比か
Yes
No
contrasts_with
類似か
Yes
No
analogous_to
適用か
Yes
No
applies_to
associated_with を暫定使用

この Vault における実装方針

本文・property・Hub のいずれでもよいが、
最低限、本文中で relation の型が読めるようにする。

記述例:

- 韓国併合 は 植民地化パターン の **instance_of**
- 02_zettelkasten/02_world_model/meta/model/social/information/情報非対称 は 02_zettelkasten/02_world_model/model/social/incentive/逆選択 を **causes**
- 02_zettelkasten/02_world_model/meta/model/human/congnition/限定合理性 は 不完全意思決定 を **explains**
- 責任分散 は 多段委任構造 で **operates_in**
- 比較分析 は 歴史研究 に **applies_to**

これにより、LLM も人間も「どうつながっているか」を明確に読める。

Node Type との接続

上位

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

近接

- 02_zettelkasten/04_knowledge_graph/Node Type
- 02_zettelkasten/04_knowledge_graph/Traversal
- 02_zettelkasten/04_knowledge_graph/Graph Maintenance
- 02_zettelkasten/04_knowledge_graph/Hub Design Rule

下位候補

- Causal Relation
- Taxonomic Relation
- Part-Whole Relation
- Analogy Relation

- Applicative Relation

まとめ

Edge Type は、Knowledge Graph において
ノード同士の関係に**意味と方向**を与える構造である。

Edge Type が整うことで、

- Graph が単なるリンク集で終わらない
- 因果・分類・構成・類比を区別できる
- 抽象化と具体化が安定する
- LLM が reasoning path を辿りやすくなる
- 説明文が関係に沿って生成できる

知識の価値は、ノードの量だけでなく
どの型の Edge でつながっているか によって決まる。

Graph Maintenance

Graph Maintenance

Graph Maintenance は、Knowledge Graph を
壊れにくく、辿りやすく、推論可能な状態に保つための保守構造である。

Knowledge Graph は、一度作れば終わりではない。
ノートが増えるほど、関係は自然に劣化する。

- 重複ノードが増える
- 関係語が揺れる
- case が孤立する
- hub が肥大化する
- 似ているが違う概念が混線する
- 古い仮説が残り続ける

Graph Maintenance は、こうした劣化を抑え、
Vault 全体を**思考に耐える構造**として維持する。

定義

Graph Maintenance とは、
Knowledge Graph の node・edge・hub・schema・path を点検し、
構造の整合性・探索性・再利用性を保つための運用構造である。

目的は単なる掃除ではない。
主な目的は次の通りである。

1. 重複ノードを整理する
2. relation 語彙を正規化する
3. 孤立ノードを減らす
4. 弱いリンクと強いリンクを区別する
5. pattern や mechanism への昇格を判断する
6. hub と index の肥大化を防ぐ
7. LLM が安定して辿れる graph を保つ

なぜ必要か

Knowledge Graph は、作成時よりも運用中に崩れる。

典型的な崩れ方は次の通りである。

1. 重複

- 正統性
- 正統化
- 正当性
- legitimacy

のように、近いノードが乱立する。

2. 関係語の揺れ

- causes
- leads_to
- results_in
- creates

が混在し、因果探索が不安定になる。

3. 孤立

case は増えるが、pattern や mechanism に接続されない。

4. 肥大化

Hub が巨大化し、個別ノートの役割を食い潰す。

5. 古い仮説の残骸

仮説段階の弱リンクが、そのまま強い知識のように残る。

6. 抽象偏重または具体偏重

概念ばかり増えて現実に接地しない、あるいは事例ばかり増えて抽象化できない。

Graph Maintenance は、これらを防ぐための定常運用である。

全体構造

保守対象

Graph Maintenance の対象は大きく7つある。

1. Node

ノードそのものの重複・曖昧さ・過積載を点検する。

2. Edge

関係の型・方向・強さ・妥当性を点検する。

3. Hub

束ねノートの導線・肥大化・重複を点検する。

4. Schema

Node Type / Edge Type / Rule の一貫性を点検する。

5. Traversal

辿り方が実際に機能しているかを点検する。

6. Promotion

反復知を上位抽象へ昇格させる判断を行う。

7. Archive

古くなった仮説や置換済みノートを隔離する。

---

1. Node Maintenance

Node Maintenance は、
各ノードの粒度・名前・役割・重複状態を保守する。

1-1. Duplicate Merge

意味がほぼ同じノードを統合する。

例:

- 正統性

- 正当性
- legitimacy

対処:

- 代表ノードを1つ決める
- 他ノードは統合・転送・リンク整理する
- 表記揺れは alias 的に扱う

判断基準:

- 中心概念が同じか
- edge の接続先が大きく重なるか
- 今後分ける理由があるか

1-2. Rename / Normalize

名前が曖昧・長すぎる・揺れているノードを正規化する。

悪い例:

- いろいろな正統性の問題について
- 正統性って大事だよね
- legitimation process maybe

良い例:

- 正統性
- 正統化メカニズム
- 正統性危機パターン

基準:

- 何の node type か見えやすい
- 他ノードと混線しにくい
- 再利用しやすい

1-3. Split Overloaded Node

一つのノートに複数役割が混在していたら分割する。

典型例:

- concept + mechanism + case が同居している
- hub + 本文解説 + index が混ざっている
- pattern 集と rule が同じノートにある

分割原則:

- 一ノート一中心役割
- role が違うなら分ける
- hub は hub、case は case として独立させる

1-4. Repair Isolated Node

孤立ノードを修復する。

孤立の典型:

- case があるが pattern に上がっていない
- concept があるが対比も例もない
- mechanism があるが kernel や structure に接続されていない

修復方針:

- 少なくとも1つ上位へ接続
- 少なくとも1つ横方向へ接続
- 必要なら example か contrast を追加

---

2. Edge Maintenance

Edge Maintenance は、
relation の型・方向・強さ・不要性を点検する。

2-1. Relation Normalize

relation 語彙を中核語彙へ揃える。

揺れの例:

- causes / leads_to / creates / results_in
- kind_of / type_of / is_a
- example_of / instance_of

方針:

- 因果は causes
- 分類は is_a
- 具体例は instance_of
- 部分は part_of

これにより traversal が安定する。

2-2. Strength Review

relation の強さを見直す。

誤りの例:

- ただの共起を causes にしている
- 仮説段階なのに explains と断定している
- 弱い関係なのに supports とせず causes にしている

見直し基準:

- 因果証拠があるか
- 反例が多いか
- 説明としてしか使えないか
- 仮説に留めるべきか

必要なら

- causes → supports
- explains → associated_with
  のように弱める。

2-3. Broken Direction Fix

relation の向きの誤りを直す。

例:

- 植民地化パターン instance_of 韓国併合
- 逆選択 causes 情報非対称

などの方向逆転を修正する。

2-4. Dead Link Prune

機能していない、意味の薄い、古いリンクを切る。

対象:

- 過去の仮リンク
- 説明に使われない弱関連
- もはや上位ノードへ統合済みのリンク

注意:

- 削除前に archive 化するか、変更履歴を残すとよい

---

3. Hub Maintenance

Hub Maintenance は、
hub が「便利な入口」であり続けるように保守する。

3-1. Hub Slimming

Hub が肥大化したら痩せさせる。

肥大化の兆候:

- ノート本文が長すぎる
- 個別概念の中身まで hub に書いてある
- 一覧と解説と rules が全部入っている
- 個別ノートを見なくても hub だけで済んでしまう

対処:

- hub は地図に戻す
- 内容は個別ノートへ分離
- index や rule を別ノート化する

3-2. Index Separation

一覧性が強い部分は index に切り出す。

hub の役割:

- 関係の見取り図
- 読み順
- 重要経路

index の役割:

- 網羅一覧
- ID 管理
- 形式的な並び

この分離が重要。

3-3. Route Clarification

Hub の読む順序・導線を明示する。

よい hub は、

- どこから読むか
- 何を経由するとよいか
- どのノードが橋になるか
  を示す。

---

4. Schema Maintenance

Schema Maintenance は、
Vault 全体の型定義が崩れていないかを点検する。

4-1. Node Type Review

各ノートの node type 判定を見直す。

例:

- pattern のつもりで作ったが実は case
- concept のつもりだが mechanism
- framework と hub が混ざっている

点検基準:

- そのノートの主役は何か
- どの edge が自然につながるか
- 他ノートとの役割差があるか

4-2. Edge Type Review

relation が node type と整合しているかを点検する。

例:

- case is_a pattern は不自然
- concept part_of case は不自然なことが多い
- method causes domain はズレやすい

4-3. Property Consistency

property や frontmatter が揃っているかを見る。

例:

- note_type
- layer
- structure_type
- status
- relation 記述方式

これは LLM と人間の両方に効く。

---

5. Traversal Maintenance

Traversal Maintenance は、
実際に Graph を辿ったときに詰まらないかを点検する。

5-1. Reasoning Path Check

主要な問いについて、実際に path が通るか確認する。

例:

- なぜ組織は責任回避するのか
- 韓国併合は何の一例か
- 限定合理性は就活でどう現れるか

点検内容:

- Question から必要ノードに届くか
- concept だけで止まっていないか
- case へ降りられるか

5-2. Traversal Bottleneck Check

一部ノードに依存しすぎていないかを見る。

典型 bottleneck:

- 権力
- 制度
- 合理性
- identity

これらは重要だが、曖昧なままだと全探索が濁る。

5-3. Cross Domain Route Review

分野横断経路が実際にあるかを見る。

例:

- psychology → business
- history → organization
- law → operations

もし route が細すぎるなら、
bridge concept を増やす必要がある。

---

6. Promotion Maintenance

Promotion Maintenance は、
繰り返し現れる知識を一段上の抽象へ昇格させるための保守である。

6-1. Case to Pattern Promotion

複数 case に共通形があるなら pattern 化を検討する。

条件:

- 類似 case が複数ある
- 作動順序や特徴が繰り返す
- 他分野にも応用できそう

例:

- 複数の炎上事例
  → 規範違反可視化パターン

6-2. Pattern to Mechanism Promotion

pattern の背後に作動過程が明確なら mechanism 化する。

例:

- 責任回避パターン
  → 責任分散メカニズム

6-3. Concept to Kernel Promotion

概念がより根本原理として機能しているなら kernel 化を検討する。

例:

- 制約
  → 資源制約原理
- 信頼
  → 不確実性下の協力原理

---

7. Archive Maintenance

Archive Maintenance は、
不要になったが捨てるには惜しい知識を退避させる。

7-1. Deprecated Note Archive

統合・置換済みノートを archive 化する。

例:

- 古い表記揺れノート
- 旧 version の framework
- 役割を失った仮ノート

7-2. Weak Hypothesis Archive

弱仮説や未検証リンクを別置きにする。

理由:

- graph 本体を濁らせない
- ただしアイデア資産は捨てない

7-3. Superseded Version Archive

新しい版に置き換えられた旧版を隔離する。

これにより、

- 現行 graph の見通しが良くなる
- 過去の思考経路も保存できる

保守の頻度

Graph Maintenance は毎回フルでやる必要はない。
頻度を分けると運用しやすい。

---

1. 軽保守

新ノート追加時に行う。

内容:

- node type 確認
- relation 語彙確認
- 上位接続1本
- 横接続1本

---

2. 中保守

週次または一定数追加ごとに行う。

内容:

- duplicate 点検
- isolated case 点検
- hub の肥大確認
- weak link 見直し

---

3. 重保守

月次または大規模再編時に行う。

内容:

- schema 見直し
- folder / hub 再編
- promotion 実施
- archive 整理
- traversal 実地確認

点検チェックリスト

Node 点検

- 重複していないか

- 名前が明確か
- role が一つに定まっているか
- 孤立していないか

Edge 点検

- relation 語彙が統一されているか
- 向きが正しいか
- 強すぎる断定になっていないか
- 死にリンクがないか

Hub 点検

- 地図として機能しているか
- 目次と本文が混ざっていないか
- 読み順が示されているか

Traversal 点検

- Question から必要ノードへ届くか
- 抽象と具体を往復できるか
- bottleneck が曖昧すぎないか

Promotion 点検

- 繰り返し case を pattern 化できないか
- pattern の背後 mechanism を立てられないか
- concept が kernel 化していないか

良い保守状態の条件

1. 孤立 case が少ない

2. relation 語彙が安定している

3. hub が太りすぎていない

4. abstract と concrete が往復可能

5. 主要な問いに reasoning path がある

6. 古い仮説が graph 本体を濁らせていない

悪い保守状態の兆候

1. 同じ意味のノートが増える

2. 関係語が毎回違う

3. hub だけが巨大化する

4. case が並ぶだけで抽象化されない

5. 概念ばかりで具体例がない

6. 問いから辿ると毎回違うルートになる

LLM にとっての意味

Graph Maintenance がないと、LLM は

- 似たノートを重複参照し
- relation の揺れに引っ張られ
- 弱い仮説を強い知識と誤認し
- 問いごとに不安定な path を辿りやすくなる

逆に Maintenance が効いていると、

- 代表ノードに集約され
- 中核 relation で辿れ
- 抽象化と具体化が安定し
- reasoning の質が揺れにくくなる

つまり Graph Maintenance は、
LLM の思考基盤の**精度管理**である。

この Vault における実装方針

最低限、次の運用を固定するとよい。

新ノート作成時

- node type を明示する

- 代表 relation 語彙を使う
- 上位・横・具体例のいずれかに接続する

週次見直し

- 重複ノード確認
- isolated case 確認
- hub 肥大確認

月次見直し

- promotion 候補洗い出し
- archive 移動
- 주요 question の traversal 実験

他ノートとの接続

上位

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

近接

- 02_zettelkasten/04_knowledge_graph/Node Type
- 02_zettelkasten/04_knowledge_graph/Edge Type
- 02_zettelkasten/04_knowledge_graph/Traversal
- 02_zettelkasten/04_knowledge_graph/Case to Pattern Promotion
- 02_zettelkasten/04_knowledge_graph/Hub Design Rule
- 02_zettelkasten/04_knowledge_graph/Reasoning Path

下位候補

- Duplicate Merge Rule
- Relation Normalize Rule
- Isolated Node Repair
- Weak Hypothesis Archive
- Promotion Review Checklist

まとめ

Graph Maintenance は、Knowledge Graph を
生きた思考インフラとして維持するための保守構造である。

それは単なる整理ではなく、

- node の重複を防ぎ
- edge の意味を揃え
- hub の肥大を抑え
- traversal を通るようにし
- 繰り返し知を昇格させ
- 古い知識を隔離する

ための定常運用である。

Knowledge Graph を作ることが建設だとすれば、
Graph Maintenance はその都市計画と保全である。

Node Type

Node Type

Node Type は、Knowledge Graph において
何を一つのノードとして扱うかを定義するための分類構造である。

Knowledge Graph が壊れる最大の原因の一つは、
「概念」「事例」「原理」「構造」「問い」「手法」が未分化のまま、
全部同じ粒度で並べられることである。

Node Type はそれを防ぎ、
各ノートが **どの役割の知識単位なのか** を明確にする。

定義

Node Type とは、
Knowledge Graph を構成する各ノードの**知識上の役割分類**である。

Node Type の目的は、単なる整理ではない。
主な目的は次の5つである。

1. ノードの役割を明確にする
2. 異なる種類の知識を混同しない
3. どの関係を結ぶべきかを判断しやすくする
4. 抽象と具体の移動を安定させる
5. LLM の探索経路を制御しやすくする

なぜ必要か

同じ「ノート」でも、知識上の役割は全く違う。

たとえば次は同列ではない。

- 帝国主義
- 韓国併合
- 非対称支配メカニズム
- 植民地化パターン
- 国家とは何か
- 比較史分析

- なぜ日本は韓国を併合したのか

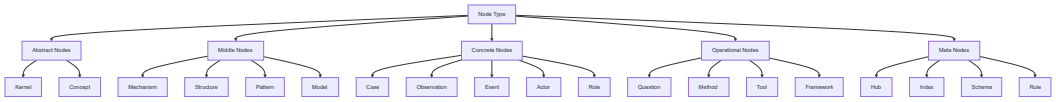
これらを全部同じノードとして扱うと、

- 概念と事例が混ざる
- 質問と答えが混ざる
- 方法と対象が混ざる
- 原理と現象が混ざる

その結果、Graph は見た目だけつながっていても、
推論には使えない。

Node Type は、
「何をどのレイヤーの知識として置くか」を決めるための土台である。

全体構造



Node Type の大分類

Node Type は大きく5群に分けると安定する。

1. Abstract Nodes

高抽象の知識単位。
原理・意味・普遍概念を担う。

- Kernel
- Concept

2. Middle Nodes

抽象と具体の中間。
「どう動くか」「どう組まれているか」「どう繰り返すか」を担う。

- Mechanism
- Structure
- Pattern
- Model

3. Concrete Nodes

現実に接地した知識単位。
観察・事例・人物・出来事を担う。

- Case
- Observation
- Event
- Actor
- Role

4. Operational Nodes

問い・分析・実務への接続を担う。

- Question
- Method
- Tool
- Framework

5. Meta Nodes

知識体系の運用そのものを支える。

- Hub
- Index
- Schema
- Rule

各 Node Type の定義

1. Kernel

最も深い原理ノード。
多数の現象や構造の背後にある、比較的普遍的な土台を表す。

例:

- 限定合理性
- 資源制約
- 情報非対称
- 権力偏在
- 制度は行動を方向づける

特徴:

- 抽象度が高い
- 長期的に有効
- 多くの mechanism / pattern を支える

向いている問い:

- 背後にある原理は何か
- なぜ繰り返し似た現象が起こるのか

---

2. Concept

重要概念だが、まだ原理や仕組みとしては確定していない意味単位。

例:

- 正統性
- 主権
- 競争
- 信頼
- 責任
- 物語
- 移行

特徴:

- 意味の輪郭を与える
- 他ノードへの入口になる
- 曖昧さを管理する

向いている問い:

- この言葉は何を意味するか
- 近い概念とどう違うか

---

3. Mechanism

結果を生む作動過程。
「何がどう作用して、何が起きるか」を説明する。

例:

- 同調形成
- シグナリング
- フリーライダー化
- 注意の偏在
- 責任分散
- 動機づけ歪み

特徴:

- 因果的
- 動態的
- 説明力が高い

向いている問い:

- 何がどう動いてこうなったのか
- 結果に至る作動過程は何か

---

4. Structure

要素の配置・階層・接続関係。
動いていなくても、現象を方向づける骨格を表す。

例:

- 中心周辺構造
- 寡占構造
- 主従構造
- ネットワーク構造
- 委任連鎖
- フィードバックループ

特徴:

- 静的または準静的
- 位置関係を重視する
- ボトルネック分析に強い

向いている問い:

- どういう配置だから起きやすいのか
- どこに偏りや詰まりがあるのか

5. Pattern

繰り返し現れる典型形。
特定分野や複数分野で反復される現象の型。

例:

- 炎上パターン
- 責任回避パターン
- 外部敵形成パターン
- 制度疲労パターン
- 立ち上がり失敗パターン

特徴:

- 反復性がある
- case を束ねる
- 予測のたたき台になる

向いている問い:

- これはよくある型か
- 他にも似た例があるか

---

6. Model

複数の concept / mechanism / structure / pattern を統合した説明枠組み。

例:

- 行動モデル
- 国家形成モデル
- 学習モデル
- 営業プロセスモデル
- 認知処理モデル

特徴:

- 複合的
- ある範囲を体系的に説明する
- 単独ノードより大きいが、Hub よりは内容中心

向いている問い:

- この領域をどうまとめて理解するか
- どの構成要素がどうつながるか

---

7. Case

具体的事例ノード。
特定の現実事象・歴史事例・案件・プロジェクトなどを扱う。

例:

- 韓国併合
- ドイツ革命
- 特定企業の炎上
- 特定案件の失敗
- ある制度改正

特徴:

- 具体的
- 時間・場所・主体を持つ
- 抽象化の素材になる

向いている問い:

- 実際には何が起きたのか
- この出来事は何の例か

---

8. Observation

まだ十分に解釈されていない観察単位。
生データと case の中間に位置する。

例:

- 発言の記録
- 現場で見た挙動
- 数値の偏り
- 質的インタビューの一断片

特徴:

- 具体的だが未整理
- case の材料になる
- hypothesis の起点になる

向いている問い:

- 何が観察されたか
- どんな異変や兆候があるか

---

9. Event

時間的に区切られた出来事ノード。
Case よりも出来事単位に焦点を当てる。

例:

- クーデター発生
- 法律制定
- 炎上発火
- 会議決裂
- 価格暴落

特徴:

- 時点・時系列が重要
- Event 同士で連鎖する
- case の中の部分としても使える

向いている問い:

- 何がいつ起きたか
- どの出来事が転換点か

---

10. Actor

行為主体ノード。
個人・組織・国家・集団など、行為や判断の担い手。

例:

- ビスマルク
- 日本政府

- 消費者集団
- 経営陣
- 官僚制組織

特徴:

- 意思・資源・制約を持つ
- role と分けると整理しやすい
- event / case への埋め込み先になる

向いている問い:

- 誰が動いたのか
- 誰にどんな利害があったか

---

11. Role

Actor が特定文脈で担う役割ノード。
主体そのものではなく、関係上の位置を表す。

例:

- 仲介者
- 監督者
- 支配者
- 被支配者
- 反対派
- 正統性供給者

特徴:

- Actor と切り分けると再利用しやすい
- 抽象比較に向く
- 組織・政治・物語分析で有用

向いている問い:

- この主体はその場でどんな役割を担っていたか
- 他事例でも同じ役割はあるか

---

12. Question

問いノード。
調査・推論・読書・設計の入口を担う。

例:

- なぜ人は不合理な選択を続けるのか
- なぜ制度は自己目的化するのか
- なぜこの事業は立ち上がらないのか

特徴:

- Graph の起点になる
- 関連ノード群を束ね直せる
- AI にとって retrieval の入口になる

向いている問い:

- 何を知りたいのか
- どこから考え始めるべきか

---

13. Method

分析・思考・調査のやり方ノード。

例:

- causal chain analysis
- 比較分析
- five whys
- power mapping
- 構造分解

特徴:

- 対象そのものではなく、扱い方
- Question と対象ノードを橋渡しする

向いている問い:

- どう分析すればよいか
- どの順で考えればよいか

---

14. Tool

実務で使うテンプレート・チェックリスト・フォーム・プロンプト。

例:

- case 記述テンプレート
- 観察シート
- 問題切り分け表
- KPI チェックリスト

特徴:

- 即使える
- Method より実装寄り
- 知識を作業に変える

向いている問い:

- 実際に何を書けばいいか
- どの形式で記録するか

---

15. Framework

複数の視点や手順を整理した枠組みノード。
Method と Model の中間に位置することが多い。

例:

- 3層分析フレーム
- 観察→仮説→検証フレーム
- 顧客課題→提案設計フレーム

特徴:

- 方法論の見取り図
- 操作順序を与える
- 複数 Method を束ねることが多い

---

16. Hub

ノード群を束ねる中枢ノード。
単なる目次ではなく、関係の見取り図を示す。

例:

- Human Model Hub
- Social Structure Hub
- Knowledge Graph Hub
- Pattern Hub

特徴:

- Graph の可読性を上げる
- 読む順序を与える
- 中距離探索を助ける

---

17. Index

機械的・一覧的に並べる補助ノード。
Hub より意味づけが薄い。

例:

- ノート一覧
- ID 一覧
- folder index

特徴:

- 管理寄り
- 検索補助
- 推論の中心にはしない

---

18. Schema

型や項目の定義ノード。
Graph の設計原理を定める。

例:

- Node Type
- Edge Type

- Property Rule
- Case Template Schema

特徴:

- メタレベル
- 運用を揃える
- Vault 全体の安定化に寄与する

---

19. Rule

書き方・昇格条件・命名規則などの運用ルール。

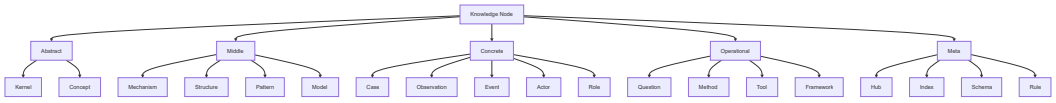
例:

- Case の書き方ルール
- Pattern 昇格条件
- Hub 設計ルール
- relation 語彙ルール

特徴:

- 品質維持
- ばらつき防止
- LLM の読み取り安定化

Node Type の階層関係



抽象度の並び

Knowledge Graph を安定させるには、
ノードを抽象度順に見る癖が重要である。

おおむね次の順になる。

1. Kernel

2. Concept
3. Mechanism / Structure / Pattern / Model
4. Case / Event / Observation / Actor / Role
5. Tool / Method / Framework
6. Hub / Schema / Rule / Index

ただしこれは厳密な上下関係ではなく、
「知識上の機能の違い」として理解する。

どう使い分けるか

Concept と Kernel の違い

- Concept は「重要な意味単位」
- Kernel は「背後にある根本原理」

例:

- 信頼 = Concept
- 情報不完備下で協力が不安定になる = Kernel

Mechanism と Structure の違い

- Mechanism は「どう動くか」
- Structure は「どう組まれているか」

例:

- 責任分散 = Mechanism
- 多段委任構造 = Structure

Pattern と Case の違い

- Pattern は「繰り返す型」
- Case は「具体的事例」

例:

- 炎上パターン = Pattern

- ある企業Xの炎上 = Case

Actor と Role の違い

- Actor は「誰か」
- Role は「その文脈で何を担うか」

例:

- 日本政府 = Actor
- 併合推進主体 = Role

Method と Tool の違い

- Method は「どう考えるか」
- Tool は「どう書くか・どう使うか」

例:

- 比較分析 = Method
- 比較表テンプレート = Tool

Node Type 判定のための質問

新しいノートを作るときは、次の質問で判定できる。

1. これは具体例か

はい → Case / Event / Observation / Actor

2. これは繰り返し現れる型か

はい → Pattern

3. これは結果を生む作動過程か

はい → Mechanism

4. これは要素の配置や接続か

はい → Structure

5. これは広く効く原理か

はい → Kernel

6. これは重要概念の意味整理か

はい → Concept

7. これは問いの入口か

はい → Question

8. これは分析のやり方か

はい → Method

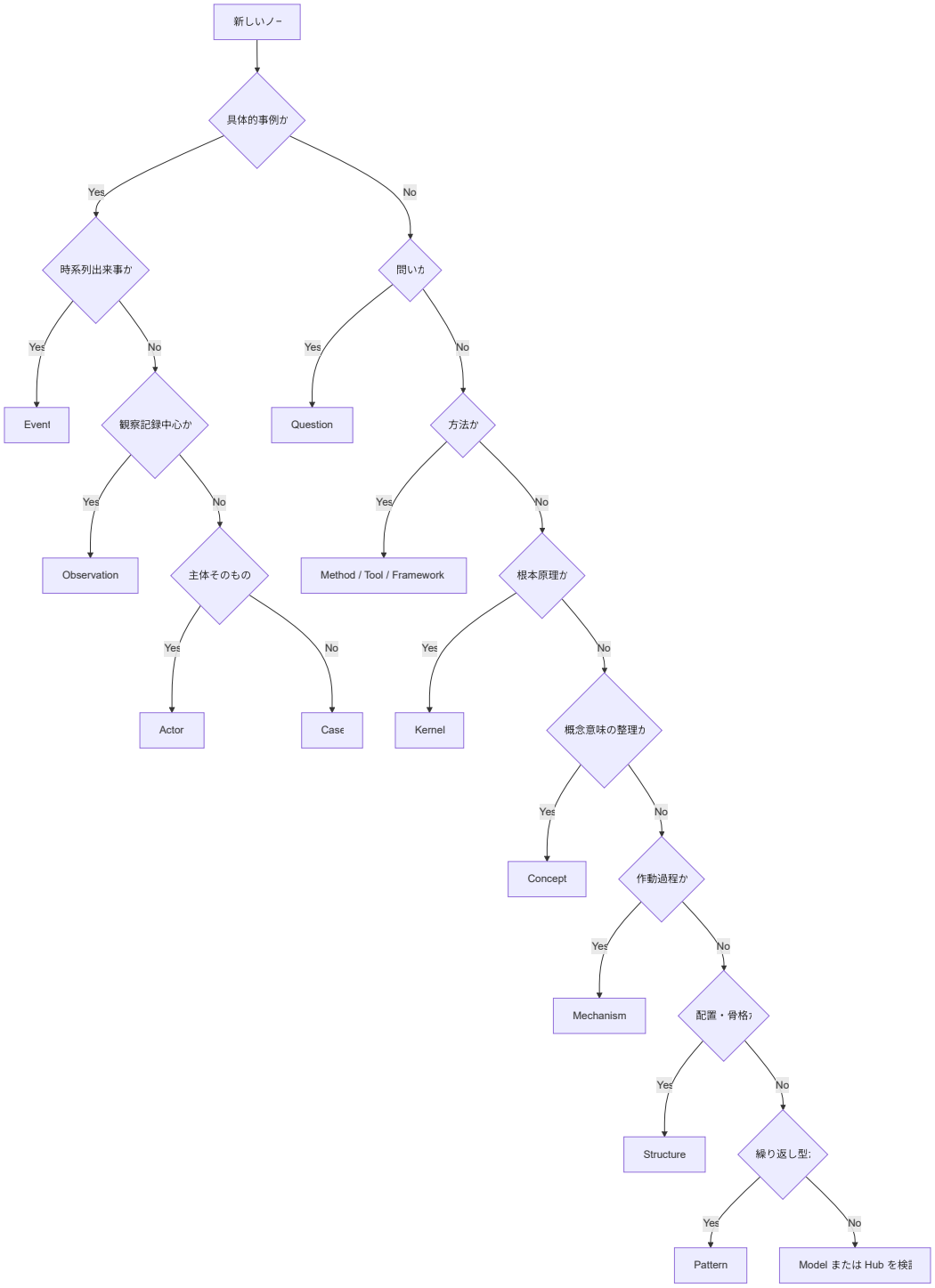
9. これはテンプレートや実務道具か

はい → Tool

10. これは一覧や束ねノートか

はい → Hub / Index / Schema / Rule

Node Type 判定フロー



良い Node Type 設計の条件

1. 一つのノートに一つの主役

一ノートに mechanism と case と concept を全部詰め込まない。

2. 似た役割を混ぜない

特に Concept / Kernel / Mechanism / Pattern / Case を混ぜない。

3. 具体から抽象へ上がれる

Case を置いたら、少なくとも Pattern か Mechanism へつなぐ。

4. 抽象から具体へ降りられる

Kernel や Concept から、関連 Case を辿れるようにする。

5. Meta Node を増やしすぎない

Hub, Schema, Rule ばかり増えると中身の知識が痩せる。

悪い Node Type の典型

1. 何でも Concept にしてしまう

概念ノートが膨張し、仕組みや事例が埋もれる。

2. Case を Pattern と呼んでしまう

一回きりの事象を一般化しすぎる。

3. Mechanism と Structure の未分離

「責任分散」と「多段承認構造」が混ざる。

4. Actor と Role の混同

人そのものと、その場の機能が区別できなくなる。

5. Hub が内容ノートを食う

Hub が巨大化し、本来の個別ノートが不要になる。

この Vault における実装方針

この Vault では、最低でも次の Node Type を中核に据えると安定する。

- Kernel
- Concept
- Mechanism
- Structure
- Pattern
- Case
- Question
- Method
- Tool
- Hub

補助的に必要に応じて追加する。

- Observation
- Event
- Actor
- Role
- Model
- Rule
- Schema
- Framework
- Index

最小セットと拡張セット

最小セット

最初はこれだけで十分回る。

- Concept
- Kernel
- Mechanism
- Structure
- Pattern
- Case
- Question
- Method
- Tool
- Hub

拡張セット

運用が進んだら追加する。

- Observation
- Event
- Actor
- Role
- Model
- Framework
- Rule
- Schema
- Index

他ノートとの接続

上位

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

近接

- 02_zettelkasten/04_knowledge_graph/Edge Type
- 02_zettelkasten/04_knowledge_graph/Traversal
- 02_zettelkasten/04_knowledge_graph/Graph Maintenance
- 02_zettelkasten/04_knowledge_graph/Hub Design Rule

下位候補

- Kernel
- Concept
- Mechanism
- Structure
- 02_zettelkasten/04_knowledge_graph/Pattern
- Case
- Question
- Method
- Tool

まとめ

Node Type は、Knowledge Graph において
「このノートは何者か」を定義するための基本分類である。

Node Type を明確にすることで、

- 抽象と具体が混ざらない
- 関係の型が決めやすい
- Hub が機能しやすい
- LLM の探索経路が安定する
- Vault 全体の知識設計が崩れにくくなる

知識の質は、内容量だけでなく
どの役割のノードとして置かれているか に強く依存する。

Traversal

Traversal

Traversal は、Knowledge Graph において
どのノードからどのノードへ、どの順序と規則で辿るか を定義する構造である。

Knowledge Graph が存在しても、Traversal が設計されていなければ、
知識は「つながっているだけ」で終わる。
逆に Traversal が明確であれば、Graph は検索補助を超えて、
推論・比較・抽象化・設計 の基盤として機能する。

Node Type が「何があるか」を定め、
Edge Type が「どうつながるか」を定めるなら、
Traversal は「どう辿るか」を定める。

定義

Traversal とは、
Knowledge Graph 上で知識ノードを**意味のある順路で移動するための探索構造**である。

単なるグラフ探索ではなく、次のような知的目的に応じて経路を選ぶ。

- 具体から抽象へ上がる
- 抽象から具体へ降りる
- 原因から結果へ辿る
- 結果から原因へ戻る
- 類似事例へ横断する
- 問いから説明候補へ絞る
- 方法から適用領域へ接続する

Traversal の目的は次の通りである。

1. Graph の探索方向を定義する
2. 問いに応じた経路を選択できるようにする
3. 無関係なノードへの拡散を防ぐ
4. reasoning path を再現可能にする
5. LLM の retrieval と explanation を安定させる

なぜ必要か

Knowledge Graph が大きくなると、問題は「情報不足」ではなく
探索の迷子 になることである。

典型的には次のような崩れ方をする。

1. 近い単語ばかり拾って本質に届かない
2. 具体事例から抽象へ上がれない
3. 抽象概念が具体に降りない
4. 因果と類似を混同する
5. 枝分かれしすぎて結論に辿り着かない
6. 毎回違う経路を辿って回答が揺れる

Traversal はこれを防ぐ。
つまり Traversal は、Knowledge Graph を
歩ける地図 にするための構造である。

全体構造

Traversal の基本原則

1. 探索には目的がある

Traversal は「何でも多く辿る」ことではない。
問いに応じて最短で意味のある経路を選ぶことが重要である。

2. Edge Type に従って進む

Traversal は Node の近さではなく、
関係の型 に従って進む。

3. 抽象と具体を往復できることが重要

上がるだけでも、降りるだけでも不十分。
往復可能性が Graph の質を決める。

4. 深く掘る方向と広げる方向を分ける

原因探索のように深く掘る動きと、
類似事例を広げる動きは別である。

5. Traversal は止まる条件が必要

どこで探索を打ち切るかがないと、
Graph は連想ゲームになる。

Traversal の大分類

1. Vertical Traversal

抽象と具体のあいだを上下する探索。

2. Causal Traversal

原因と結果の連鎖を辿る探索。

3. Structural Traversal

部分と全体、配置と作動の関係を辿る探索。

4. Lateral Traversal

類似・対比・分野横断によって横に移る探索。

5. Question Driven Traversal

問いを起点に、答え候補へ絞っていく探索。

6. Operational Traversal

Method, Tool, Framework を実務や task に接続する探索。

7. Meta Traversal

Hub, Schema, Rule を使って Graph の歩き方自体を制御する探索。

---

1. Vertical Traversal

Vertical Traversal は、
具体から抽象へ上がる、または
抽象から具体へ降りる 探索である。

Knowledge Graph の中核はこの往復にある。

1-1. Abstract Upward

具体事例から上位の pattern / mechanism / kernel へ上がる。

典型経路:

- Case → Pattern
- Event → Pattern
- Pattern → Mechanism
- Mechanism → Kernel
- Concept → Kernel

例:

- 韓国併合
- 植民地化パターン
- 非対称支配メカニズム
- 権力偏在原理

用途:

- 事例の一般化
- 本質把握
- 理論化
- case の意味づけ

1-2. Concrete Downward

抽象概念や原理から、具体的事例や適用先へ降りる。

典型経路:

- Kernel → Mechanism
- Mechanism → Pattern
- Pattern → Case
- Model → Domain Case
- Concept → Example

例:

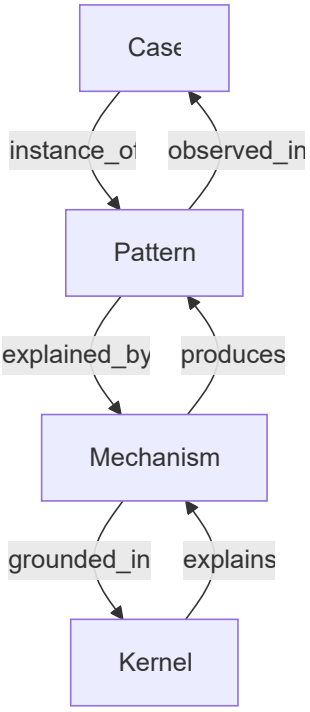
- 限定合理性
- 判断短絡メカニズム
- 誤判断パターン

- 就活時の企業選択失敗事例

用途:

- 抽象概念の理解
- 教材化
- 実務適用
- 例示

Vertical Traversal の図



---

2. Causal Traversal

Causal Traversal は、
原因と結果の流れを辿る探索である。

これは problem solving、historical analysis、organizational diagnosis で特に重要。

2-1. Cause to Result

原因から結果へ順方向に辿る。

典型経路:

- Constraint → Mechanism
- Mechanism → Event
- Event → Outcome
- Outcome → Secondary Effect

例:

- 情報非対称
- 逆選択
- 品質の悪い候補の流入
- 市場の信頼低下

用途:

- 将来予測
- 影響分析
- 施策の副作用確認
- シナリオ設計

2-2. Result to Cause

結果から原因へ逆向きに辿る。

典型経路:

- Failure → caused_by Mechanism
- Mechanism → caused_by Structure
- Structure → caused_by Constraint / Incentive

例:

- 意思決定遅延
- 承認多段化
- 責任分散
- リスク回避文化

用途:

- root cause analysis
- 診断
- なぜなぜ分析の補強
- 問題特定

2-3. Condition Trace

直接原因ではなく、成立条件や制約条件を辿る。

典型経路:

- Outcome → enabled_by Condition
- Mechanism → operates_in Structure
- Action → constrained_by Resource / Rule

例:

- 同調形成
- 集団可視性
- 共通規範
- 異論コストの高さ

用途:

- 「なぜ可能だったか」を見る
- 単純因果では拾えない背景条件を拾う
- 制度設計に使う

Causal Traversal の図

---

3. Structural Traversal

Structural Traversal は、
要素の配置、階層、部分全体、作動場所を辿る探索である。

Mechanism だけを見ると「動き」は分かるが、
どの配置だからそう動くか が見えなくなる。
Structural Traversal はそれを補う。

3-1. Part to Whole

部分から全体へ上がる。

典型経路:

- Step → Process
- Element → Structure
- Unit → System

例:

- 承認段階
- 多段承認構造
- 意思決定体制

用途:

- 全体像把握
- ボトルネックの位置確認
- 部分最適の発見

3-2. Whole to Part

全体から構成要素へ降りる。

典型経路:

- Structure → has_part Element
- System → has_part Unit
- Process → has_part Step

例:

- 総力戦体制
- 動員装置
- 情報統制機構
- 正統化装置

用途:

- 分解
- 設計
- チェックリスト化
- 欠落要素の確認

3-3. Mechanism in Structure

Mechanism がどの構造で作動するかを辿る。

典型経路:

- Mechanism → operates_in Structure
- Structure → enables Mechanism
- Structure → constrains Action

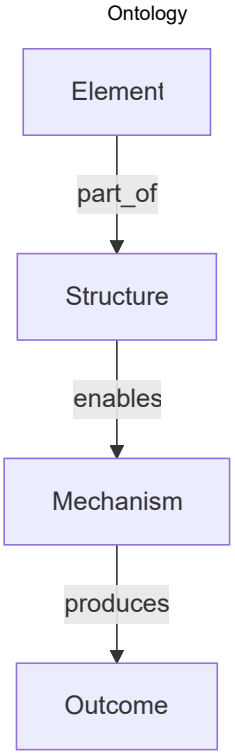
例:

- 責任分散
- 多段委任構造
- 承認責任の曖昧化

用途:

- 個人原因論の修正
- 組織問題の構造理解
- 再設計ポイントの特定

Structural Traversal の図



---

4. Lateral Traversal

Lateral Traversal は、
同じ抽象度の別ノードへ横移動する探索である。

これは発想の転用や、分野横断理解に不可欠。

4-1. Analogy Jump

類似構造を持つ別ノードへ飛ぶ。

典型経路:

- Pattern ↔ analogous_to Pattern
- Structure ↔ analogous_to Structure
- Case ↔ analogous_to Case

例:

- fandom 炎上
- 宗教的異端審問

- 規範逸脱者の可視化と排除

用途:

- 異分野比較
- 新しい見方の獲得
- 抽象化の確認

4-2. Contrast Jump

対比ノードへ飛ぶ。

典型経路:

- Concept ↔ contrasts_with Concept
- Pattern ↔ contrasts_with Pattern
- Strategy ↔ contrasts_with Strategy

例:

- 正統性 ↔ 強制
- 協力均衡 ↔ 裏切り均衡
- 分散型意思決定 ↔ 集中型意思決定

用途:

- 概念明確化
- 境界理解
- 誤認防止

4-3. Cross Domain Jump

別分野の適用先へ移動する。

典型経路:

- Mechanism → applies_to Domain
- Method → applies_to Domain
- Model → applies_to New Domain

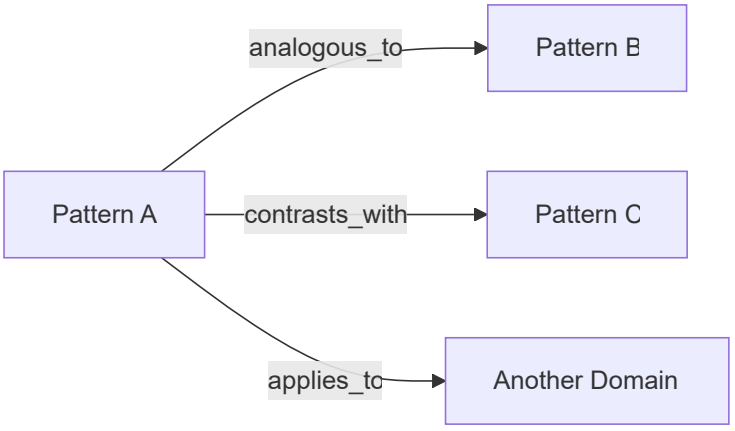
例:

- シグナリング
- 就活市場
- 観光レビュー市場
- 政治的支持形成

用途:

- 横断知
- 応用設計
- domain OS の構築

Lateral Traversal の図



---

5. Question Driven Traversal

Question Driven Traversal は、
問いを起点に必要なノード群へ到達する探索である。

これは LLM にとって特に重要で、
「何を答えるべきか」に応じて経路が変わる。

5-1. Question to Concept

問いの語彙や意味領域を特定する。

例:

- 植民地化とは何か
- 正統性とは何か
- 合理性とは何か

典型経路:

- Question → Concept
- Concept → contrasts_with Other Concept
- Concept → is_a Higher Concept

用途:

- 用語定義
- 概念整理
- 前提確認

5-2. Question to Pattern

問いが「よくある型か」を探る。

例:

- なぜまた炎上したのか
- なぜ組織が責任回避するのか

典型経路:

- Question → Pattern
- Pattern → Case Cluster
- Pattern → Mechanism

用途:

- 類例探索
- 再発理解
- 予測

5-3. Question to Mechanism

問いに対して「どう作用したか」を探る。

例:

- なぜ判断を誤ったのか
- なぜ集団が暴走したのか

典型経路:

- Question → Mechanism
- Mechanism → Kernel
- Mechanism → Structure

用途:

- 説明
- 介入点発見
- 問題解決

5-4. Question to Case

問いに対して具体例を探す。

例:

- その原理はどこに現れたか
- 類似事例は何か

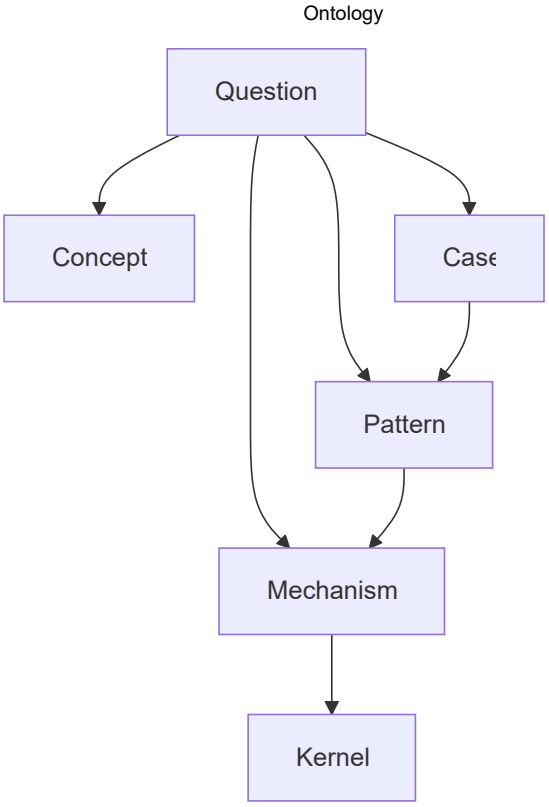
典型経路:

- Question → Case
- Case → Pattern
- Case → Actor / Event

用途:

- 例示
- 歴史比較
- ケースベース reasoning

Question Driven Traversal の図



---

6. Operational Traversal

Operational Traversal は、
知識を実務・分析・設計・記録へつなぐ探索である。

Graph が思想や理解に留まらず、
行動や作業に接続されるために必要。

6-1. Method to Domain

分析方法を適用先に接続する。

例:

- power mapping → 組織分析
- causal chain analysis → 事故分析
- comparative analysis → 歴史研究

用途:

- 適切な分析法選択
- method の使い分け

6-2. Tool to Task

Tool を具体作業へ接続する。

例:

- case template → 事例記述
- 観察シート → 現場記録
- 比較表 → 事例比較

用途:

- 即実行
- 作業の標準化
- LLM 出力テンプレ化

6-3. Framework to Workflow

Framework を実務プロセスへ接続する。

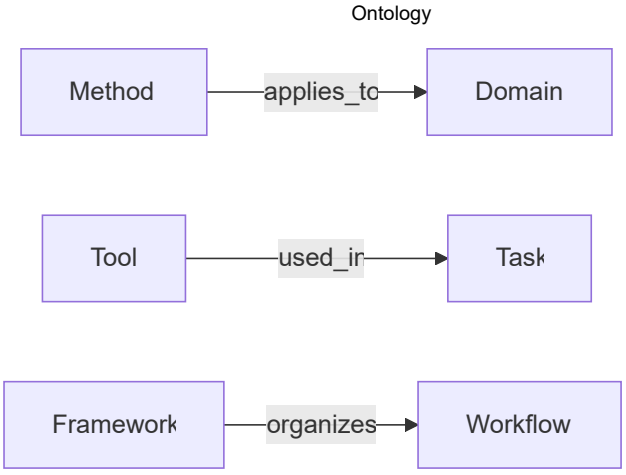
例:

- 観察→仮説→検証フレーム
- inquiry→reasoning→decision→solution design

用途:

- thinking engine 設計
- 業務OS構築
- step の可視化

Operational Traversal の図



---

7. Meta Traversal

Meta Traversal は、
Hub, Schema, Rule を使って探索自体を整理する探索である。

Graph が大きくなるほど、直接ノード探索だけでは不安定になる。
そのため中継点として Meta Node が必要になる。

7-1. Hub Guided Traversal

Hub を経由して中距離探索を行う。

例:

- Human Model Hub
- Social Pattern Hub
- Knowledge Graph Hub

用途:

- 初学者の導線
- 関連群の把握
- 重要ノードへの到達

7-2. Schema Guided Traversal

Node Type, Edge Type, Rule に従って探索する。

用途:

- ノート作成判断
- relation の選定
- graph 品質維持

7-3. Maintenance Traversal

重複・孤立・弱リンクを点検する探索。

用途:

- Duplicate node の統合
- 孤立 case の昇格
- outdated link の修正

停止条件

Traversal には停止条件が必要である。
止まらない探索は reasoning を壊す。

---

1. 十分な説明に達した

問いに対して、Kernel / Mechanism / Case が揃ったら一旦止める。

2. 新規情報が増えなくなった

同じようなノードばかり出てきたら打ち切る。

3. 問いから離れ始めた

類比や関連が広がりすぎ、問いとの距離が増えたら止める。

4. 必要な抽象度に達した

用語定義が目的なら Concept で止める。
介入設計が目的なら Mechanism / Structure まで行く。

5. 実務に必要な出力形式に届いた

Method / Tool / Framework まで接続できたら止める。

Traversal の典型パターン

1. 事例理解型

Case → Pattern → Mechanism → Kernel

2. 例示型

Concept → Pattern → Case

3. 診断型

Outcome → caused_by Mechanism → Structure → Constraint

4. 比較型

Case A → Pattern → analogous_to Pattern B → Case B

5. 定義型

Question → Concept → contrasts_with Related Concept

6. 実装型

Question → Method → Tool → Task

Traversal の典型テンプレート

1. Why 系

「なぜXなのか」

推奨経路:

- Question → Outcome
- Outcome → caused_by Mechanism
- Mechanism → operates_in Structure
- Mechanism → grounded_in Kernel

---

2. What is 系

「Xとは何か」

推奨経路:

- Question → Concept
- Concept → is_a Higher Concept
- Concept → contrasts_with Neighbor Concept
- Concept → Case Example

---

3. How 系

「どうやってXが起きるか」

推奨経路:

- Question → Mechanism
- Mechanism → Step / Process
- Mechanism → operates_in Structure
- Mechanism → produces Pattern

---

4. Where else 系

「それは他にどこで見られるか」

推奨経路:

- Concept / Pattern → analogous_to Other Pattern
- Pattern → applies_to Other Domain
- Pattern → Case Cluster

---

5. What to do 系

「どう対処するか」

推奨経路:

- Problem → caused_by Mechanism
- Mechanism → constrains / enables
- Method → applies_to Problem Domain
- Tool → used_in Intervention Task

Traversal の誤り

1. 近い単語だけで辿る

semantic similarity だけに頼ると、
概念近接はあっても reasoning として弱い。

2. 何でも上位概念へ上がる

抽象化は便利だが、具体を失うと空論になる。

3. 何でも類比で飛ぶ

analogous_to は強力だが、飛びすぎると問いから逸脱する。

4. 因果と説明を混同する

explains を causes 扱いすると誤診断が増える。

5. 停止条件がない

Graph 全体を歩こうとすると、結論が遅れ、精度も落ちる。

良い Traversal の条件

1. 問いに応じて経路が選ばれている

2. Edge Type に従って移動している

3. 抽象と具体の往復ができる

4. 不要な拡散を抑えている

5. reasoning path を再記述できる

LLM にとっての意味

Traversal が明確だと、LLM は

- 関連ノートを闇雲に集めるのでなく
- 問いに応じた探索順を持ち
- 抽象化と具体化を制御し

- 因果と類似を分けて辿り
- 説明経路を文章化しやすくなる

つまり Traversal は、
LLM に「何を参照するか」だけでなく
どう考えるかの順路 を与える。

この Vault における実装方針

本文や Hub に、Traversal の典型経路を明記しておくとよい。

記述例:

- この問いは **Question** → **Mechanism** → **Structure** → **Constraint** の順で辿る
- この case は **Case** → **Pattern** → **Mechanism** → **Kernel** で抽象化する
- この概念は **Concept** → **contrasts_with** → **Concept** → **Case** で輪郭化する

また、Hub に「読む順序」を書くことで、
Hub Guided Traversal が機能する。

他ノートとの接続

上位

- 02_zettelkasten/04_knowledge_graph/Knowledge Graph

近接

- 02_zettelkasten/04_knowledge_graph/Node Type
- 02_zettelkasten/04_knowledge_graph/Edge Type
- 02_zettelkasten/04_knowledge_graph/Graph Maintenance
- 02_zettelkasten/04_knowledge_graph/Reasoning Path
- 02_zettelkasten/04_knowledge_graph/Hub Design Rule

下位候補

- Abstract Upward Traversal
- Concrete Downward Traversal
- Causal Traversal
- Question Driven Traversal
- Operational Traversal

まとめ

Traversal は、Knowledge Graph において
ノード間を**意味ある順序で辿るための探索構造**である。

Traversal が整うことで、

- Graph が歩けるようになる
- 抽象と具体を往復できる
- 問いに応じた探索ができる
- reasoning path を再現できる
- LLM の retrieval と explanation が安定する

Knowledge Graph が「知識の地図」だとすれば、
Traversal は「その地図の歩き方」である。

Concept Graph Hub

Knowledge Graph

Knowledge Graph は、ノート群・概念群・事例群・問い群のあいだにある**関係そのもの**を明示し、
LLM や人間が「何が何とどうつながっているか」を辿れるようにするための構造である。

単なるリンク集ではなく、以下を担う。

- 概念間の意味関係の保持
- 抽象と具体の往復
- 因果・構造・所属・類似の区別
- 問いから関連ノート群への探索
- case から pattern / mechanism / kernel への昇格
- reasoning のための探索経路の提供

定義

Knowledge Graph とは、
Node（対象） と **Edge（関係）** を用いて、知識体系をネットワークとして表現する構造である。

ここで重要なのは、
「ノートがあること」ではなく、
ノート同士の関係が型付きで保持されていることである。

つまり Knowledge Graph は、

- ノートの倉庫
- タグの束
- 単純な双方向リンク集

ではなく、

- どのノートが
- どの種類の関係で
- どの方向に
- どの粒度で
- どの推論に使えるか

を表現する知識地図である。

この構造が必要な理由

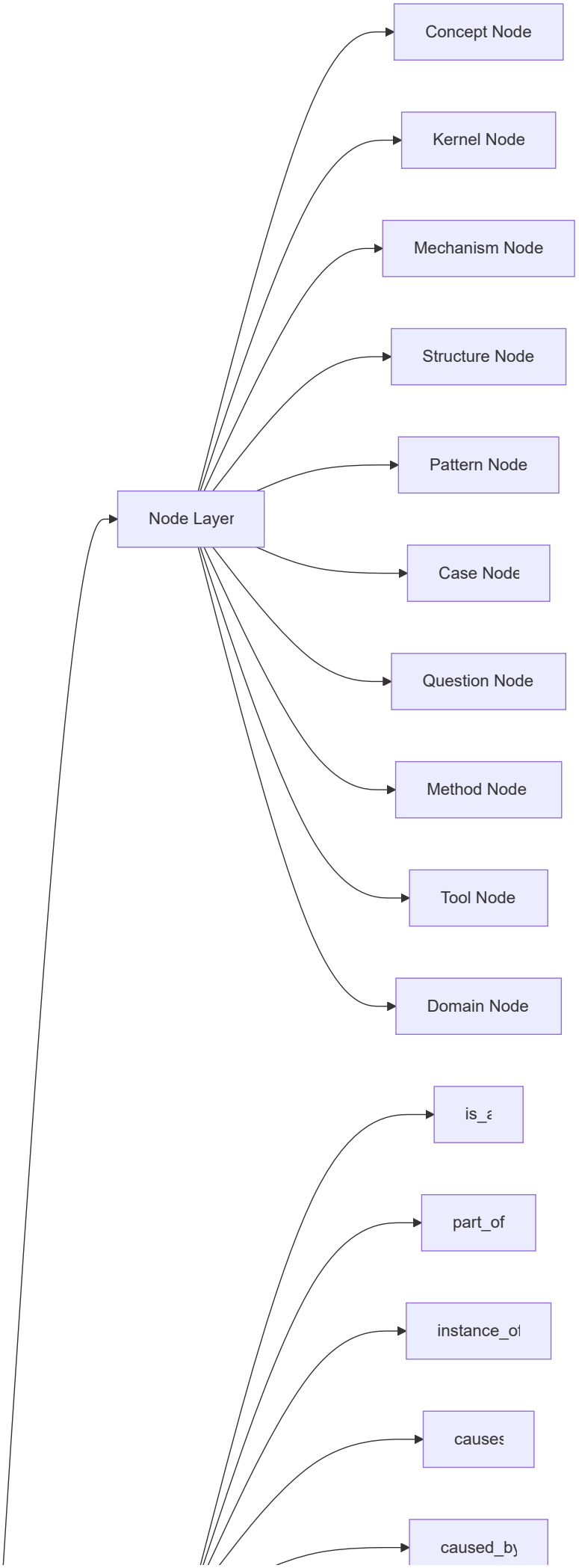
Zettelkasten を大きくすると、問題は「ノート不足」ではなく**接続不足**になる。

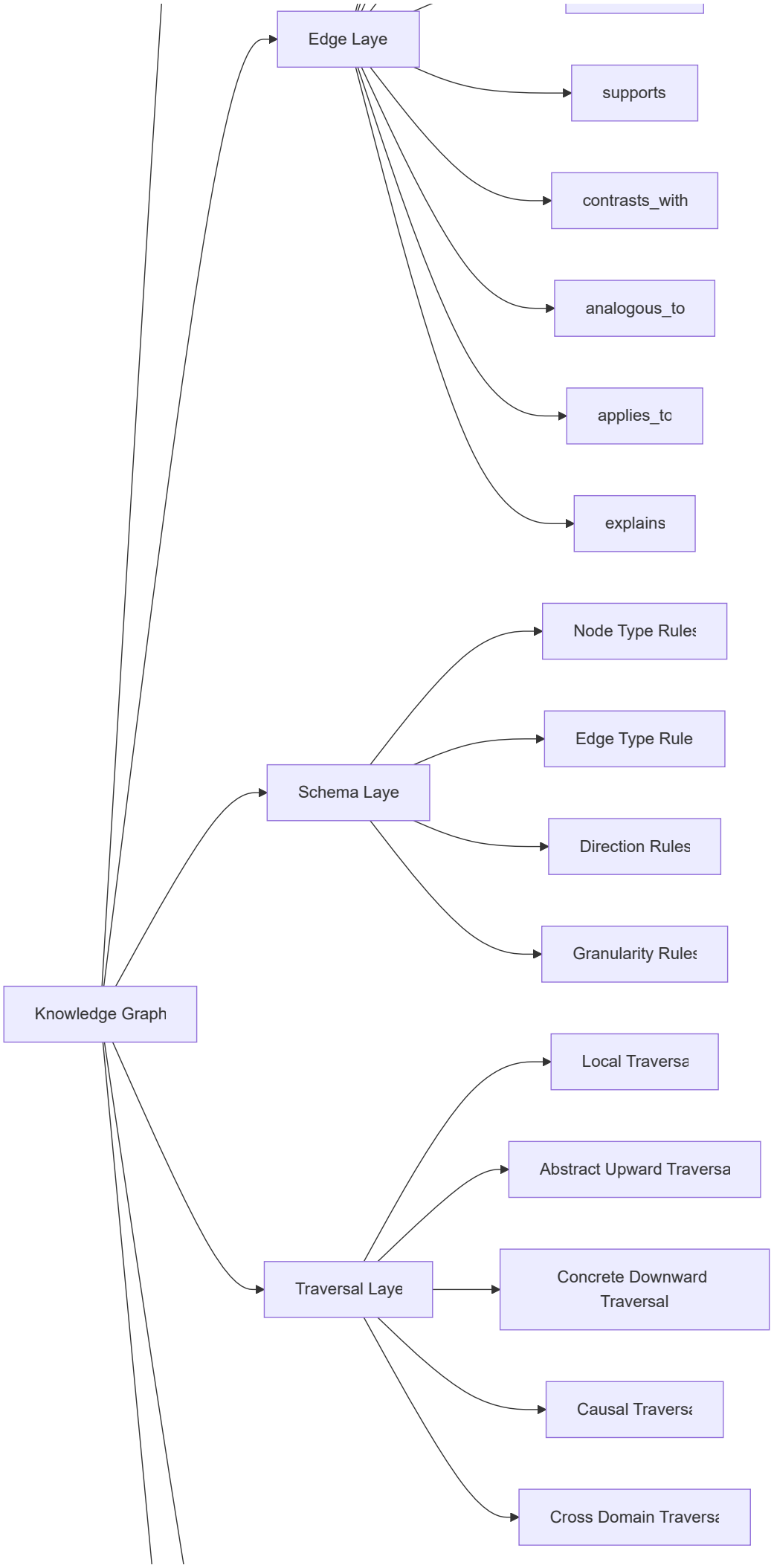
典型的には次のような破綻が起こる。

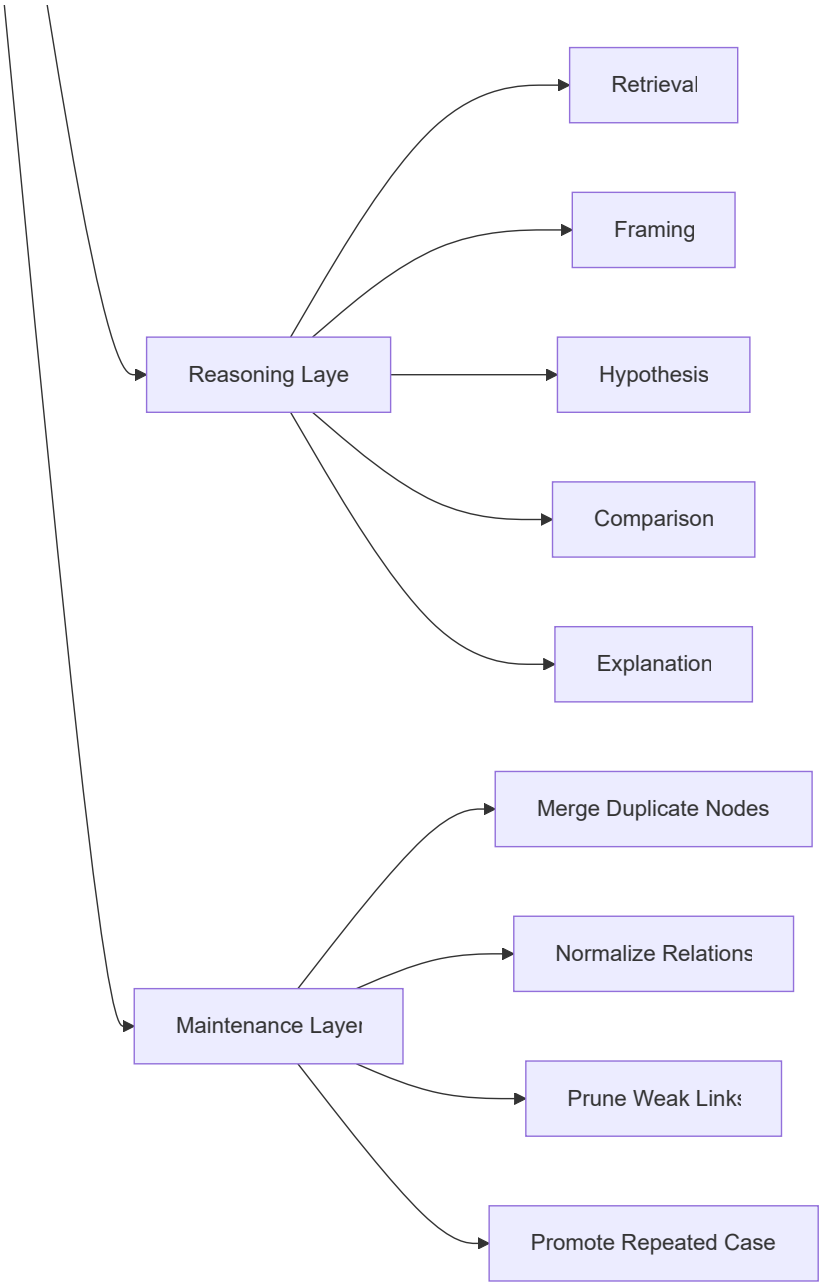
1. ノートはあるが関連が見えない
2. case が pattern に接続されない
3. concept が domain ごとに孤立する
4. 同じ内容が別名で重複する
5. LLM が近いが違うノートを拾って誤推論する
6. 逆に重要な橋渡しノートが埋もれる

Knowledge Graph はこれを防ぎ、
Vault 全体を**検索可能な構造体**へ変える。

全体構造


Node Layer
Concept Node
Kernel Node
Mechanism Node
Structure Node
Pattern Node
Case Node
Question Node
Method Node
Tool Node
Domain Node
is_a
part_of
instance_of
causes
caused_by


Edge Laye
supports
contrasts_with
analogous_to
applies_to
explains
Node Type Rules
Edge Type Rule
Schema Laye
Direction Rules
Granularity Rules
Knowledge Graph
Local Traversa
Abstract Upward Traversa
Traversal Laye
Concrete Downward Traversal
Causal Traversa
Cross Domain Traversa


Reasoning Laye
Retrieval
Framing
Hypothesis
Comparison
Explanation
Maintenance Layer
Merge Duplicate Nodes
Normalize Relations
Prune Weak Links
Promote Repeated Case

基本発想

Knowledge Graph の中核は、 **知識を点ではなく経路として扱う**ことである。

たとえば「韓国併合」は単独で理解するのでなく、

- 韓国併合
- 帝国主義
- 非対称権力構造
- 正統化言説
- 植民地化パターン
- 国家形成
- 国際秩序
- 支配の制度化

という経路で辿られる。

このとき価値があるのは個別ノートの記述量だけでなく、
どの順で辿れるかである。

Node の種類

1. Concept Node

最も一般的な意味単位。
まだ十分に構造化されていないが、重要な概念。

例:

- 正統性

- 動員
- 抑圧
- アイデンティティ
- 協調
- 制約

役割:

- 語彙の入口
- 他ノードへの橋
- 概念の曖昧さを管理する

---

2. Kernel Node

より深い原理・恒常性・世界理解の土台。

例:

- 資源制約
- 限定合理性
- 権力偏在
- 情報非対称
- 制度は行動を方向づける

役割:

- 多数の現象の背後にある根本原理
- pattern や mechanism の基礎

---

3. Mechanism Node

何がどう作用して結果を生むかの作動単位。

例:

- シグナリング
- フリーライダー化
- 同調形成
- 注意資源の偏り
- インセンティブ歪み

役割:

- 因果の説明
- 動態の記述
- reasoning の中心

---

4. Structure Node

要素の配置・接続・階層・循環の形。

例:

- 主従構造
- 中心周辺構造
- 寡占構造
- フィードバックループ
- 委任連鎖

役割:

- 静的な骨格の把握
- どこにボトルネックがあるかの可視化

---

5. Pattern Node

反復して現れる典型形。

例:

- 炎上パターン
- 責任回避パターン
- 外部敵形成パターン
- 制度疲労パターン

役割:

- case を束ねる
- 予測のたたき台になる

---

6. Case Node

具体的事例・歴史事象・観察記録。

例:

- 韓国併合
- ドイツ革命
- 具体的な炎上事例
- ある企業の失敗案件

役割:

- 現実の接地面
- 抽象化の素材
- pattern / mechanism への入力

---

7. Question Node

問いそのものをノード化したもの。

例:

- なぜ人は不合理な選択を続けるのか
- なぜ制度は自己目的化するのか
- なぜ同じ失敗が反復するのか

役割:

- 探索の起点
- reasoning の入口
- 関連ノート群の束ね直し

---

8. Method Node

分析・観察・設計の方法。

例:

- causal chain analysis
- five whys
- power mapping
- comparative analysis

役割:

- 問いと知識をつなぐ操作手順

---

9. Tool Node

実務上のテンプレート・フレーム・チェックリスト・プロンプトなど。

例:

- case 分析テンプレート
- 観察シート
- 比較表
- 問題分解フォーマット

役割:

- Graph を実務に接続する

---

10. Domain Node

知識の適用領域・分野。

例:

- history
- law
- business
- tourism
- psychology

役割:

- 横断知を分野に落とす
- 検索や整理の起点にする

Edge の種類

Knowledge Graph では、リンクはすべて同じではない。
関係の型を分けることが重要である。

1. is_a

分類関係。
「A は B の一種である」

例:

- 植民地化 is_a 支配形態
- 同調 is_a 社会的影響

用途:

- 概念整理
- 上位概念への遡及

---

2. part_of

部分関係。
「A は B の一部である」

例:

- 動員装置 part_of 総力戦体制
- 問題定義 part_of problem solving

用途:

- 全体構造の把握
- 要素分解

---

3. instance_of

具体例関係。
「A は B の事例である」

例:

- 韓国併合 instance_of 植民地化パターン
- ある炎上事件 instance_of 規範違反可視化パターン

用途:

- case から pattern への接続
- 抽象化の足場

---

4. causes

原因関係。
「A が B を引き起こす」

例:

- 情報非対称 causes 逆選択
- 注意資源不足 causes 判断短絡

用途:

- 因果推論
- 問題解決
- 仮説生成

---

5. caused_by

被原因関係。
「A は B によって生じる」

例:

- 制度疲労 caused_by 過剰複雑化
- 不信拡大 caused_by 裏切り経験

用途:

- 原因探索
- 逆向き推論

---

6. supports

支援・根拠関係。
「A は B を支える」

例:

- 統計データ supports 仮説
- 事例群 supports パターン認定

用途:

- エビデンス接続
- 議論補強

---

7. contrasts_with

対比関係。
「A は B と対照的である」

例:

- 正統性 contrasts_with 強制
- 市場調整 contrasts_with 計画統制

用途:

- 概念の輪郭化
- 誤同一視の防止

---

8. analogous_to

類比関係。
「A は B に似た構造を持つ」

例:

- 企業官僚制 analogous_to 国家官僚制
- fandom 炎上 analogous_to 宗教的異端審問

用途:

- 分野横断
- 発想拡張

---

9. applies_to

適用関係。
「A は B 領域で使える」

例:

- principal-agent problem applies_to 組織運営
- シグナリング applies_to 就活市場

用途:

- 実用化
- domain への橋渡し

---

10. explains

説明関係。
「A は B を説明する」

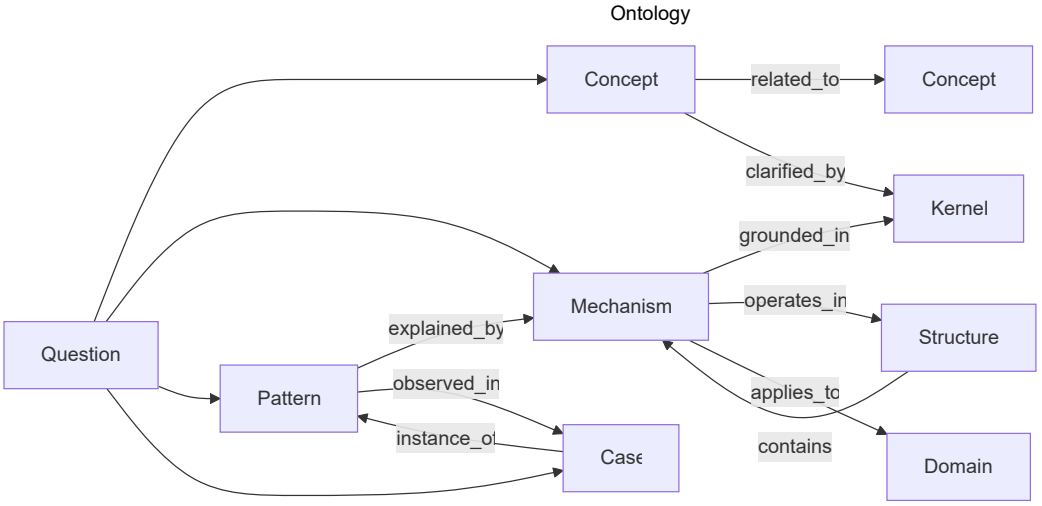
例:

- 限定合理性 explains 不十分な意思決定
- アイデンティティ防衛 explains 認知的不協和の回避

用途:

- reasoning 文生成
- 説明構造の構築

ノード間の典型接続



推論における役割

Knowledge Graph は reasoning engine の下請けではない。
むしろ reasoning の**前提インフラ**である。

1. Retrieval

問いに対し、単語一致ではなく**関係近傍**で関連ノートを拾う。

例:

- 「韓国併合は植民地化か」
  - 韓国併合
  - 植民地化
  - 帝国主義
  - 非対称支配
  - 正統化言説
  - 国家主権侵食
  - 同化政策

というように近傍探索できる。

---

2. Framing

何の問題として見るべきかを与える。

例:

- 倫理問題
- 権力問題
- 制度問題
- アイデンティティ問題
- 市場シグナル問題

Graph があると、問いを別フレームで再配置できる。

---

3. Hypothesis

未接続のノード間に仮説的エッジを引ける。

例:

- ある組織の意思決定遅延
- 既存ノード: サイロ化 / 責任分散 / 情報非対称 / 承認多段化

ここから

- 承認多段化 causes 意思決定遅延
- 責任分散 supports 先送り行動

などの仮説を出せる。

---

4. Comparison

異なる case の共通構造を見つける。

例:

- ドイツ革命
- ソ連崩壊
- ベンチャー企業の崩壊

一見違っても、

- 正統性低下
- 強制力の揺らぎ
- エリート分裂
- 代替秩序の浮上

という pattern で比較できる。

5. Explanation

説明文を「関係に沿って」生成できる。

悪い説明:

- A は複雑です
- 背景があります
- 様々な要因があります

良い説明:

- A は B による圧力の下で C が作動し、その結果 D が制度化された

Knowledge Graph は後者を可能にする。

探索方向

Knowledge Graph では、探索方向が重要である。

1. 上向き探索（抽象化）

case → pattern → mechanism → kernel

用途:

- この出来事は何の一例か
- 背後で何が働いたか
- より普遍的には何か

---

2. 下向き探索（具体化）

kernel → mechanism → pattern → case

用途:

- この原理はどこで現れるか
- 抽象概念を具体例で理解したい
- 教材や事例が欲しい

---

3. 横断探索（類比）

domain A → analogous_to → domain B

用途:

- 歴史を現代組織で読む
- 心理を営業で使う
- 法構造を制度設計へ応用する

---

4. 因果探索

result → caused_by → cause → caused_by ...

用途:

- 根本原因分析
- 問題解決
- five whys の補強

---

5. 構造探索

phenomenon → operates_in → structure

用途:

- 現象がどの配置で起きやすいかを見る
- 個人原因論への過剰還元を防ぐ

Graph における重要単位

Knowledge Graph では、特に次のノードが重要になる。

1. Hub Node

多くのノードを束ねる中心ノード。

例:

- 人間モデル Hub
- 社会構造 Hub

- Problem Solving Hub
- Knowledge Graph Hub

条件:

- 単なる目次ではなく
- 何がどうつながるかを示すこと

---

2. Bridge Node

異分野を接続する橋。

例:

- シグナリング
- 正統性
- 制約
- 委任
- アイデンティティ

こうしたノードは、

- psychology
- business
- history
- law
  を跨ぐ。

---

3. Bottleneck Node

多くの推論が集中するが未整備なノード。

例:

- 権力
- 制度
- 合理性
- 価値
- 正義

ここが曖昧だと Graph 全体が濁る。

4. Anchor Case

抽象概念の足場になる代表事例。

例:

- 韓国併合
- ドイツ革命
- 具体的炎上事例
- 就活市場

これがあると抽象が浮かない。

Knowledge Graph の作り方

Step 1: Node を置く

まず重要概念・事例・方法をノード化する。

原則:

- 1ノート1中心概念
- 曖昧なら concept として置く
- 具体事例は case に置く
- 作動説明できるなら mechanism 化する

Step 2: Edge を型付きで張る

単に `[[リンク]]` するのでなく、
本文かプロパティで関係型を明記する。

例:

- 韓国併合 instance_of 植民地化パターン
- 植民地化パターン explained_by 非対称権力構造
- 非対称権力構造 grounded_in 権力偏在原理

Step 3: Hub で束ねる

ノードが増えたら hub を作る。

Hub の役割:

- 代表ノード一覧
- 関係の見取り図
- 読む順序
- 重要経路

Step 4: 重複を正規化する

例:

- 支配
- domination
- 統制
- 管理

が混在していたら、
代表語を決めて他を寄せる。

Step 5: 繰り返し出る case を昇格させる

具体例が3回以上別 case に現れるなら、
pattern または mechanism への昇格を検討する。

良い Graph の条件

1. 関係型が明確

単なる「関連する」では弱い。
is_a / instance_of / causes などに分ける。

2. 抽象と具体が往復可能

上に上がれるだけでなく、下に降りられること。

3. 分野横断ができる

同じ mechanism が複数 domain に現れることが見える。

4. 誤接続が少ない

似ているが違う概念が混線していない。

5. 推論経路が辿れる

「なぜその結論になったか」が path として説明できる。

悪い Graph の典型

1. タグの言い換えに終わる

単に #history #power #society と付いているだけ。

2. 全リンクが同じ重み

原因も所属も類似も同列に扱っている。

3. case だらけで抽象がない

具体例の山になり、一般化できない。

4. 抽象だらけで事例がない

言葉は立派だが現実に接続しない。

5. hub が目次でしかない

一覧はあるが関係が示されていない。

Obsidian / Vault 運用上の実装方針

1. ノート単位でグラフ化する

1ノート1中心概念を守る。
複数概念を詰め込むと Graph が曖昧になる。

2. リンク本文で関係型を書く

たとえば本文中で次のように書く。

- 韓国併合 は 植民地化パターン の **instance_of**

- 02_zettelkasten/02_world_model/meta/model/social/information/情報非対称 は 02_zettelkasten/02_world_model/model/social/incentive/逆選択 を **causes**
- 02_zettelkasten/02_world_model/meta/model/human/congnition/限定合理性 は 不完全意思決定 を **explains**

3. Hub で中距離リンクを束ねる

全文検索だけに頼らず、hub で読み筋を作る。

4. property は最小限

Graph を支える主役は本文中の関係記述。
property は補助。

5. 関係語を統一する

relation 語彙を固定する。

推奨語彙:

- is_a
- part_of
- instance_of
- causes
- caused_by
- supports
- contrasts_with
- analogous_to
- applies_to
- explains

LLM にとっての意味

Knowledge Graph がない場合、LLM は

- 単語近傍
- 頻出共起
- 一般知識
- 表層的類似

に頼りやすい。

Knowledge Graph がある場合、LLM は

- どの概念が上位か
- どの case が何の例か
- どの mechanism が背後にあるか
- 何を経由して推論すべきか

を辿りやすくなる。

つまり Graph は、
LLM に「情報」ではなく**思考経路**を与える。

この Vault における位置づけ

Knowledge Graph は単独フォルダというより、Vault 全体を横断する**メタ構造**である。
ただし運用上は次の位置づけがよい。

- `zettelkasten/structure/knowledge/Knowledge Graph.md
- または
- `thinking_engine/reasoning/Knowledge Graph.md`

役割としては、

- observation が入力を取り込み
- concept / case / pattern / mechanism / kernel が内容を保持し
- knowledge graph がそれらを接続し
- reasoning が接続を辿って推論し
- decision / solution_design が出力する

という流れになる。

上位・下位関係

上位

- 02_zettelkasten/03_thinking_engine/Thinking Engine
- Reasoning Structure
- Knowledge Structure

近接

- Concept
- Case

- 02_zettelkasten/04_knowledge_graph/Pattern
- Mechanism
- Kernel
- Structure
- Question
- Hub Note

下位

- 02_zettelkasten/04_knowledge_graph/Node Type
- 02_zettelkasten/04_knowledge_graph/Edge Type
- 02_zettelkasten/04_knowledge_graph/Traversal
- 02_zettelkasten/04_knowledge_graph/Graph Maintenance
- 02_zettelkasten/04_knowledge_graph/Bridge Concept
- 02_zettelkasten/04_knowledge_graph/Anchor Case

典型ユースケース

1. 問いから探索する

「なぜ組織は責任回避するのか」
→ 責任回避
→ 委任連鎖
→ principal-agent problem
→ 情報非対称
→ インセンティブ歪み

2. 事例を抽象化する

「ドイツ革命」
→ 正統性崩壊
→ エリート分裂
→ 命令系統の揺らぎ
→ 体制転換パターン

3. 別分野へ応用する

「同調」
→ 社会心理
→ 消費行動
→ 組織文化
→ SNS 炎上

4. 誤認をほどく

「合理性」と「最適性」は同じか
→ 概念対比
→ 限定合理性
→ satisficing
→ 情報制約

最小実装ルール

Knowledge Graph を回し始めるだけなら、最低限これでよい。

1. ノート種別を分ける
   - concept / case / pattern / mechanism / kernel
2. 関係語を固定する
   - is_a / instance_of / causes / explains / part_of
3. hub を作る
   - 分野ごとに1つ
   - 重要概念ごとに1つ
4. case から必ず1つ上に上げる
   - その事例は何の例かを書く
5. mechanism か structure を必ず経由する
   - いきなり感想で終わらせない

まとめ

Knowledge Graph とは、
Vault 内の知識を**関係つきネットワーク**として運用するための中核構造である。

それは、

- ノートをつなぎ
- 抽象と具体を往復させ
- 因果と構造を区別し
- 分野横断を可能にし
- LLM に推論経路を与える

ためのものである。

Zettelkasten が「知識の蓄積」だとすれば、
Knowledge Graph は「知識の可動化」である。